

CINEVERSE

Technical documentation

Technologies

Vue 3 · TypeScript · PHP · MySQL 8 · TMDB API

Project technical document

Author: Gastón Rizzo

Table of Contents

1. CineVerse overview	3
2. Technologies and tools used	4
3. TMDB: what it is, how to obtain access, and how CineVerse uses it	6
4. How CineVerse is organized	14
5. Project structure	16
6. Configuration files and environment variables	19
7. MySQL 8 database	29
8. Test data and SQL file	32
9. How a user action flows through CineVerse	33
10. TypeScript in CineVerse	35
11. How the frontend is organized with Vue	38
12. How data is stored and reused in the frontend	41
13. Navigation without reloading the page: SPA and Vue Router	48
14. Languages: vue-i18n and language switching	53
15. Communication between the frontend, TMDB, and the backend	58
16. Movie explorer and search system	68
17. How filters are saved and reflected in the URL	77
18. How more movies are loaded	78
19. Home page	79
20. Movie details	82
21. Trailers, similar movies, and images	83
22. Actors and filmography	85
23. Favorites	87
24. How the PHP backend is organized	89
25. Authentication and account	98
26. Reviews and profile	110
27. Validation and security	117
28. Interface states and error handling	119
29. Logs: error logging and diagnosis	132
30. Local CineVerse installation	138
31. Running CineVerse in development	143
32. Publishing CineVerse on a server	150
33. Project testing and verification	153

1. CineVerse overview

CineVerse is a web application designed to provide information about movies and actors. It lets users search for movies, apply filters, view detailed information about each title, see its cast and director, watch trailers, and access actor information and filmographies.

In addition to browsing information, users can save movies as favorites. They can also create an account to post reviews, edit or delete their own reviews, and view their activity from their profile. Movie and actor information comes from TMDB, while CineVerse-specific data, such as users, login attempts, and reviews, is stored in MySQL 8.

Core idea

CineVerse does not store a complete copy of the movies and actors it displays in MySQL. That information is queried from TMDB when needed. MySQL is used for data that belongs to CineVerse: user accounts, email verification, password recovery, login attempts, and reviews.

1.1 What the user can do

- Explore trending, now playing, upcoming, popular, and top-rated movies.
- Search by text and combine the search with genre, year, rating, and sorting options.
- View a movie's details, cast, director, trailer, and similar movies.
- Open actor profiles and browse their filmographies.
- Save up to 20 favorites in the browser without signing in.
- Register, verify an email address, sign in/out, and recover a password.
- Create one review per movie, edit it, delete it, and view profile activity.

1.2 How it works at a high level

CineVerse has one part that runs in the browser and another that runs on the server. The visible part, called the frontend, is built with Vue 3 and TypeScript. It displays the screens, forms, filters, movie cards, and navigation.

The backend is the PHP code that runs on the server. It handles features that require CineVerse's own data: registration, sign-in and sign-out, email verification, password recovery, profiles, and reviews. When it needs to store or read that data, it communicates with MySQL.

An API is a way for programs to communicate with one another. CineVerse uses two: TMDB's external API to obtain movie and actor information, and its own PHP API to communicate with the server and work with accounts and reviews. The frontend sends requests to one or the other depending on the information it needs.

The frontend is built as an SPA. SPA stands for Single Page Application. The name does not mean that CineVerse has only one screen: it means that the browser loads the application initially and, when moving for example from Home to Movies, to a movie detail page, or to an actor page, Vue changes the content without downloading an entirely new HTML page. Vue Router keeps the URL synchronized with the section being visited.

1.3 Basic concepts for reading this documentation

Before continuing, it is useful to establish a minimal vocabulary. These terms appear frequently throughout the project's workflows; they are explained briefly here, while later sections show how they work in greater depth.

Term	What it means in CineVerse
API	A way for programs to communicate with one another. CineVerse uses TMDB's external API and its own PHP API.
Endpoint	A specific route within an API that represents an operation or resource. Example: <code>/movie/550/videos</code> requests the videos for movie 550.

Term	What it means in CineVerse
Request	An HTTP message that the frontend sends to an API to request information or submit data.
Response	The message the API returns after processing a request; in CineVerse, it normally contains JSON.
JSON	A structured text format made up of properties and values that CineVerse uses to exchange data with APIs.
Query parameter	A value included in the URL after ?. Example: page=2&language=es.
View	A complete Vue screen normally associated with a route, for example MovieDetailView.vue.
Component	A reusable part of a screen, for example MovieHero.vue or MovieCard.vue.
Composable	A Vue function that groups reusable reactive logic. It can coordinate state, Vue Query, and services, but it does not render a screen by itself.
Frontend service	A TypeScript module that centralizes data-access operations and knows which endpoint to use, for example movies.service.ts.
Store	State shared across different parts of the frontend, such as the authenticated user or favorites.

Do not confuse the two uses of “service.” A *.service.ts file belongs to the frontend and performs data-access operations over HTTP. A *Service.php class belongs to the backend and applies business rules. They share the term service, but they live in different layers.

```

Carlos opens a movie
-> MovieDetailView.vue      [View]
-> useMovieDetail()         [Composable]
-> getMovieVideos()         [Service operation]
-> movies.service.ts        [frontend Service]
-> GET /movie/550/videos    [Endpoint]
-> TMDB                    [API]
-> JSON                    [Response]
-> Vue displays the result

```

2. Technologies and tools used

CineVerse uses several technologies because each part of the project has different requirements. Vue builds the interface seen by the user; TypeScript helps control the shape of the data; PHP implements the application's own API; MySQL stores account and review information; and TMDB provides movie and actor information. The tools in this section are explained in terms of their actual role within the project.

Technology	Actual use in CineVerse
Vue 3	Builds the interface that runs in the browser. It allows the screen to be divided into components and only the parts that change to be updated.
TypeScript 5.9	Adds types to JavaScript. CineVerse uses it to describe, for example, the shape of a movie, a TMDB response, a user, or data sent to the backend.
Vite 7	Starts the frontend development server and generates the optimized files published in production.
Vue Router	Maps each URL to a Vue screen and enables navigation within the SPA without reloading a complete HTML document.

Technology	Actual use in CineVerse
Pinia	Stores information that must be available from different parts of the frontend, such as the authenticated user and favorites.
TanStack Vue Query	Organizes data requests without blocking the interface, reuses temporary results, and manages loading, error, and pagination states.
Axios	Sends HTTP requests, meaning messages that the frontend sends to an API to request or submit information. CineVerse creates one client for TMDB and another for its own backend.
vue-i18n	Manages interface text in Spanish and English.
Tailwind CSS 4	It is installed and connected to the Vite process, but it is not the main styling system. The CineVerse interface is built primarily with custom CSS and styles defined within Vue components.
PHP	Implements CineVerse's own API. Laravel or any other framework is not used.
PDO	The PHP interface used to connect to MySQL. It allows prepared SQL statements to be executed, separating the statement itself from the values received.
MySQL 8	Stores users, reviews, and login attempts and applies rules to prevent invalid data or inconsistent relationships.
TMDB API v3	External service from which movie and actor information is obtained: titles, synopses, genres, cast, credits, videos, and images.
Monolog	Writes backend technical errors that require diagnosis to log files.
PHPMailer	Sends account verification and password-recovery emails through an SMTP mail server.

2.1 Relevant interface libraries

Package	Responsibility
pinia-plugin-persistedstate	Extends Pinia so the favorites store can be kept in localStorage and restored after the browser is reloaded.
swiper	Provides the carousel engine used on the Home page. CineVerse combines SwiperSlide, navigation, autoplay, and transitions with its own loading and visual-state logic.
vue-multiselect	Provides enhanced selectors for filters, especially when the user needs to choose options such as genres.
vue-slider-component	Provides the sliders used to select year and rating ranges without building those controls from scratch.
vuedraggable	Allows the filter-system panels to be reordered by drag and drop; the new order can be persisted in the browser.

Package	Responsibility
lucide-vue-next	Provides icons as reusable Vue components inside buttons, notices, and interface controls.

These dependencies do not all serve the same purpose. Vue Router, Pinia, TanStack Vue Query, and vue-i18n integrate with the application as cross-cutting infrastructure; pinia-plugin-persistedstate extends a Pinia store; Axios handles HTTP requests; and Swiper, vue-multiselect, vue-slider-component, vuedraggable, and lucide-vue-next address specific interface needs. Section 12 explains in depth how each one is used within CineVerse.

The current installation also includes @vueuse/core and @vueuse/motion, but in the reviewed code version they are not imported anywhere inside src. For that reason, they are not presented as functional parts of CineVerse. Likewise, Tailwind CSS is connected to Vite, but the interface is implemented mainly with custom CSS and component styles; it is not the project's primary styling system.

2.2 Environment variables

CineVerse needs certain configuration values that should not be written directly into the code. For example, the frontend needs to know where the TMDb API is and how to reach the backend; the backend needs the MySQL and mail-server settings. These values are stored in .env files. Section 6 explains where those files are located, what each one contains, and how Vite and PHP read them.

3. TMDb: what it is, how to obtain access, and how CineVerse uses it

CineVerse depends on The Movie Database for audiovisual information, but TMDb is not part of the project's code and is not installed as a library. It is an external Internet service. This section starts from that point and follows the complete flow: what TMDb is, how a developer obtains access, which credential CineVerse uses, which files build the requests, and how the JSON response ultimately becomes a Vue interface.

3.1 What The Movie Database (TMDb) is

TMDb stands for The Movie Database. Its API allows other applications to automatically query information about movies, TV series, and people through HTTP requests, without requiring someone to visit the TMDb website and search for that data manually. CineVerse uses API v3 to request the information it needs for each screen from TMDb, while MySQL stores only CineVerse's own data, such as accounts, login attempts, and reviews.

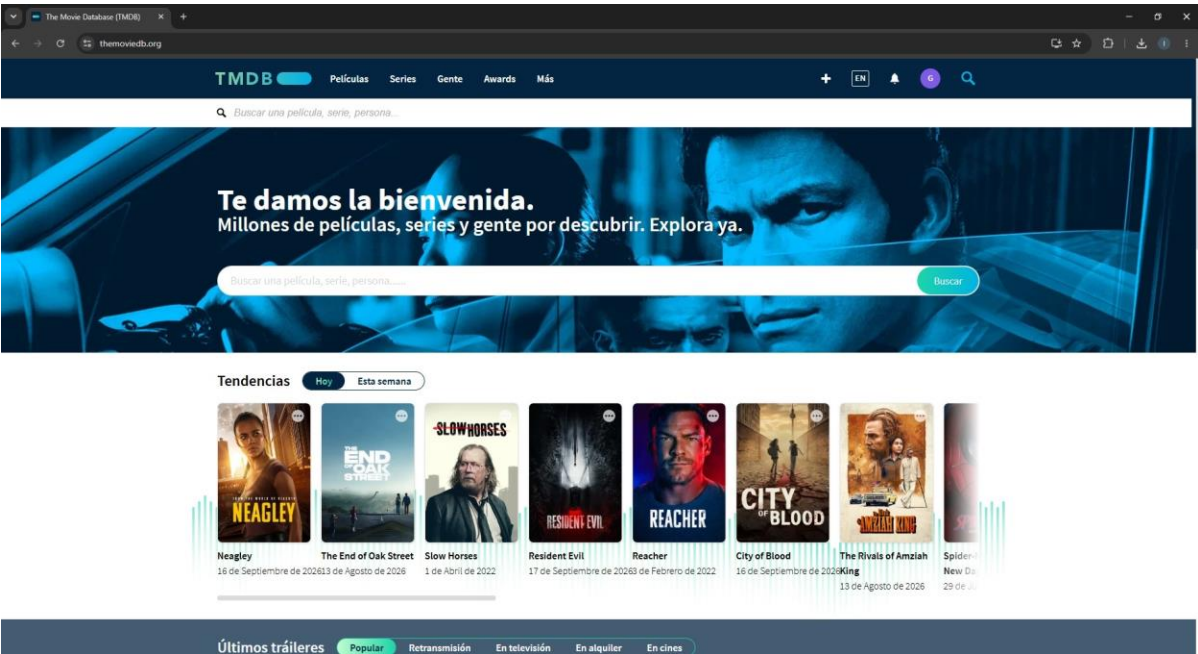


Figure. Public website of The Movie Database (TMDB). CineVerse does not copy this page: it programmatically queries the API that TMDB provides to developers.

It is useful to distinguish three related addresses. `www.themoviedb.org` is the website a person visits; `api.themoviedb.org` is the API to which CineVerse sends HTTP requests; and `image.tmbd.org` is the infrastructure used to serve posters, backdrops, and other images. They are parts of the same ecosystem, but they serve different purposes.

Address	Function in CineVerse
<code>www.themoviedb.org</code>	TMDB public website and developer-account configuration.
<code>api.themoviedb.org/3</code>	API v3: receives requests and returns data in JSON.
<code>image.tmbd.org</code>	Server/CDN from which the browser downloads images.

3.2 What information CineVerse obtains from TMDB

TMDB provides the external catalog: titles, synopses, dates, genres, ratings, posters, backdrops, cast, crew, videos, similar movies, and person data. CineVerse selects only what it needs. For example, a visual movie page can combine the movie details with credits and videos obtained through different endpoints.

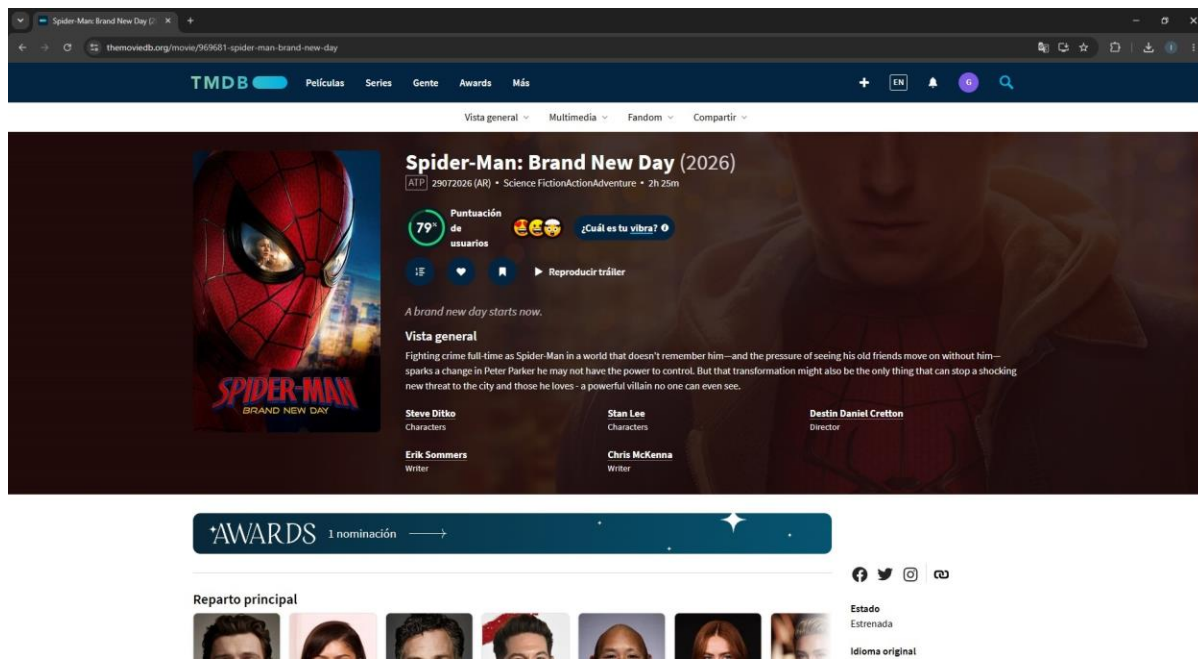


Figure. Example of a movie page on TMDB's public website. The API exposes the data that CineVerse uses to build its own interface; CineVerse does not copy this HTML page.

The separation from MySQL is intentional: CineVerse does not maintain a movies table containing a complete copy of the catalog. A local review stores only movie_id and movie_title as enough of a reference to associate the user's activity with a TMDB movie.

3.3 How a developer obtains access

API access is managed through a TMDB account. According to the official documentation, the developer signs in, opens Account Settings, and goes to the API section. Before receiving the credentials, the developer must accept the applicable terms of use.

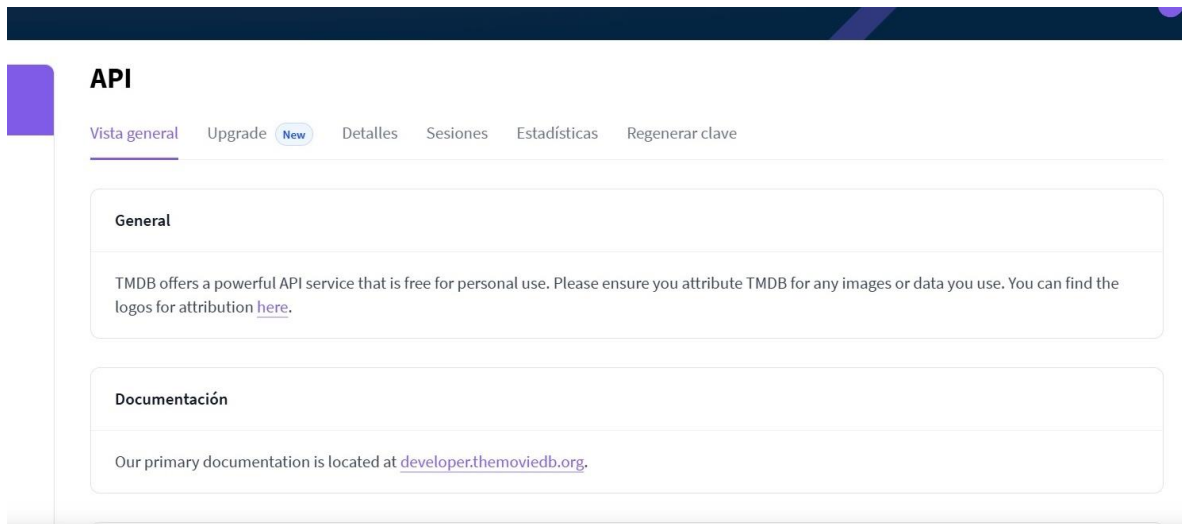


Figure. API section of a TMDB account's settings. Developer access is managed here, and the official documentation can be accessed from this section.

This step belongs to environment setup, not to Vue execution. Once the credential has been obtained, it is copied to frontend/.env, and Vite exposes it to the frontend code through import.meta.env.

3.4 API Read Access Token and API Key

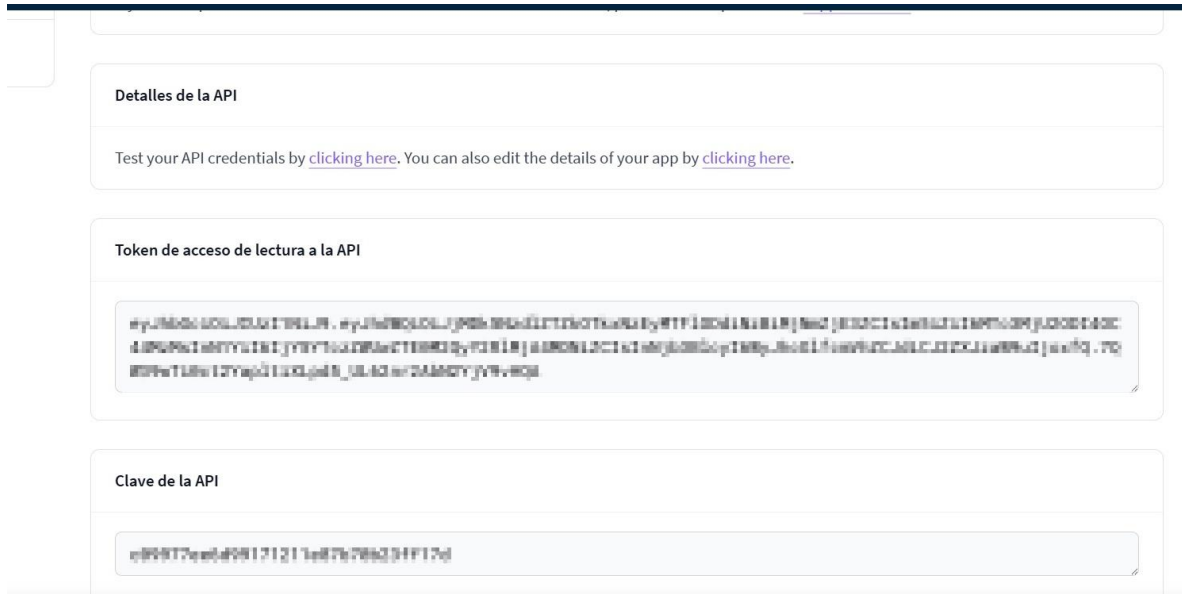


Figure. TMDB credentials. The screenshot shows the API Read Access Token and the API Key separately; sensitive values remain redacted.

For API v3, TMDB supports two forms of application-level authentication. The API Key can be sent through the `api_key` URL parameter. The API Read Access Token can be sent in the Authorization header as a Bearer token. The official documentation considers the Bearer token the default method and notes that the same token can also be used with v4 methods.

Credential	How it is sent	Use in CineVerse
API Read Access Token	Authorization: Bearer <token>	Yes. It is the value of <code>VITE_TMDB_TOKEN</code> .
API Key v3	?api_key=<key>	Not in CineVerse code; it is used only in some manual-test screenshots.

Important. `VITE_TMDB_TOKEN` is not placed after `api_key=`. In CineVerse, that value is the API Read Access Token, and `tmdbApi.ts` sends it as a Bearer header.

3.5 How TMDB is configured within CineVerse

The credential passes through a specific chain. The developer obtains it from TMDB, stores it in `frontend/.env`, Vite exposes it through `import.meta.env`, and `tmdbApi.ts` creates an Axios instance that is reused by the project's services.

API Read Access Token obtained from TMDB

```
-> frontend/.env
-> VITE_TMDB_TOKEN
-> Vite / import.meta.env
-> tmdbApi.ts
-> Authorization: Bearer <token>
-> api.themoviedb.org/3
```

```
# frontend/.env
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="<API Read Access Token>"

// frontend/src/core/api/tmdbApi.ts
const baseUrl = import.meta.env.VITE_TMDB_BASE_URL;
const token = import.meta.env.VITE_TMDB_TOKEN;
```

```
const api = axios.create({
  baseURL,
  headers: {
    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json",
  },
});
```

axios.create() does not send a request yet. It only builds a reusable client. tmdbApi.ts remembers the base address and the credential; it does not decide whether to search for Batman, load a person, or request the top-rated movies.

Credential security in this design. The VITE_* variables used by the frontend are embedded in the JavaScript generated by Vite and can be inspected from the browser. For that reason, VITE_TMDB_TOKEN must not be treated as a server-side secret. If a credential needed to remain private, it would have to be used from a server-side layer.

3.6 Which files actually communicate with TMDB

CineVerse does not have a class named TMDB. The integration is distributed among a shared HTTP client, TypeScript modules that know the endpoints, and composables that coordinate data for the screens.

```
View / component
-> composable
-> movies.service.ts / person.service.ts
-> tmdbApi.ts
-> Axios
-> HTTPS
-> TMDB API
-> JSON
-> Vue Query / composable
-> interface
```

Part	Responsibility
tmdbApi.ts	Creates the Axios client with baseURL and Authorization: Bearer.
movies.service.ts	Knows the movie endpoints: lists, search, Discover, genres, details, credits, videos, and similar movies.
person.service.ts	Queries people and filmographies.
composables	Decide when to request data, coordinate Vue Query, and prepare information for the view.
Views / components	Visually render the result and react to user actions.

For example, useHomeMovies creates three independent queries for trending, top-rated, and now-playing movies. useMoviesQuery chooses among lists, text search, and Discover. useMovieDetail combines details, credits, videos, and similar movies. usePersonDetail retrieves a person's information and filmography.

3.7 Complete example: the user opens “Top Rated”

The flow starts with a visible action. The user enters the explorer and selects Top Rated. useMoviesQuery detects listType="top Rated", requests the corresponding page, and Vue Query retains the response so MovieResults can render it.

```
User selects "Top Rated"
-> MoviesView.vue keeps listType="top Rated"
-> useMoviesQuery.ts executes queryFn
-> getMovieList("top Rated", page, language)
-> movies.service.ts
-> api.get("/movie/top Rated", { params: { page, language } })
-> tmdbApi.ts adds baseURL + Authorization
-> TMDB returns page + results + total_pages + total_results
```

- > Vue Query keeps the received page
- > MovieResults merges the loaded pages
- > MovieCard renders each movie

3.7.1 What HTTP request this flow builds

```
GET https://api.themoviedb.org/3/movie/top_rated?language=es&page=1
Authorization: Bearer <VITE_TMDB_TOKEN>
Accept: application/json
```

The URL contains the base address, the endpoint, and the query parameters. The credential does not travel inside the JSON and is not part of the route used by CineVerse: it is sent in the Authorization header. TMDB responds with JSON, and Axios places that content in response.data.

3.7.2 Manually testing an endpoint in Postman

TMDB calls can also be tested outside CineVerse. The following screenshot uses the other authentication method supported by API v3: the API Key as the api_key parameter. The value is redacted. This test makes it possible to inspect the endpoint, parameters, HTTP status, and JSON; CineVerse code uses Bearer authentication, not api_key.

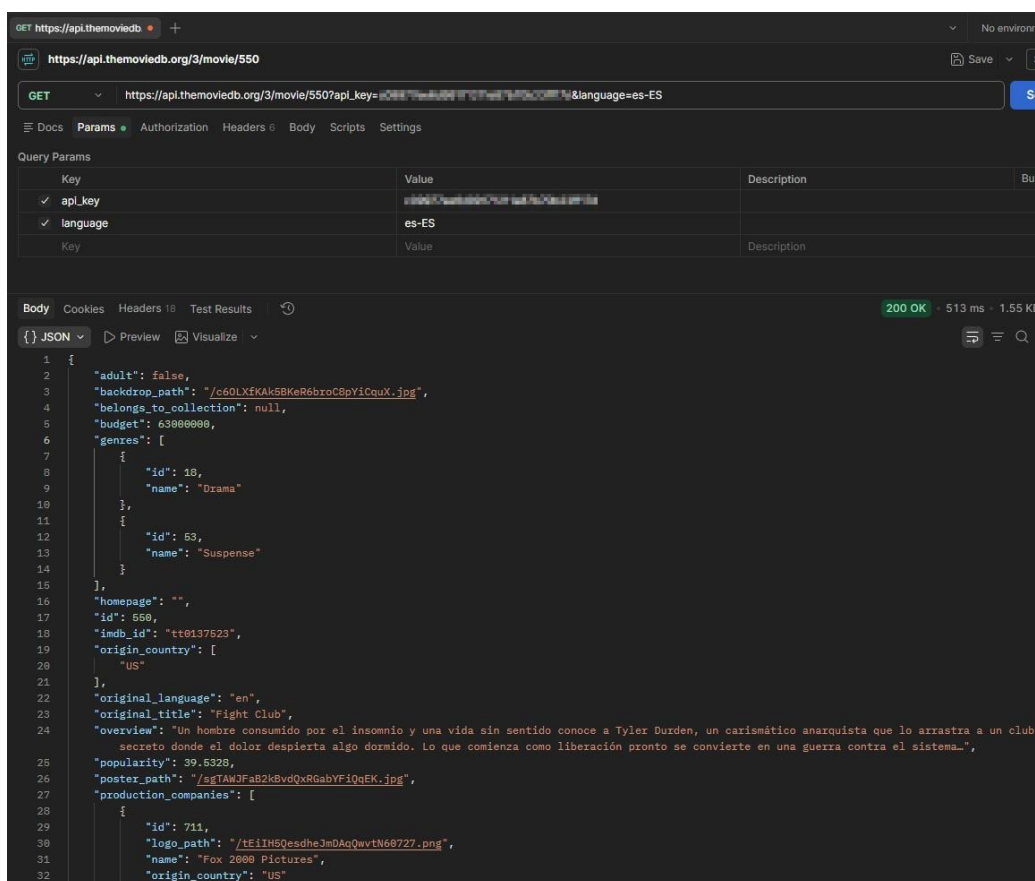


Figure. Manual test of GET /3/movie/550 in Postman: request parameters, 200 OK response, and the beginning of the JSON. The screenshot uses the API Key v3; CineVerse uses the API Read Access Token as a Bearer token.

3.8 What TMDB returns and how TypeScript interprets it

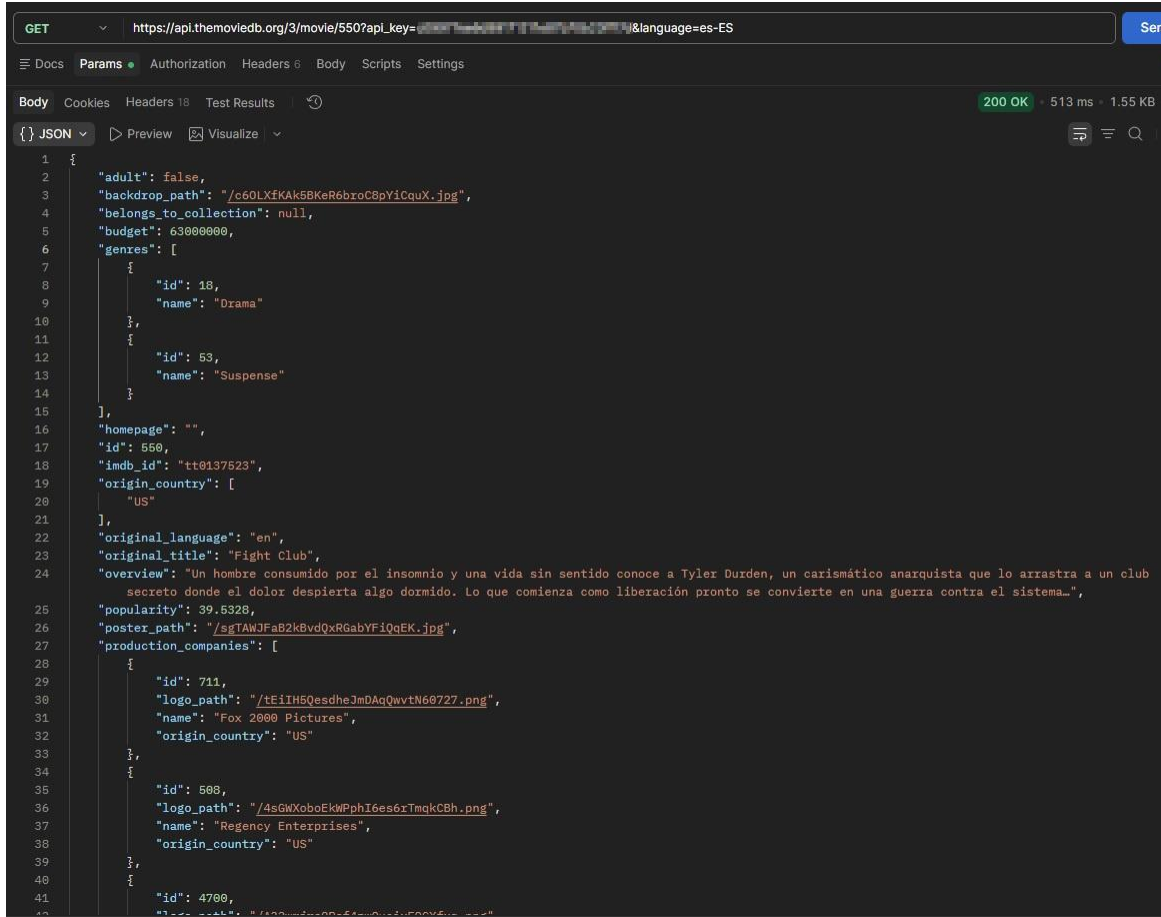


Figure. JSON response for a movie in TMDB. Fields such as `id`, `genres`, `overview`, `poster_path`, and `production_companies` are visible.

A detail endpoint returns a movie object; paginated endpoints, such as `top_rated` or `search/movie`, return a structure containing `page`, `results`, `total_pages`, and `total_results`. CineVerse declares TypeScript types so it can work safely with the structure it expects.

```
export interface PaginatedResponse<T> {
  page: number;
  results: T[];
  total_pages: number;
  total_results: number;
}
```

The generic type helps during development, but it does not validate JSON received at runtime. If TMDB unexpectedly changed the structure of a response, TypeScript could not stop data that had already arrived over the network; error states and application checks therefore remain necessary.

3.9 TMDB endpoints used by the project

TMDB endpoint	Use in CineVerse
<code>/trending/movie/day</code>	Trending movies on the Home page.
<code>/movie/popular</code>	Popular catalog.
<code>/movie/now_playing</code>	Now-playing movies.
<code>/movie/upcoming</code>	Upcoming releases.

TMDB endpoint	Use in CineVerse
/movie/top Rated	Top-rated movies.
/search/movie	Text search.
/discover/movie	Advanced filters when there is no search text.
/genre/movie/list	Genre catalog.
/movie/{id}	Main movie details.
/movie/{id}/credits	Cast and crew.
/movie/{id}/videos	Videos and trailers.
/movie/{id}/similar	Similar movies.
/person/{id}?append_to_response=movie_credits	Person details + filmography.

Not all endpoints are used in the same way. /search/movie is designed for text; /discover/movie accepts structured filters. Section 16 explains how CineVerse chooses between those paths and what limitation appears when it combines text search with local filters.

3.10 How images are built

TMDB does not normally send the image file inside the JSON. Objects include paths such as poster_path or backdrop_path. The browser then builds an image URL by combining the image server, a size, and the file_path returned by the API.

JSON response:
 "poster_path": "/abc123.jpg"

Constructed URL:
 https://image.tmdb.org/t/p/w500/abc123.jpg

The official TMDB documentation describes the image URL as the combination of base_url + file_size + file_path. It also provides the /configuration endpoint for querying sizes and configuration data. CineVerse uses specific sizes such as w185, w500, w1280, or original depending on where the image appears in the interface.

3.11 Language and fallback mechanisms

TMDB supports localization for much of its metadata, although not all fields have complete translations. CineVerse includes fallbacks to try to keep a page useful when the first response comes back empty.

- getMovieDetail requests the other language again only if both poster and backdrop are empty.
- getMovieCredits requests the other language again if both cast and crew are empty.
- getMovieVideos requests /videos again without language when there are no results.
- getSimilarMovies currently uses a fixed language="en-US", regardless of the interface language.

TMDB documents ISO 639-1 for languages and commonly uses combinations with country codes, such as en-US or pt-BR. CineVerse also maintains its own interface language through vue-i18n; translating buttons and localizing TMDB data are related but distinct concerns.

3.12 People, append_to_response, and user accounts

For a person, person.service.ts uses /person/{id} together with append_to_response=movie_credits. append_to_response is a TMDB capability that allows certain subresources to be included in the same response, avoiding separate requests. Section 22 shows the complete PersonDetailView and filmography flow.

CineVerse account != TMDB account. The API Read Access Token authenticates the application to TMDB. Users who register, verify their email, sign in, and write reviews belong to CineVerse's PHP/MySQL backend. TMDB also offers user authentication for its own accounts, but CineVerse does not use that flow.

3.13 Official TMDB resources

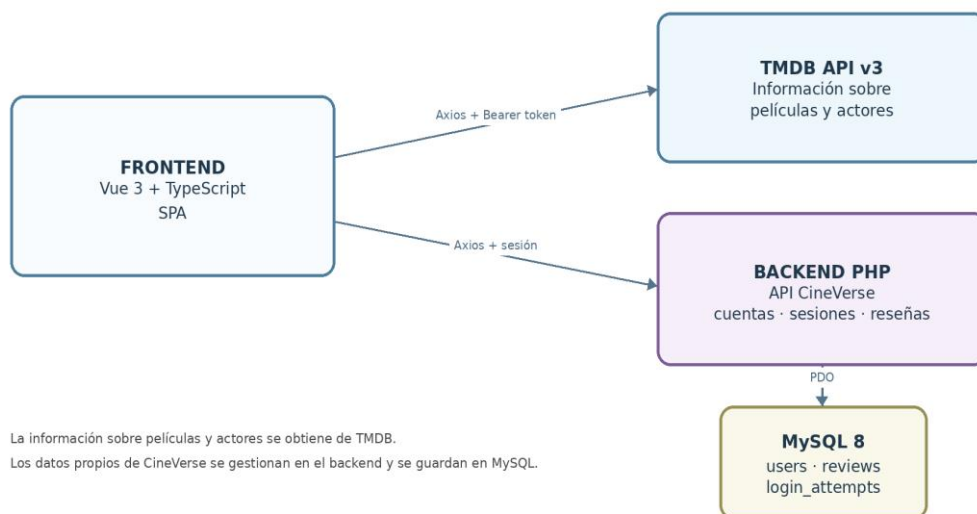
The following links are the recommended official sources for checking changes to authentication, endpoints, images, and terms of use:

- [TMDB public website](#)
- [TMDB Developer - Getting Started](#)
- [TMDB API Reference](#)
- [Application authentication](#)
- [Building image URLs](#)
- [FAQ, attribution, and API use](#)

4. How CineVerse is organized

CineVerse obtains information from two different places. When the user searches for a movie, opens its details, or looks up an actor, the frontend requests that data directly from TMDB. When the user registers an account, signs in, posts a review, or opens the profile, the frontend communicates with CineVerse's PHP backend. The backend uses MySQL when it needs to store or retrieve information that belongs to the application.

Arquitectura general de CineVerse



General architecture: the Vue frontend obtains information from TMDB and uses the PHP backend for CineVerse-specific features.

Before separating responsibilities, it is useful to look at two real flows. In both cases Vue displays the interface, but the destination of the data changes depending on what the user needs.

```

User opens "Top Rated"
-> MoviesView / useMoviesQuery
-> movies.service.ts
-> tmdbApi.ts
-> TMDB
-> JSON response
-> Vue displays the movies

```

```

User signs in
-> LoginView
-> auth.service.ts
-> backendApi.ts
-> PHP backend
-> MySQL
-> JSON response
-> Pinia updates the authentication state

```

The first flow does not pass through PHP: the catalog is queried directly from TMDB. The second does use the application's own API because the session and accounts belong to CineVerse.

In these flows, files ending in `.service.ts` are frontend TypeScript modules that centralize data-access operations. `tmdbApi.ts` and `backendApi.ts` are also CineVerse files: they create Axios clients configured to communicate, respectively, with TMDB and the PHP API. They are shown here only to place the overall flow in context; Sections 11.2.2 and 15 later explain what each file type does, where it is located, and how it is used.

4.1 Responsibilities by system

System	What it does	Examples
Vue frontend	Displays the interface, manages navigation, and coordinates the information each screen needs.	Views, components, filters, navigation, and shared state.
TMDB	External service from which CineVerse obtains movie and actor information.	Movies, actors, genres, credits, videos, and images.
PHP backend	Manages CineVerse-specific features that require security or user data.	Registration, session, reviews, profile, security, and email delivery.
MySQL 8	Permanently stores CineVerse's own data and applies integrity constraints.	users, reviews, and login_attempts.

4.2 Why the backend does not act as a TMDB proxy

In the current implementation, `movies.service.ts` and `person.service.ts` send requests to TMDB directly from the frontend through `tmdbApi.ts`. The PHP backend does not sit between those requests. A server that receives a request and forwards it to another service is commonly called a proxy; CineVerse does not use its backend as a TMDB proxy.

For that reason, there are two different forms of identification. TMDB's Bearer token is the credential the frontend sends to the external service to identify the application. The PHP session cookie is a small identifier the

browser keeps so subsequent requests can be associated with the user who signed in to CineVerse. One credential does not replace the other.

5. Project structure

The project is divided into two parts: frontend and backend. The frontend groups files according to the CineVerse feature they belong to. For example, auth groups registration and sign-in; movies groups search, filters, and movie details; person contains the actor screen; and profile groups the profile. These groupings are located inside the features folder, which is the name the project uses to keep each feature in one place.

5.1 Frontend

Responsibilities are also separated within each feature. A view is a complete screen; a component is part of a screen; a service contains requests to TMDB or the backend; a composable groups reusable Vue logic; a store maintains shared data; and types uses TypeScript to describe the expected shape of certain data.

```

frontend/
├── public/
├── src/
│   ├── assets/
│   ├── core/
│   │   ├── api/
│   │   ├── composables/
│   │   ├── i18n/
│   │   └── router/
│   ├── features/
│   │   ├── auth/
│   │   ├── home/
│   │   ├── movies/
│   │   ├── person/
│   │   └── profile/
│   ├── layouts/
│   ├── shared/
│   ├── views/
│   ├── App.vue
│   ├── main.ts
│   └── style.css
├── index.html
├── package.json
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.node.json
└── vite.config.ts

```

Location	Responsibility
src/core/api	Shared Axios configuration: one client for TMDB and another for the backend.
src/core/router	Defines the application's URLs, language, navigation guards, and 404 screen.
src/features/*/views	Complete screens associated with Vue Router routes.
src/features/*/components	Interface parts that belong to a specific feature.
src/features/*/services	Functions that send HTTP requests to TMDB or the backend.
src/features/*/composables	Vue functions that group reusable logic, for example filters or data loading.
stores	Data shared across different screens, mainly authentication and favorites.

Location	Responsibility
types	TypeScript definitions that describe the structure of the data.

Example: what the movies folder contains

```

features/movies/
├── components/
│   └── movie-detail/
├── composables/
│   └── movie-detail/
├── services/
├── stores/
├── types/
│   └── movie-detail/
├── utils/
└── views/

```

The movies folder brings together everything related to movies in one place: TypeScript types, services that query TMDB or the backend, filters, result loading, favorites, movie details, and visual components. This makes it easier to find the files for a feature without searching through general-purpose folders that lack context.

5.2 Backend

The backend separates responsibilities differently. When an HTTP request arrives, each class type has a specific task: middleware performs preliminary checks; a controller receives the data and decides which HTTP response to return; a validator checks the format; a service applies CineVerse rules; and a model executes MySQL queries. Those checks may include the CSRF token, a security value required for operations that modify data. This organization was implemented directly in PHP, without Laravel or another framework.

```

backend/
├── config/
├── controllers/
├── core/
│   ├── auth/
│   ├── http/
│   ├── logging/
│   ├── middleware/
│   └── validation/
├── database/
├── models/
├── public/
│   ├── index.php
│   └── .htaccess
├── routes/
│   └── api.php
├── services/
├── storage/
│   └── logs/
├── templates/
│   └── emails/
├── validators/
├── composer.json
└── composer.lock

```

Layer	What it does	What it should not do
Controller	Receives the HTTP request, obtains the submitted data, calls the required classes, and builds the response.	It should not write SQL queries directly.
Service	Applies application rules and coordinates multiple operations when an action requires them.	It should not contain frontend presentation details.
Model	Reads and modifies the database through PDO prepared statements.	It should not decide which HTTP status code the browser receives.
Validator	Checks that received data has the permitted format, length, and range.	It does not query MySQL to apply business rules.
Middleware	Performs checks before the request reaches the controller, for example a valid session or CSRF token.	It does not execute the final feature requested by the user.

5.3 How to navigate the project

There is no need to memorize every file. The quickest way to navigate the project is to identify whether you are looking for shared infrastructure, a frontend feature, or a backend layer.

Area	What it contains and when to look there
src/core/api	The two shared Axios clients. This is where the frontend connection to TMDB and to the PHP backend is defined.
src/features	User-facing features. auth, movies, person, and profile group their views, services, composables, stores, and types.
src/layouts and src/shared	Reusable interface or logic pieces that do not belong to a single feature.
backend/routes and controllers	HTTP entry points: which URL exists and which controller receives the request.
backend/services and validators	Business rules and data validation before information is stored or modified.
backend/models and database	MySQL access through PDO and prepared statements.
backend/core	Shared infrastructure: session, HTTP, middleware, logging, bootstrap, and base validation.

5.4 Two flows across folders

```
Explore movies
MoviesView.vue
-> useMoviesQuery.ts
-> movies.service.ts
-> core/api/tmdbApi.ts
-> TMDB

Sign in
LoginView.vue
-> auth.service.ts
-> core/api/backendApi.ts
-> routes/api.php
-> AuthController.php
-> AuthService.php
-> User.php / PDO / MySQL
```

These flows are a guide for reading the repository. Later sections explain each step in greater detail.

6. Configuration files and environment variables

CineVerse needs certain values to know which services it must communicate with and how to do so. The frontend needs, for example, the TMDB address, the token that authorizes queries to its API, and the backend address. The backend needs the MySQL connection settings, mail-server configuration, and the folder where logs are stored. Instead of writing those values inside each TypeScript or PHP file, the project stores them in .env files.

An environment variable is a name associated with a configuration value. For example, DB_NAME=cineverse means that the DB_NAME variable contains the name of the database the backend must use. When the code needs that value, it reads the variable instead of having the name cineverse written directly inside Database.php.

The purpose of this section is to follow each value to the place where it has an effect. In the frontend, Vite exposes VITE_ variables through import.meta.env. In the backend, phpdotenv loads backend/.env and makes its values available in \$_ENV for classes such as Database, AppLogger, and MailService.

6.1 Map of frontend configuration files

Before studying a variable or command, it is useful to locate the pieces physically. In CineVerse, the files that control frontend installation and startup are located at the root of the frontend folder. They are not Vue screens: they are files that npm, Vite, and TypeScript read to determine what to install, how to start the project, and which configuration values to expose to the code.

```
cineverse/
├── frontend/
│   ├── package.json
│   ├── package-lock.json
│   ├── vite.config.ts
│   ├── .env
│   ├── .env.example
│   ├── tsconfig.json
│   ├── tsconfig.app.json
│   ├── tsconfig.node.json
│   └── src/
```

File / folder	What it is	Who uses it
frontend/package.json	JavaScript project manifest: declares npm packages and commands.	npm.
frontend/package-lock.json	Record of the exact versions resolved by npm.	npm during reproducible installations.
frontend/node_modules/	Folder generated by npm containing the installed packages.	Node.js, Vite, and code that imports dependencies.

File / folder	What it is	Who uses it
frontend/vite.config.ts	Vite-specific configuration for this project.	Vite, running under Node.js.
frontend/.env	Local frontend configuration values.	Vite.
frontend/.env.example	Template containing the expected variable names.	The developer; Vite does not use it as an automatic replacement for .env.
frontend/tsconfig*.json	TypeScript rules for the application and tooling.	TypeScript / vue-tsc.

This map prevents a common source of confusion: package.json does not contain Axios, vite.config.ts does not contain the VITE_* variables, and .env does not install packages. Each piece has a different responsibility and connects to the next one during startup.

6.2 package.json, npm, package-lock.json, and node_modules

Node.js is the environment CineVerse uses to run JavaScript tools outside the browser. npm is the package manager installed with Node.js. The file npm reads in this project is frontend/package.json.

6.2.1 What package.json is and where it is located

package.json is a JSON file that describes the frontend project. It is located at frontend/package.json. It is not a screen, it is not imported from Vue components, and the browser does not read it directly. npm uses it to determine which dependencies CineVerse needs and which commands it can run.

```
// frontend/package.json (actual excerpt)
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "@tanstack/vue-query": "^5.92.9",
    "axios": "^1.13.6",
    "pinia": "^3.0.4",
    "vue": "^3.5.25",
    "vue-i18n": "^11.2.8",
    "vue-router": "^5.0.3"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^6.0.2",
    "typescript": "~5.9.3",
    "vite": "^7.3.1",
    "vue-tsc": "^3.1.5"
  }
}
```

dependencies contains packages used by the application, such as Vue, Pinia, Axios, and Vue Router. devDependencies contains tools needed to develop, check, or build the project, such as Vite, TypeScript, and vue-tsc. The distinction describes the purpose of each package; npm install installs both groups in a normal development environment.

The symbols shown before some versions are compatibility ranges used by npm. For versions above zero, ^1.13.6 allows a compatible version from 1.13.6 up to, but not including, 2.0.0; ~5.9.3 normally limits updates to patches within the 5.9 series, meaning from 5.9.3 up to, but not including, 5.10.0. package.json declares these ranges; package-lock.json then records the exact versions npm resolved for a reproducible installation.

6.2.2 How package.json scripts become commands

```
npm run dev
-> npm opens frontend/package.json
-> looks for scripts.dev
-> finds "vite"
```

```

-> runs Vite

npm run build
-> npm looks for scripts.build
-> runs "vite build"
-> Vite generates the deployable version

```

Command	What it does in CineVerse
npm install	Reads package.json and package-lock.json and installs dependencies into node_modules.
npm run dev	Runs the dev script and starts the Vite development server.
npm run build	Runs vite build and generates frontend/dist for production.
npm run preview	Serves the generated build locally for verification.

6.2.3 package-lock.json and node_modules: declaration, resolution, and installation

package.json declares which packages and version ranges the project needs. package-lock.json records the exact versions and transitive dependencies resolved by npm. node_modules is the physical folder npm creates when those packages are installed. In CineVerse, node_modules should not be edited to configure Axios, Vue Router, or Vite: the project's own code is kept outside that folder.

```

frontend/package.json
  "axios": "^1.13.6"
    |
    | npm install
    v
frontend/node_modules/axios/
    |
    | import axios from "axios"
    v
CineVerse TypeScript code

```

6.3 Vite and vite.config.ts

Vite is the tool that starts the frontend during development and generates the production build. Vite is part of the project because it is declared in frontend/package.json and npm installs it in node_modules. When npm run dev executes the vite command, Vite loads its own configuration file: frontend/vite.config.ts.

6.3.1 What vite.config.ts is, where it is located, and when it is used

vite.config.ts is a TypeScript file dedicated to frontend configuration. It is located at frontend/vite.config.ts. The browser does not execute it directly, and it is not a Vue view. Vite reads it through Node.js when starting the development server or building the application.

```

npm run dev
-> package.json: scripts.dev = "vite"
-> Vite starts
-> Vite reads frontend/vite.config.ts
-> configures plugins, alias, and proxy
-> serves the frontend

```

6.3.2 Source code of vite.config.ts

```

import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import tailwindcss from '@tailwindcss/vite'
import { fileURLToPath, URL } from 'node:url'

export default defineConfig({
  base: '/',
  plugins: [
    vue(),
    tailwindcss(),
  ],
  resolve: {
    alias: {

```

```

    '@': fileURLToPath(
      new URL('./src', import.meta.url)
    )
  },
  server: {
    proxy: {
      '/api': {
        target: 'http://cineverse.local',
        changeOrigin: true,
        secure: false,
        rewrite: (path) => path.replace(/^\/api/, '')
      }
    }
  }
})

```

Part	What it means in CineVerse
base: "/"	The build assumes the frontend is published from the root of the site.
vue()	Allows Vite to process .vue components.
tailwindcss()	Connects Tailwind to the Vite process; the interface still relies primarily on custom CSS.
alias "@"	Makes @ represent frontend/src and avoids long relative paths.
server.proxy["/api"]	Intercepts development requests whose path begins with /api.
target	Defines the local backend to which Vite forwards those requests: http://cineverse.local.
rewrite	Removes /api before passing the route to the backend.

`defineConfig(...)` is a Vite helper function that allows the configuration to be written with better type support and autocompletion; it does not start the server by itself. The `URL` class and `import.meta.url` make it possible to build an address relative to `vite.config.ts` itself, and `fileURLToPath(...)` converts that file address into a normal operating-system path. Together, these pieces associate the `@` alias with `frontend/src`. As a result, an import such as `import backendApi from "@core/api/backendApi"` actually points to `frontend/src/core/api/backendApi.ts` and avoids long relative paths.

Inside `server.proxy["/api"]`, `target` specifies the proxy's destination server. `changeOrigin: true` makes Vite adjust the `Host` header to match the forwarded destination; it does not authenticate the user or modify the session. `secure: false` controls certificate verification when the destination uses HTTPS; because the current target is `http://cineverse.local`, TLS validation does not apply in this environment. `rewrite` removes the `/api` prefix before passing the route to Apache, so `/api/user` becomes `/user`.

6.3.3 Real example of the development proxy

In this example, `baseUrl` means the common part of an address that `Axios` prepends to relative endpoints. If `backendApi` has `baseUrl="/api"` and the code requests the endpoint `"/user"`, the resulting browser path is `"/api/user"`.

```

backendApi.get("/user")
  -> baseUrl = "/api"
  -> browser requests http://localhost:5173/api/user
  -> Vite detects /api
  -> rewrite removes /api
  -> proxy forwards to http://cineverse.local/user
  -> Apache / PHP respond

```

This proxy exists during development because the browser is using the frontend served by Vite at `localhost:5173` while the PHP backend runs at `cineverse.local`. In production, `VITE_BACKEND_URL` can point directly to the backend's public URL, and the development proxy is no longer the primary path.

6.4 frontend/.env, .env.example, and VITE_ variables

frontend/.env is the frontend's local-values file. It is located at the root of frontend and may vary between computers or environments. frontend/.env.example is a template that documents which variables must exist, but Vite does not automatically use it as a replacement for .env.

```
frontend/
├── .env           <- active values for that installation
├── .env.example  <- template for creating .env
├── package.json
└── vite.config.ts
```

Do not confuse frontend/.env with backend/.env

frontend/.env is processed by Vite, and its VITE_* variables become available to the code delivered to the browser. backend/.env is loaded by PHP and contains server-side values such as MySQL or SMTP connection settings. They are different files read by different systems.

6.4.1 How Vite exposes VITE_ variables to the code

When Vite starts or builds the frontend, it reads frontend/.env. For a custom variable to be available in browser code, its name must begin with VITE_. Vite exposes it through import.meta.env. The prefix is not required by TMDB or PHP: it is Vite's convention for variables that are accessible from the frontend.

```
frontend/.env
VITE_BACKEND_URL="/api"
    |
    | Vite reads .env
    v
import.meta.env.VITE_BACKEND_URL
    |
    v
frontend TypeScript code
```

6.4.2 VITE_TMDB_BASE_URL: TMDB base address

VITE_TMDB_BASE_URL is a frontend environment variable. It is defined in frontend/.env, and its template appears in frontend/.env.example. Its value specifies the common address of TMDB API v3. In Axios, baseUrl is precisely that common part of the address: it is prepended to every relative endpoint requested by the code. The file frontend/src/core/api/tmdbApi.ts reads VITE_TMDB_BASE_URL directly and uses it as the baseUrl of the shared Axios instance.

```
# frontend/.env
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"

// frontend/src/core/api/tmdbApi.ts
const baseUrl = import.meta.env.VITE_TMDB_BASE_URL;
```

6.4.3 VITE_TMDB_TOKEN: TMDB Bearer credential

VITE_TMDB_TOKEN is also defined in frontend/.env. It contains the API Read Access Token provided by TMDB in the account's API settings. CineVerse reads it from import.meta.env, and tmdbApi.ts places it in the shared Axios instance's Authorization header using the format Bearer <token>. Section 3 explains from the beginning what TMDB is, how this credential is obtained, and how it moves through the frontend until it reaches the API.

```
# frontend/.env
VITE_TMDB_TOKEN="<token provided by TMDB>"

// tmdbApi.ts
const token = import.meta.env.VITE_TMDB_TOKEN;
Authorization: `Bearer ${token}`
```

Important note about VITE_TMDB_TOKEN

A VITE_ variable is ultimately embedded in the JavaScript delivered to the browser. For that reason, this token must not be treated as a private server secret. A credential that must remain secret should be used from a backend layer.

6.4.4 VITE_BACKEND_URL: how the frontend finds the PHP API

VITE_BACKEND_URL is a frontend environment variable that specifies which base address CineVerse should use to communicate with its own PHP API. It is defined in frontend/.env and appears as an example in frontend/.env.example. During development its value is /api. When backendApi uses that value as baseUrl, Axios prepends it to relative endpoints such as /login, /user, or /profile; for example, baseUrl=/api plus the endpoint /user produces /api/user.

```
# frontend/.env
VITE_BACKEND_URL="/api"
```

The variable is not a function, and it is not a URL invented by Axios. backendApi.ts reads it directly when creating the backend Axios instance. auth.service.ts, reviews.service.ts, and profile.service.ts then reuse backendApi and therefore receive that configuration indirectly.

```
frontend/.env
VITE_BACKEND_URL="/api"
|
| Vite expone import.meta.env
v
frontend/src/core/api/backendApi.ts
axios.create({ baseUrl: "/api" })
|
+--> auth.service.ts
+--> reviews.service.ts
+--> profile.service.ts
|
v
API PHP
```

6.4.5 Summary: who uses each frontend variable

Variable	Location	Who reads it directly	Who uses it afterward
VITE_TMDB_BASE_URL	frontend/.env	tmdbApi.ts	movies.service.ts and person.service.ts through api.
VITE_TMDB_TOKEN	frontend/.env	tmdbApi.ts	movies.service.ts and person.service.ts through api.
VITE_BACKEND_URL	frontend/.env	backendApi.ts	auth.service.ts, reviews.service.ts, and profile.service.ts through backendApi.

Section 15 returns to these pieces from an execution perspective: it shows how tmdbApi.ts and backendApi.ts create Axios clients and how *.service.ts files use those clients to send real requests.

6.5 Backend configuration: backend/.env

The backend has its own backend/.env file. PHP does not read that file by itself. CineVerse uses a library called vlucas/phpdotenv to open it and convert its values into variables that PHP code can access.

The phpdotenv library is declared in backend/composer.json. Composer is the backend dependency manager: it reads composer.json, installs the required libraries when composer install is run, and generates vendor/autoload.php so PHP can load them.

```
"require": {
  "vlucas/phpdotenv": "^5.6",
  "monolog/monolog": "^3.10",
  "phpmailer/phpmailer": "^7.1"
}
```

The sequence is as follows: composer.json declares phpdotenv; composer install installs it inside vendor/; vendor/autoload.php makes it loadable; and core/bootstrap.php uses it to read backend/.env. The values then become available through \$_ENV.

6.5.1 Backend variables: what each one controls

```

APP_ENV=local
APP_DEBUG=true

DB_HOST=localhost
DB_PORT=3306
DB_NAME=cineverse
DB_USER=user
DB_PASSWORD=password
DB_CHARSET=utf8mb4

CINEVERSE_LOG_PATH=storage/logs

MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=user
MAIL_PASSWORD=password
MAIL_FROM=no-reply@example.com
MAIL_FROM_NAME=CineVerse

FRONTEND_URL=http://localhost:5173

```

Variable	What it controls	Where it is used in CineVerse
APP_ENV	Identifies the declared environment, for example local or production.	core/bootstrap.php checks that it exists. The current code does not change responses automatically based solely on this value.
APP_DEBUG	Stores the installation's debug flag.	bootstrap.php validates the value; it currently does not automatically enable different traces or messages for the browser.
DB_HOST	Address of the server where MySQL runs.	Database::connect() includes it in the DSN used by PDO.
DB_PORT	Port used to access MySQL.	Database::connect() adds it to the DSN together with the host, database, and charset.
DB_NAME	Name of the database containing users, reviews, and login_attempts.	Database::connect() uses it as dbname in the PDO connection.
DB_USER	MySQL user used to open the connection.	It is passed as the second argument when creating PDO.
DB_PASSWORD	Password for the MySQL user.	It is passed to PDO when opening the connection; it may be empty only if that MySQL installation is configured that way.
DB_CHARSET	Connection character set; the project uses utf8mb4.	It is part of the DSN so PDO and MySQL can exchange Unicode text correctly.
CINEVERSE_LOG_PATH	Folder where backend logs are written.	AppLogger reads it to create Monolog's rotating files.

Variable	What it controls	Where it is used in CineVerse
MAIL_HOST	SMTP server to which PHPMailer connects.	MailService assigns it as the mail connection host.
MAIL_PORT	SMTP server port.	MailService uses it when configuring PHPMailer.
MAIL_USERNAME	Account CineVerse uses to authenticate with SMTP.	MailService uses it as the authentication username.
MAIL_PASSWORD	Password for the SMTP account.	MailService uses it when authenticating before sending the message.
MAIL_FROM	Address that the recipient sees as the sender.	MailService configures it with setFrom(); it does not necessarily have to match MAIL_USERNAME.
MAIL_FROM_NAME	Visible sender name.	MailService displays it alongside MAIL_FROM in CineVerse emails.
FRONTEND_URL	Public frontend address.	MailService uses it to build verification and recovery links that return to Vue.

bootstrap.php requires DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_CHARSET, APP_ENV, APP_DEBUG, and CINEVERSE_LOG_PATH. Mail variables and FRONTEND_URL are not checked during startup; if they are missing or misconfigured, the problem may appear only when an action attempts to send an email.

6.6 core/bootstrap.php: how backend/.env is loaded

public/index.php loads core/bootstrap.php at the beginning of every request. bootstrap.php prepares what the backend needs before processing a route: it loads the libraries installed by Composer, sets the time zone, and uses phpdotenv to read backend/.env.

```
require_once __DIR__ . '/../vendor/autoload.php';

date_default_timezone_set('America/Argentina/Buenos_Aires');

$dotenv = Dotenv\Dotenv::createImmutable(__DIR__ . '/..');
$dotenv->load();
```

vendor/autoload.php is the file generated by Composer that allows library classes and project classes to be loaded without writing a different require_once for every file. Dotenv::createImmutable(__DIR__ . "/..") tells phpdotenv to look for .env at the backend root. After load(), the code can read values such as \$_ENV["DB_HOST"] or \$_ENV["FRONTEND_URL"].

createImmutable() prevents the .env file from replacing a variable that was already defined by the operating system or server before PHP started.

6.7 composer.json and automatic class loading

In addition to installing dependencies, Composer configures automatic loading of CineVerse classes. The backend uses the PSR-4 convention, which maps a class namespace to a project folder. A namespace is the hierarchical name PHP uses to organize classes and prevent naming conflicts.

```

"autoload": {
    "psr-4": {
        "App\\Controllers\\": "controllers/",
        "App\\Core\\": "core/",
        "App\\Database\\": "database/",
        "App\\Models\\": "models/",
        "App\\Services\\": "services/",
        "App\\Validators\\": "validators/"
    }
}

```

For example, when PHP needs `App\Services\MailService`, Composer knows it should look for the class in `services/MailService.php`. This mechanism is called autoloading. `composer.lock` preserves the exact library versions resolved by Composer.

6.8 Connecting to MySQL with PDO

`Database.php` is located at `backend/database/Database.php`. It is a CineVerse class responsible for creating and returning the PDO connection used by the backend to communicate with MySQL.

`public/index.php` calls `Database::connect()` at the start of each request; the same PDO object is then passed to the layers that need to query or modify MySQL. `Database::connect()` reads `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, and `DB_CHARSET` from `$_ENV`. It uses them to build a DSN, which is a string containing the data required to open the connection, and creates the PDO object.

```

$dsn = "mysql:host={$host};port={$port};dbname={$dbName};charset={$charset}";

return new PDO(
    $dsn,
    $user,
    $password,
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
    ]
);

```

PDO is the PHP interface used to communicate with MySQL. `ERRMODE_EXCEPTION` causes a MySQL error to become an exception that the backend can catch. `FETCH_ASSOC` causes a row to be returned as an array whose indexes are the column names; for example, `["id" => 15, "username" => "user"]`.

`config/config.php` can also return a PDO connection, but the main API entry point does not depend on that file: `public/index.php` loads `bootstrap.php` and calls `Database::connect()` directly inside a block that can detect a MySQL outage and respond with HTTP 503. Status code 503 indicates that the service is temporarily unavailable, in this case because the database could not be used.

6.9 Log configuration

The log location is configured from `backend/.env` through `CINEVERSE_LOG_PATH`. This variable belongs to CineVerse: it is not a built-in PHP or Monolog option. `core/bootstrap.php` loads `backend/.env` with `phpdotenv` and makes the value available in `$_ENV` so `AppLogger` can read it.

```

# backend/.env
CINEVERSE_LOG_PATH=storage/logs

```

With the relative value `storage/logs`, `AppLogger` normally resolves the `backend/storage/logs` folder. The base file used by the code is `cineverse-backend.log`, and Monolog appends the date during daily rotation, generating names such as `cineverse-backend-2026-09-07.log`.

This section only describes the configuration. Section 29 develops the full implementation: how `AppLogger` obtains the path, how it creates the Monolog Logger, how it configures `RotatingFileHandler` and `LineFormatter`, and how `AppLogger::error()` ultimately writes an entry to the file.

6.10 Mail configuration and FRONTEND_URL

MailService uses PHPMailer to send account-verification and password-recovery emails. PHPMailer connects to an SMTP server, which is the service responsible for sending the message. The connection settings are obtained from backend/.env.

```
MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=no-reply@example.com
MAIL_PASSWORD=<password>
MAIL_FROM=no-reply@example.com
MAIL_FROM_NAME=CineVerse
FRONTEND_URL=http://localhost:5173
```

MAIL_USERNAME is the account used to authenticate with the SMTP server. MAIL_FROM is the address the recipient sees as the sender; they may be the same, but they serve different purposes. FRONTEND_URL specifies the frontend address and is used to build the links included in emails.

Example: with FRONTEND_URL=http://localhost:5173, language es, and token abc123, MailService can build http://localhost:5173/es/verify-email?token=abc123. In production, FRONTEND_URL must point to the frontend's actual HTTPS domain.

CORS (Cross-Origin Resource Sharing) is a browser policy that determines which website may communicate with an API when the two are at different addresses. Changing FRONTEND_URL does not automatically change that rule. CorsMiddleware still contains http://localhost:5173 directly in the code, so the allowed origin must be reviewed separately when the project is published.

The configuration above indicates which server will be used to send email. The complete flow - language, link construction, HTML template, PHPMailer, SMTP connection, sending, and failure handling - is developed in Section 25.

6.11 Values that are still hard-coded

Not all CineVerse configuration is obtained from .env. Some values remain written directly inside PHP or TypeScript files. They work in the current local environment, but they must be reviewed if the frontend address, backend location, or deployment method changes.

Configuration	Where it is located	Current value or behavior
Time zone	backend/core/bootstrap.php	America/Argentina/Buenos_Aires.
Allowed CORS origin	backend/core/middleware/CorsMiddleware.php	http://localhost:5173.
CORS-allowed methods and headers	CorsMiddleware.php	GET, POST, PUT, PATCH, DELETE, OPTIONS; Content-Type and X-CSRF-Token.
Session inactivity timeout	backend/core/auth/Session.php	1800 seconds / 30 minutes.
Session cookie	Session.php	lifetime=0, path=/, HttpOnly=true, SameSite=Lax; Secure if PHP detects HTTPS.
SMTP security	backend/services/MailService.php	SMTP authentication, STARTTLS (connection encryption), and UTF-8.
Log rotation	backend/core/logging/AppLogger.php	30 files; minimum level ERROR.
Base route prefix	backend/public/index.php	/cineverse/backend/public.

Configuration	Where it is located	Current value or behavior
Development proxy target	frontend/vite.config.ts	http://cineverse.local.

A value being written directly in the code does not mean the project fails during development. The important point is to know that these locations must also be reviewed when moving CineVerse to another server. For example, a different domain requires CORS to be reviewed, and a different subfolder may require changing the base route prefix.

7. MySQL 8 database

CineVerse uses MySQL to store information that belongs to the application itself. Movies, actors, genres, images, and credits are not copied into this database: they are queried from TMDB when needed. The local schema has three tables, each with a different responsibility.

This separation allows MySQL to store CineVerse accounts and activity without attempting to duplicate TMDB's external catalog.

Table	What it is used for in CineVerse
users	Stores accounts: username, email, password_hash, email verification, and temporary password-recovery data.
reviews	Stores reviews written by users and associates them with a movie through TMDB's movie_id.
login_attempts	Maintains a temporary counter of failed attempts for each IP + email combination to enforce the login limit.

Three simple examples

Successful registration

-> a row is created in users

User posts a review

-> a row is created in reviews

-> user_id points to users.id

-> movie_id identifies the movie in TMDB

Incorrect password

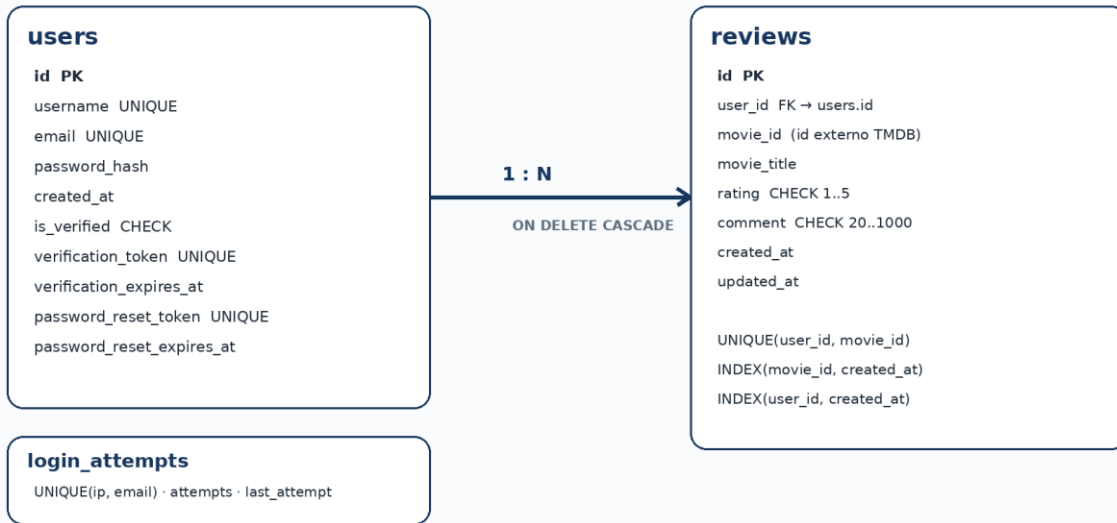
-> login_attempts is created or incremented for that IP + email

From that practical role, the schema's technical rules can be understood. A table groups records of the same type; each row is a record, and each column represents a piece of data.

MySQL uses PRIMARY KEY to identify a row; AUTO_INCREMENT to generate IDs; UNIQUE to prevent duplicates; FOREIGN KEY to relate tables; CHECK to constrain values; and INDEX to speed up certain searches. INT UNSIGNED, VARCHAR, TEXT, and DATETIME indicate what kind of data each column can store.

The SQL file creates the tables with InnoDB and uses utf8mb4. InnoDB supports transactions and foreign keys; utf8mb4 allows Unicode text to be stored, including accented characters and other symbols. The following diagram summarizes the actual relationship among the three tables.

Modelo de datos real - MySQL 8



Actual relationship among the three tables defined by the SQL file.

7.1 users table

Column	Type	Rule / function
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT.
username	VARCHAR(20)	NOT NULL, UNIQUE. Public name and duplicate prevention.
email	VARCHAR(80)	NOT NULL, UNIQUE. Login, verification, and recovery.
password_hash	VARCHAR(255)	Result generated from the password; the original password is never stored.
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP.
is_verified	TINYINT(1)	DEFAULT 0 and CHECK IN (0,1).
verification_token	VARCHAR(255)	UNIQUE; may be NULL. Confirmation token.
verification_expires_at	DATETIME	Verification-token validity period.
password_reset_token	VARCHAR(255)	UNIQUE; may be NULL. Recovery token.
password_reset_expires_at	DATETIME	Recovery-token validity period.

7.2 reviews table

Column	Type	Rule / function
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT.
user_id	INT UNSIGNED	FOREIGN KEY to users.id; ON DELETE CASCADE deletes the user's reviews if the user is deleted.
movie_id	INT UNSIGNED	External movie ID in TMDb.

Column	Type	Rule / function
movie_title	VARCHAR(255)	Title persisted together with the review.
rating	TINYINT UNSIGNED	CHECK between 1 and 5.
comment	TEXT	CHECK requires between 20 and 1000 characters.
created_at	DATETIME	Creation date.
updated_at	DATETIME	Updated automatically when the row is modified.

Important integrity rule

UNIQUE(user_id, movie_id) enforces in MySQL the rule of one review per user and movie. The application checks that rule before the INSERT, but the MySQL constraint is the final guarantee even if two requests occur concurrently.

Indexes and relationship with users

- idx_reviews_movie_created (movie_id, created_at): supports listing a movie's reviews ordered by date.
- idx_reviews_user_created (user_id, created_at): supports the profile and the user's own reviews.
- fk_reviews_user: guarantees that user_id exists and deletes the user's reviews if the user is deleted.

7.3 login_attempts table

Column	Type	Use
id	INT UNSIGNED	Internal identifier.
ip	VARCHAR(45)	IP address; length compatible with IPv6 representations.
email	VARCHAR(80)	Email associated with the attempt.
attempts	TINYINT UNSIGNED	Number of accumulated failures.
last_attempt	DATETIME	Updated with each attempt and allows old records to be cleaned up.

login_attempts is not a general access history. It is a temporary counter used by AuthService to limit failed login attempts. The IP + email combination is unique, so only one row exists per pair; last_attempt makes it possible to locate records that have fallen outside the 15-minute window.

7.4 Why there is no movies table

CineVerse does not create a movies table because the complete movie information continues to belong to TMDB. Each review stores movie_id to identify the corresponding TMDB movie and movie_title so the saved title can be displayed alongside the user's activity without storing a complete copy of the movie in MySQL.

7.5 Concrete example: what is stored when a review is written

The reviews table clearly shows which data belongs to TMDB and which belongs to CineVerse. When a user writes a review, the backend does not store a complete copy of the movie. It stores only the reference needed to associate the review with the movie and the content written by the user.

Conceptual example of a row created in reviews:

```
id           = 8301
user_id      = 125
movie_id     = 550
movie_title  = "Fight Club"
rating       = 5
comment      = "A review written by the user..."
created_at   = 2026-09-08 13:20:00
updated_at   = 2026-09-08 13:20:00
```

user_id comes from the PHP session and links the row to users.id. movie_id and movie_title come from the movie the frontend obtained from TMDB. rating and comment are the values entered by the user. created_at and updated_at are maintained by MySQL. Section 26 follows this complete flow from the Vue form to this INSERT.

If the user later deletes the account, ON DELETE CASCADE on the reviews.user_id foreign key causes MySQL to delete the user's reviews as well. If an individual review is deleted, only that row disappears; the movie continues to exist in TMDB because CineVerse does not own that catalog.

8. Test data and SQL file

The SQL file is prepared to create the CineVerse database and load test data in a single execution. It first configures MySQL, creates the database if it does not exist, drops any applicable previous tables, creates the three current tables, and finally inserts the users and reviews.

It is not an incremental migration. A migration usually applies a small change to an existing database while attempting to preserve its data. This file, by contrast, contains DROP TABLE IF EXISTS and is intended to recreate a controlled environment from scratch.

Before loading users and reviews, the script executes START TRANSACTION to group the bulk data load. At the end, it also adjusts the next AUTO_INCREMENT values for users and reviews so new records continue after the preloaded set. Because MySQL can perform implicit commits when executing certain definition statements, such as ALTER TABLE, the complete file should not be interpreted as a single transaction that can be rolled back from beginning to end.

Item	Quantity/ status
Synthetic users	5.000
Verified users	4.800
Unverified users	200
Reviews	24.322
Distinct comments	24.322
Shared password for the test data	password (tests only)

Import warning

The script contains DROP TABLE IF EXISTS and also removes legacy tables that are no longer used by the plain-PHP backend. It should be treated as a recreation/loading script for a controlled environment, not as a harmless operation on a production database.

General script structure

```
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS `reviews`;
DROP TABLE IF EXISTS `login_attempts`;
...
DROP TABLE IF EXISTS `users`;

CREATE TABLE `users` (...);
CREATE TABLE `login_attempts` (...);
CREATE TABLE `reviews` (...);

SET FOREIGN_KEY_CHECKS = 1;
START TRANSACTION;
-- user and review INSERTs
COMMIT;
```

The shared password `password` belongs only to the test dataset. It does not represent the password rule of the actual form. When registering a new account, CineVerse requires between 8 and 32 characters and checks for an uppercase letter, a lowercase letter, a number, and a special character.

8.1 Actual execution order of the SQL file

The file is not merely a collection of INSERT statements. It is organized so a complete test environment can be rebuilt without relationships between tables interrupting the process.

```
SET FOREIGN_KEY_CHECKS = 0
    -> allows related tables to be dropped during reconstruction
DROP TABLE IF EXISTS ...
    -> removes the previous schema covered by the script
CREATE TABLE users / login_attempts / reviews
    -> creates the active schema
SET FOREIGN_KEY_CHECKS = 1
    -> reenables foreign-key checks
START TRANSACTION
    -> begins the bulk load of test data
INSERT ...
    -> loads users and reviews
ALTER TABLE users / reviews AUTO_INCREMENT = ...
    -> sets the next identifiers; this DDL operation causes an implicit commit
COMMIT
    -> final explicit close of the script
```

Disabling `FOREIGN_KEY_CHECKS` is useful during recreation because `reviews` depends on `users`. It does not mean the database remains without relationships: after the tables are created again, the checks are re-enabled and the FOREIGN KEY, UNIQUE, and CHECK constraints operate normally again.

9. How a user action flows through CineVerse

Using the basic vocabulary from Section 1.3, it is useful to follow what happens from the moment the user performs an action until the screen receives the result. The examples use HTTP requests and responses, JSON, and endpoints already introduced there. When CSRF appears, it refers to a security token that protects operations that modify data.

The following three examples show different flows: viewing a movie uses TMDB; signing in uses the backend and MySQL; posting a review combines TMDB information with a CineVerse account.

9.1 Opening a movie

1. The user opens a movie URL. Vue Router recognizes the numeric id and displays `MovieDetailView`.
2. The view uses `useMovieDetail`, which calls `getMovieDetail(id, language)` to request the main information.
3. `movies.service.ts` sends the request to TMDB through `tmdbApi.ts`. The client adds the Bearer token automatically.

4. If TMDB returns the movie, CineVerse requests credits, videos, and similar movies at the same time so it does not wait for one query after another.
5. `useMovieDetail` prepares the data needed by the screen: director, cast, trailer, and similar movies.
6. Reviews do not come from TMDB. The Cast tab is the initial tab; the PHP reviews API is queried the first time the user opens the Reviews tab and `MovieReviews` is mounted.

9.2 Signing in

1. The user completes the form. `LoginView` performs basic checks before submitting it.
2. `auth.service.ts` sends the data to the backend with a `POST /login` request through `backendApi.ts`.
3. `AuthController` receives the data, checks its basic shape, and passes the work to `AuthService`.
4. `AuthService` applies the sign-in rules: it reviews previous attempts, finds the user, verifies the password, and checks that the email address has been verified.
5. If the login is successful, `Session` regenerates the session identifier, stores `user_id`, and creates a CSRF token.
6. The backend returns the user data and CSRF token. The Pinia store updates the frontend authentication state.

9.3 Posting a review

1. The screen already knows the movie id and title because they were obtained from TMDB.
2. The user selects a rating and writes a comment.
3. `reviews.service.ts` sends `POST /reviews` to the backend and includes `X-CSRF-Token` to demonstrate that the request belongs to the active session.
4. `AuthMiddleware` and `CsrfMiddleware` check the session and token before allowing the request to reach the controller.
5. `ReviewController` receives the data, and `ReviewValidator` sanitizes the comment and checks its length, movie-title, and rating rules.
6. `ReviewService` enforces the rule of one review per user and movie.
7. If everything is valid, `Review::create()` executes the prepared statement that inserts the review into MySQL.
8. The backend returns a JSON response, and the frontend updates the displayed statistics and reviews.

How TMDB and MySQL are related in a review
`movie_id` and `movie_title` identify the movie obtained from TMDB. `user_id` identifies the user in the CineVerse session. `rating` and `comment` are the data entered by the user and stored in MySQL.

9.4 Saving a movie as a favorite: example without a backend

Not every action passes through the frontend, backend, and MySQL. Favorites provides a useful contrast because the entire flow occurs inside the browser. The user can do this even without an account.

```
User clicks the heart on a MovieCard
-> MovieCard calls onFavoriteClick(movie)
-> useFavoritesStore() obtains the shared Pinia store
-> isFavorite(movie.id) checks whether it already exists
-> addFavorite(movie) adds the Movie object to the favorites array
-> Vue detects the reactive change and updates the heart/counter
-> pinia-plugin-persistedstate persists the state in localStorage
```

```
No HTTP request is made
PHP does not run
MySQL is not modified
```

If the user reloads the page, the persistence plugin restores the saved state and the store once again has those movies available. Section 23 develops the store, the limit of 20, removal, the Favorites view, and the interface behavior.

9.5 Writing a review: complete flow summarized

Posting a review does cross all CineVerse-owned layers. The frontend already knows `movie_id` and `movie_title` because it previously obtained the movie from TMDB; it then combines those values with the rating and comment entered by the user.

```
MovieReviews.vue
-> POST /reviews + session cookie + X-CSRF-Token
-> Router
-> AuthMiddleware
-> CsrfMiddleware
-> ReviewController::create()
-> Request::json()
-> Validator + ReviewValidator
-> ReviewService::create()
-> Review::existsReviewByUserIdAndMovieId()
-> Review::create()
-> PDO INSERT INTO reviews
-> MySQL
-> Response::success("REVIEW_CREATED")
-> frontend requests review page 1 again
-> the user's review, the new average, and the new total appear
```

Section 26 develops each step, shows what is sent, which column receives each value, and what happens when reviews are edited, deleted, or additional reviews are loaded.

10. TypeScript in CineVerse

TypeScript is an extension of JavaScript that makes it possible to specify what type of data is expected in each part of the code. For example, CineVerse can declare that a movie id is a number, the title is text, and `poster_path` may be an image path or null when TMDB does not provide a poster. These checks help detect errors during development, before the application is opened in the browser.

In CineVerse, types also describe HTTP responses, users, reviews, filters, component props, and values returned by composables. The project enables `strict: true`, so TypeScript performs more demanding checks than a basic configuration.

10.1 Strict configuration

```
{
  "compilerOptions": {
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true
  }
}
```

With these options, the compiler can warn, for example, that code is trying to read a property that does not exist, that a value may be null, that a string was passed where a number was expected, or that a variable was declared but never used.

10.2 Domain types

Example: Movie

```
export interface Movie {
  id: number;
  title: string;
  release_date: string;
  poster_path: string | null;
  backdrop_path: string | null;
  vote_average: number;
  popularity: number;
  genre_ids?: number[];
  overview: string;
}
```

interface is one of the mechanisms TypeScript provides for describing the structure of an object. In `Movie`, `poster_path: string | null` indicates that the value can be a text path or can be absent. The interface must account for both cases instead of assuming that every movie has a poster.

10.3 Generics: `PaginatedResponse<T>`

```
export interface PaginatedResponse<T> {
  page: number;
  results: T[];
  total_pages: number;
  total_results: number;
}

const { data } = await api.get<PaginatedResponse<Movie>>(
  "/search/movie",
  { params: { query, page, language } }
);
```

`PaginatedResponse<T>` is a generic type: `T` represents the specific type of the items contained in results. When `PaginatedResponse<Movie>` is used, TypeScript knows that `results` is `Movie[]` and that `page`, `total_pages`, and `total_results` are numbers. The same pagination structure can be reused with other types without copying it.

10.4 Reusing types in filters

```
export interface BaseFilters {
  genres?: number[];
  yearFrom?: number;
  yearTo?: number;
  minRating?: number;
  maxRating?: number;
  sortBy?: string;
}

export interface DiscoverFilters extends BaseFilters {}

export interface SearchWithTextAndFilters extends BaseFilters {
  query: string;
}
```

`BaseFilters` groups the properties shared by several search types. `DiscoverFilters` reuses those properties, and `SearchWithTextAndFilters` adds `query`. In this way, genre, years, rating, and sorting are defined only once instead of being duplicated in each type.

10.5 Deriving a type from a composable

```
export type MovieFilters = ReturnType<typeof useMoviesFilters>;

export function useMoviesQuery(
  filters: MovieFilters,
  language: Ref<string>
) { ... }
```

`ReturnType` is a TypeScript utility that automatically obtains the type returned by a function. `MovieFilters` is derived from `useMoviesFilters`; if that composable changes what it returns, the type updates along with it, reducing the possibility that the two definitions diverge.

`Ref<string>` also appears in the same example. `Ref` is a reactive Vue container: it stores a value that can change and allows Vue to detect the change. `Ref<string>` indicates that the reactive value contains text; in `useMoviesQuery` it represents the active language.

10.6 TypeScript does not replace backend validation

Having TypeScript specify rating: number protects the code itself during development, but it does not turn the browser into a security boundary. Someone can manually send a request with `rating = 999` without using the interface. For that reason, the backend validates received data again, and MySQL maintains `CHECK(rating BETWEEN 1 AND 5)`.

Each layer serves a different purpose

TypeScript helps detect errors while programming. PHP Validators check the data that actually arrives over HTTP. PDO prepared statements protect MySQL access. UNIQUE, FOREIGN KEY, and CHECK constraints preserve the integrity of stored data.

10.7 Areas where typing can be strengthened

The project uses a strict configuration, but some areas are still less strongly typed. There are `Record<string, any>` values, errors caught as `any`, and some credit arrays using `any[]`. `any` means TypeScript stops checking that value precisely. It should also be remembered that an `Axios` type describes what the code expects to receive, but it does not validate at runtime that TMDB or the backend actually sent that exact structure.

10.8 Typed props and events between components

TypeScript also protects communication between Vue components. Props are values that a parent component passes to a child. Events enable the reverse direction: the child signals that something happened, and the parent decides what to do.

```
// MovieHero receives data from the parent
defineProps<{
  movie: MovieDetail;
  cast: CastMember[];
  director: { name: string } | null;
  hasTrailer: boolean;
}>();

// MovieHero notifies the parent
const emit = defineEmits(["openTrailer", "ready"]);
```

With these definitions, TypeScript can warn if the parent tries to pass the wrong type. In `MovieDetailView`, `openTrailer` is connected to the event emitted by `MovieHero`, and `ready` is used to remove the loading state once the hero has finished preparing its image.

10.9 Types for backend responses

The same approach is used with the PHP API. `reviews.service.ts` declares that `getMovieReviews()` returns `ReviewsResponse` and that `getUserReviews()` returns `UserReviewsResponse`. This allows `MovieReviews` to know that data contains stats, pagination, `user_review`, and reviews without working with completely unknown objects.

The type helps during development, but it still does not validate the actual JSON at runtime. If a server returned a different structure, TypeScript could not stop that network response; error states and backend validation therefore remain necessary.

11. How the frontend is organized with Vue

`main.ts` is the file that starts the frontend. It creates the Vue application, connects Pinia for shared state, Vue Router for navigation, Vue Query for data queries, and `i18n` for languages. It also imports the global styles and starts checking the user's session.

Main mounting process

```
const app = createApp(App);
app.use(pinia);

authStore.initialize(); // triggered, but not awaited

app
  .use(router)
  .use(VueQueryPlugin)
  .use(i18n)
  .mount("#app");
```

11.1 Checking the session when the application starts

At startup, `useAuthStore.initialize()` calls `GET /user` to determine whether the browser already has a valid session. `main.ts` starts that check in the background so public screens, such as Home or Movies, can be displayed without waiting for the backend. Routes that do depend on the session wait for the result before deciding whether to allow navigation.

Advantage

Home and Movies are not blocked by a request to `/user`. Routes that do need to know the session state wait for the result selectively.

A Promise is a JavaScript object that represents an asynchronous operation that may complete later. The implementation reuses a single Promise to prevent `main.ts` and a Vue Router guard from sending two `GET /user` requests at the same time. `initializePromise` currently remains stored even after it resolves. If the first request fails because the backend was temporarily unavailable, another call during the same SPA lifetime reuses that already-completed promise instead of automatically trying again.

Current implementation detail. `main.ts` emits the `app:ready` event after mounting Vue, but `index.html` does not register any listener for that event. The initial loading screen is currently hidden through `window.onload`, so `app:ready` has no visible effect.

11.2 Organizing files by features

The features folder groups files by feature. `auth` contains registration, login, and recovery; `movies` groups exploration, filters, details, reviews, and favorites; `person` contains actor information; and `profile` groups profile screens. `core` is reserved for shared code used by multiple features, such as Axios, the router, and languages.

11.2.1 What file types a feature can contain

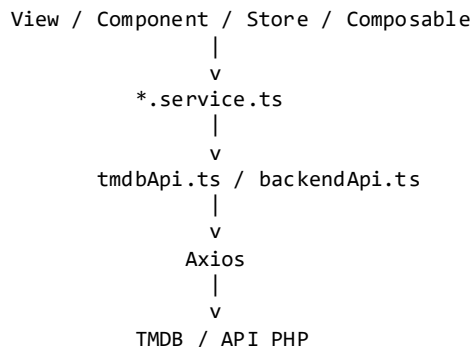
A feature groups files that participate in the same functionality, but each file type has a different responsibility. The folder name does not change how Vue works; it is a project-organization decision that keeps related code close together.

Type	Responsibility in CineVerse	Example
view	Complete screen normally associated with a route.	LoginView.vue, MoviesView.vue, ProfileView.vue
component	Reusable visual part of a screen.	MovieCard.vue, MovieReviews.vue
composable	Function that groups reusable reactive logic and can integrate Vue Query.	useMoviesQuery.ts, useMovieDetail.ts
store	Shared state available from different parts of the SPA.	useAuthStore.ts, useFavoritesStore.ts
service	Data-access operations: prepares and executes calls to TMDB or the backend through a shared HTTP client.	movies.service.ts, auth.service.ts
types	TypeScript types that describe payloads, responses, and entities.	movie.ts, auth.ts, profile.ts

11.2.2 What *.service.ts means in CineVerse

The .service.ts suffix is a project-specific organizational convention: Vue, TypeScript, and Axios do not require that name. In CineVerse, it identifies modules that centralize operations related to a data source. A service usually receives parameters, chooses an endpoint, prepares query parameters, a body, or headers, invokes a shared Axios client, and returns the received data.

As introduced in Section 1.3, an endpoint is the specific route of an API that represents an operation or resource, for example /movie/top Rated or /login. Query parameters are values included in the URL after ?, such as page=2&language=es. The body is the set of data sent inside the request, commonly with POST or PUT. Headers are additional information that accompanies the request, such as Authorization or X-CSRF-Token.



A service does not render the interface, register routes, maintain a reactive store by itself, or replace Vue Query. For example, useMoviesQuery manages queryKey, pagination, and loading states; movies.service.ts handles the specific operations that obtain movies.

```

src/features/
├── auth/
│   └── services/
│       └── auth.service.ts
├── movies/
│   └── services/
│       ├── movies.service.ts
│       └── reviews.service.ts
├── person/
│   └── services/
│       └── person.service.ts
└── profile/

```

```
└─ services/
  └─ profile.service.ts
```

Section 15 explains these five services one by one: location, HTTP client used, exported functions, source code, and examples of who invokes them.

11.2.3 Maintainability of large components

The feature-based organization is modular, but some components have grown considerably and concentrate templates, state, events, validation, and styles in a single file. As a future evolution, form logic, filters, or DOM effects should be extracted into composables and smaller components when that division genuinely reduces complexity, without fragmenting the interface based on size alone.

11.3 Components and composables

A view represents a complete screen. Components divide that screen into smaller parts and receive data or emit events. Composables are functions that group reusable Vue logic: `useMoviesFilters` controls filters, `useMoviesQuery` loads explorer pages, `useMovieDetail` combines movie data, and `usePersonDetail` obtains actor information.

11.4 How Vue reactivity works in CineVerse

Vue keeps reactive values so the interface updates when data changes without manually rebuilding the HTML. CineVerse mainly uses `ref`, `computed`, `reactive`, and `watch`.

- `ref(value)` stores a reactive value. Example: `MovieReviews` uses `formRating`, `formComment`, `isSubmitting`, and `reviews` as refs.
- `computed(...)` calculates a value from other reactive values. `favoritesCount` is recalculated from `favorites.length`, and `canSubmitForm` depends on the rating and comment length.
- `reactive({...})` groups several reactive properties in one object; `useMoviesFilters` uses it for `listType`, `text`, `genres`, `years`, `rating`, and `sorting`.
- `watch(...)` runs logic when a value changes. CineVerse uses it, for example, to debounce search, lock scrolling when a modal opens, or reset the grid when filters change.

Example: when `favorites.value.push(movie)` is executed, there is no need to find the counter in the DOM and modify it. `favoritesCount` depends on `favorites`, Vue detects the change, and template expressions that use that counter are automatically reevaluated.

11.5 Parent components, props, and emits

Components do not share local variables directly. The parent passes data through props, and the child can emit events. `MovieTabs` receives `cast`, `similar`, `movieId`, and `movieTitle` from `MovieDetailView`; it then passes `movieId` and `movieTitle` to `MovieReviews`. `MovieHero`, by contrast, emits `openTrailer` so `MovieDetailView` is the component that opens the modal.

```
MovieDetailView
-> props -> MovieHero
<- emit "openTrailer" - MovieHero

MovieDetailView
-> props movieId/movieTitle -> MovieTabs
-> props movieId/movieTitle -> MovieReviews
```

11.6 Lifecycle: onMounted and onUnmounted

`onMounted` runs after the component has been created and placed in the DOM. `MovieReviews` uses it to request the first page of reviews. `ProfileView` uses it to load the profile. `MovieDetailView` registers the Escape-key listener there to close the trailer.

`onUnmounted` runs when the component is destroyed. `MovieDetailView` removes the global keyboard listener, and `MovieResults` disconnects its `IntersectionObserver`. This cleanup prevents functions from remaining active after the user leaves a screen.

11.7 App.vue, RouterView, and when a view is recreated

App.vue keeps AppHeader and AppFooter outside RouterView so the overall frame remains in place while the central screen changes. RouterView receives the component associated with the current URL.

routeKey(route) builds a key from the route name and its parameters, but excludes lang. Therefore, moving from /es/movies/550 to /es/movies/272 changes the id and forces the view to be recreated, while changing only /es to /en preserves the instance and lets language reactivity and Vue Query update the data.

12. How data is stored and reused in the frontend

The frontend has three different needs that should be kept separate. Shared state represents information that several parts of the interface need to know at the same time; remote data comes from APIs and has a loading, error, cache, and update lifecycle; and persistence keeps certain values even when the page is reloaded. CineVerse uses a different tool for each problem instead of concentrating everything in a single store.

Need	Primary tool	Example in CineVerse
Shared state	Pinia	Authenticated user, CSRF token, and favorites.
Data obtained from APIs	TanStack Vue Query	Home, explorer, movie details, and actors.
HTTP requests	Axios	TMDB client and PHP-backend client.
Local persistence	localStorage / sessionStorage	Favorites and visual preferences; most recent filter state.

Key idea

Pinia and Vue Query do not compete with each other. Pinia stores state owned by the frontend; Vue Query manages data that comes from an API. Axios is what performs the actual HTTP request.

12.1 Pinia: shared application state

Pinia is the state-management library used by CineVerse. A Vue component can store data with ref() or reactive(), but that local state belongs to that instance. When the Header, Router, a profile screen, and a reviews view all need to know the same authenticated user, maintaining independent copies would be difficult to synchronize. A Pinia store provides a single shared point that different parts of the application can access.

12.1.1 How it is installed and what a shared store means

main.ts creates a single Pinia instance and registers it with Vue. From that point on, useAuthStore() and useFavoritesStore() access stores identified by stable names. Calling useAuthStore() from several components does not create multiple users: they all observe the same reactive state while the application is mounted.

```
const pinia = createPinia();
pinia.use(piniaPluginPersistedState);

const app = createApp(App);
app.use(pinia);
```

12.1.2 Anatomy of a store: state, derived values, and actions

The current stores use the Setup Store style. Inside defineStore, refs are created for mutable data, computed values for derived data, and functions that act as actions. A component that consumes the store does not need to know how those pieces are coordinated internally.

```
export const useAuthStore = defineStore("auth", () => {
  const user = ref<AuthUser | null>(null);
  const csrfToken = ref<string | null>(null);
  const isInitialized = ref(false);

  const isAuthenticated = computed(() => user.value !== null);

  function clear() {
    user.value = null;
    csrfToken.value = null;
  }

  return { user, csrfToken, isInitialized, isAuthenticated, clear };
});
```

`user`, `csrfToken`, and `isInitialized` are state. `isAuthenticated` does not need to be stored separately: it is calculated from `user`. `clear()` centralizes a related state change so consumers do not have to remember which properties must be cleared.

12.2 useAuthStore: representing the session in the frontend

The authentication store maintains the current user, CSRF token, and initialization state. Pinia, however, does not authenticate the user and is not the source of truth for the session. The real session lives in PHP and is associated with the browser through a cookie. Pinia keeps a reactive representation of that state so the interface can respond immediately.

```

PHP session cookie
  ↓
GET /user
  ↓
useAuthStore.initialize()
  ↓
user + csrfToken
  ↓
Header / Router / Profile / Reviews
```

12.2.1 What happens after a successful login

LoginView sends the credentials to the backend. If PHP responds with `LOGIN_SUCCESS`, the frontend receives `user` and `csrf_token` and updates the store. Because `user` is reactive, `isAuthenticated` changes automatically, and any component that depends on that value is reevaluated.

```

successful sign-in
-> setUser(response.data.user)
-> setCsrftoken(response.data.csrf_token)
-> isAuthenticated becomes true
-> Header and protected routes observe the new state
```

12.2.2 initialize(): rebuilding the session when the SPA starts

When the browser is reloaded, Pinia's in-memory state disappears. The session cookie may still be valid, so `initialize()` calls `GET /user`. If the backend recognizes the cookie, it returns the user and a CSRF token again; if there is no valid session, the store remains without a user. The implementation reuses an `initializePromise` to prevent `main.ts` and a Vue Router guard from issuing two `GET /user` requests simultaneously.

The shared Promise is useful for deduplicating initialization, although the current implementation retains that Promise even after it completes. If the first attempt failed because of a temporary backend outage, another call during the same SPA lifetime does not automatically force a new request.

12.2.3 Why the authentication store is not persisted in localStorage

Storing `user` in `localStorage` would not make that value a valid session. A user could remain visible even after the PHP session had expired. For that reason, authentication is rebuilt by querying the server. Local persistence is reserved for data that can belong to the browser without representing an access credential, such as favorites.

12.3 useFavoritesStore and persistence with pinia-plugin-persistedstate

Favorites are a different case. The store keeps Movie objects, prevents duplicates, and enforces MAX_FAVORITES = 20. They are not sent to the backend and are not linked to a MySQL account. For that reason, it is appropriate to preserve them in the browser.

```
const favorites = ref<Movie[]>([]);
const favoritesCount = computed(() => favorites.value.length);

function isFavorite(id: number) { ... }
function addFavorite(movie: Movie) { ... }
function removeFavorite(id: number) { ... }
function clearFavorites() { ... }

// store configuration
persist: true
```

12.3.1 What pinia-plugin-persistedstate provides

By itself, Pinia keeps state in memory. pinia-plugin-persistedstate watches stores configured with persist: true, serializes their state to localStorage, and restores it when the application is created again. This means an F5 reload does not erase the favorites list.

```
addFavorite(movie)
  -> favorites changes
  -> Vue updates hearts and the counter
  -> pinia-plugin-persistedstate serializes the store
  -> localStorage
```

```
F5
  -> Pinia is created
  -> the store is instantiated
  -> the plugin restores the state
  -> favorites again contains the saved movies
```

Implication

Favorites belong to the browser, not to the authenticated user. Two accounts that use the same browser profile share that local list; changing browsers or clearing localStorage can remove it.

12.4 TanStack Vue Query: remote data, cache, and loading lifecycle

TanStack Vue Query manages data that the application obtains asynchronously, especially from TMDB. This type of information is commonly called server state: CineVerse does not control its source, must request it over HTTP, and needs to know whether it is loading, whether it failed, whether a previous result can be reused, or whether there are still more pages to retrieve.

Without Vue Query, each view would have to manually implement variables for data, loading, and error; decide when to repeat a request; keep previous pages; and avoid unnecessary queries. Vue Query manages that process, while Axios continues to perform the HTTP requests and Pinia continues to manage the application's global state.

```
Component / composable
  ↓
TanStack Vue Query
  queryKey + queryFn + states
  ↓
frontend service
  ↓
Axios
  ↓
TMDB / API
```

12.4.1 VueQueryPlugin and access from composables

main.ts registers VueQueryPlugin through `app.use(VueQueryPlugin)`. That installation adds the TanStack Query infrastructure to the Vue application. Composables such as `useHomeMovies`, `useMoviesQuery`, `useMovieDetail`, and `usePersonDetail` can then create queries with `useQuery()` or `useInfiniteQuery()`.

```
app
  .use(router)
  .use(VueQueryPlugin)
  .use(i18n)
  .mount("#app");
```

12.4.2 queryKey: the identity of the data

Each query has a `queryKey`. It is not a decorative label: it identifies what information the result stored by Vue Query represents. If a material part of the key changes, Vue Query treats the new combination as different data.

```
["movies", "topRated", "es"]
  !=
["movies", "topRated", "en"]

["search", "Batman", [28], "es"]
  !=
["search", "Batman", [18], "es"]
```

The language is part of the relevant keys. Therefore, changing `/es` to `/en` does more than translate buttons: the key changes, and the queries can request the corresponding localized version from TMDB.

12.4.3 queryFn: what work runs when the query needs data

`queryFn` is the function Vue Query executes to obtain the information. In CineVerse, it normally does not contain all the HTTP logic: it calls a service, and that service uses the appropriate Axios instance. This separation keeps the composable focused on query state and the service focused on endpoints and parameters.

12.4.4 useQuery(): queries that represent one result

`useQuery()` is used when a query essentially represents one identifiable result: a Home row, movie details, actor details, or the genre catalog. It exposes data and states such as `isLoading` or `isError` and can rerun the query according to its configuration.

12.4.5 useInfiniteQuery(): results made up of multiple pages

The explorer needs to accumulate pages. `useInfiniteQuery()` keeps each response inside `data.pages` and provides `fetchNextPage()`, `hasNextPage`, and `isFetchingNextPage`. `getNextPageParam()` decides which page should be requested next and returns undefined when there are no new pages.

```
getNextPageParam: (lastPage) => {
  if (lastPage.page < lastPage.totalPages) {
    return lastPage.page + 1;
  }
  return undefined;
}
```

`MovieResults` joins the pages with `flatMap` and then applies its visual rules, such as discarding movies without a poster. This lets the user see one continuous list even though TMDB continues delivering data page by page.

12.4.6 Freshness and retention: staleTime and gcTime

`staleTime` indicates how long a result is considered fresh before being marked stale. `gcTime` indicates how long an unused query can remain before Vue Query removes it from memory. These are separate decisions: retaining data does not necessarily mean considering it fresh.

Area	Observed configuration / intent
Home	Three independent queries. Data is considered fresh for 5 minutes, and the previous version is temporarily retained when the language changes.
Explorer	useInfiniteQuery; staleTime and gcTime set to 0 to prioritize current results and avoid retaining inactive queries.
Movie details	Key by id and language; retry=false; first obtains the details and then credits, videos, and similar movies.
Actor	Key by id and language; retry=false; the response includes movie_credits.

12.4.7 keepPreviousData and visual continuity

On the Home page, changing the language changes the queryKey. While the new response is arriving, keepPreviousData temporarily preserves the previous result instead of immediately clearing the carousel. It does not prevent the new request: it prevents an unnecessary visual jump while that request completes.

12.4.8 enabled, retries, and refetching

Each query's options make it possible to decide when it may run and when it should be repeated. enabled can prevent a query until an id or valid condition exists; retry controls automatic retries; refetchOnWindowFocus and refetchOnMount determine whether returning to a window or mounting a component should trigger a new request.

Current implementation detail

In useMoviesQuery.ts, the enabled expression ends with `|| true`. As a result, it is currently always enabled, and the preceding conditions do not determine the outcome. This is an implementation detail that can be simplified or corrected without changing Vue Query's overall role.

12.4.9 Real example: the three Home queries

The Home page does not wait to gather all data in a single operation. It creates independent queries for trending, top-rated, and now-playing movies. If trending returns first, that row can display content while the others continue loading. Each queryKey includes the language, so ES and EN maintain separate results.

```

HomeView
-> useHomeMovies(locale)
  -> useQuery(trending)
  -> useQuery(top_rated)
  -> useQuery(now_playing)
-> each query updates its own data
-> HomeRow / HomeCarousel

```

12.4.10 Pinia, Vue Query, and Axios: who does what

Tool	Responsibility in CineVerse	Not to be confused with
Pinia	Shared state owned by the SPA.	Cache of remote responses.
Vue Query	Lifecycle, cache, pagination, and states of remote data.	The HTTP client that sends the request.
Axios	Builds and sends HTTP requests to TMDB or the PHP backend.	A global store or caching system.
Persistedstate	Persists selected Pinia stores in the browser.	The server's authenticated session.

12.5 sessionStorage and localStorage

Data	Where it is stored	Duration / purpose
Most recent filter query	sessionStorage	Lives in the current tab and allows filters to be rebuilt when returning to the explorer.
Favorites	Pinia + localStorage	Survives reloads and new browser sessions.
Dismissed Firefox banner	localStorage	Prevents the notice from being shown again after it is closed.
Panel order and visual state	localStorage	Restores interface preferences across visits.

`sessionStorage` and `localStorage` are browser mechanisms, not backend databases. `sessionStorage` is isolated per tab; `localStorage` can persist after the site is closed and reopened. Neither replaces MySQL for information that must belong to an account or guarantee server-side integrity.

12.6 Specialized interface libraries

In addition to Pinia, Vue Query, Router, `i18n`, and `Axios`, CineVerse uses specialized libraries to implement specific interface controls. Not all of them are registered globally with `app.use()`: `Swiper`, `vue-multiselect`, `vue-slider-component`, `vuedraggable`, and `lucide-vue-next` are imported only in the components that need them. The idea is to delegate a specific interaction to a library while keeping business logic and screen state in CineVerse.

12.6.1 Swiper: Home-page carousels

`Swiper` provides the engine for the main carousel. `HomeCarousel.vue` imports the `Swiper` and `SwiperSlide` components and the `Navigation`, `Autoplay`, and `EffectFade` modules. The library handles movement between slides, autoplay, and transitions; CineVerse retains control over when the carousel is actually ready and which movie is displayed.

```
import { Swiper, SwiperSlide } from "swiper/vue";
import { Navigation, Autoplay, EffectFade } from "swiper/modules";

<Swiper
  :modules="[Navigation, Autoplay, EffectFade]"
  effect="fade"
  :autoplay="{ delay: 3500, disableOnInteraction: false }"
  @swiper="onSwiper"
  @slideChange="onSlideChange"
>
  <SwiperSlide v-for="(movie, i) in movies" :key="movie.id">
    <MovieCardCarousel
      :movie="movie"
      :priority="i === 0"
      @loaded="i === 0 && onImageLoaded()"
    />
  </SwiperSlide>
</Swiper>
```

The `:modules` prop specifies which Swiper modules this instance needs. `effect="fade"` changes the transition to a fade, and `autoplay` advances the carousel automatically. The `@swiper` and `@slideChange` events return to CineVerse's own code. `HomeCarousel` stores the received instance, determines the edges through `activeIndex`, and emits `ready` only after Swiper has finished initializing and the first visible image has been painted. Swiper therefore controls the carousel, but it does not replace Home's loading logic.

12.6.2 vue-multiselect: selecting filter options

`MovieFilters.vue` uses `vue-multiselect` for catalog-type and sorting selectors. The component receives a list of objects with `label` and `value`, displays the translated text, and returns the selected object through `@select`. CineVerse then transforms that choice into the reactive value used by `useMoviesFilters` and the queries.

```
import Multiselect from "vue-multiselect";

<Multiselect
  :model-value="listTypesOptions.find((o) => o.value === listType)"
  :options="listTypesOptions"
  label="label"
  track-by="value"
  :searchable="false"
  @select="(o: ListTypeOption) => (listType = o.value)"
  @clear="listType = 'discover'"
/>
```

options contains the available alternatives; label indicates which property is shown to the user, and track-by identifies each option by value. Instead of storing the entire selector object, the @select event takes o.value and updates listType. That value ultimately participates in the URL, Vue Query's queryKey, and the decision about which TMDB endpoint to query. The library handles the visual control; the meaning of the value still belongs to CineVerse.

12.6.3 vue-slider-component: year and rating ranges

Year and rating ranges use vue-slider-component. MovieFilters keeps temporary values for control movement and copies those values to the actual filters only when dragging ends. This way, moving the mouse within the slider does not trigger a new query for every intermediate change.

```
import VueSlider from "vue-slider-component";

const sliderRating = ref([
  ratingRange.value?.[0] ?? 0,
  ratingRange.value?.[1] ?? 10,
]);

function applyRatingFilter() {
  const [min, max] = sliderRating.value;
  ratingRange.value = [min ?? 0, max ?? 10];
}

<VueSlider
  v-model="sliderRating"
  :min="0"
  :max="10"
  :interval="0.5"
  :marks="ratingMarks"
  :clickable="false"
  @drag-end="applyRatingFilter"
/>
```

v-model keeps sliderRating synchronized with the control's visual position. @drag-end runs when movement ends, and only then does applyRatingFilter() update ratingRange, which is the filter used by the rest of the application. The year slider follows the same pattern with sliderYears, yearFrom, and yearTo.

12.6.4 vuedraggable: panel reordering and persistence

vuedraggable is used to let the user change the order of sidebar panels. The library works directly with the reactive panels array. When an item is dragged, v-model changes the array order; because Vue renders the panels from that same array, the interface is reordered without manually rebuilding the DOM.

```
import draggable from "vuedraggable";

const panels = ref<PanelId[]>(
  readPanelsOrder() ?? DEFAULT_PANELS_ORDER
);

<draggable
  v-model="panels"
  item-key="id"
  :disabled="!customizingPanels"
  ghost-class="drag-ghost"
  animation="200"
>
  ...
</draggable>

watch(panels, (newOrder) => {
  writePanelsOrder(newOrder);
}, { deep: true });
```

item-key identifies each panel, and :disabled prevents drag and drop outside customization mode. After the change, a watch calls writePanelsOrder(), which serializes the array to localStorage with the key moviesFilters:panelsOrder. The new order therefore does more than change visually at that moment: it can also be restored when the user returns to the explorer.

12.6.5 lucide-vue-next: icons as Vue components

lucide-vue-next provides SVG icons as Vue components. CineVerse imports them by name inside the screens that need them, so there is no need to maintain separate SVG files or repeat the full markup for each icon.

```
import {
  MessageSquareText,
  Pencil,
  Trash2,
  AlertTriangle,
} from "lucide-vue-next";

<AlertTriangle :size="28" />
<Pencil :size="15" />
<Trash2 :size="15" />
```

The size prop changes the size of the SVG generated by the component. Button text, events, and styles still belong to CineVerse. For example, MovieReviews combines Pencil and Trash2 with the edit and delete actions; lucide-vue-next only provides the graphical representation.

12.6.6 What responsibility CineVerse retains

These libraries do not replace the application's architecture. Swiper does not decide which movies to request; Multiselect and VueSlider do not build TMDB queries; vuedraggable does not define what a panel means; and Lucide does not execute actions. Vue components connect each library to refs, props, events, composables, stores, and services. This separation allows a visual control to be changed without moving business logic into the external package.

13. Navigation without reloading the page: SPA and Vue Router

Because CineVerse is an SPA (Single Page Application), the browser loads the application once and Vue changes the visible view without downloading a complete HTML document on every navigation. Vue Router is the library that keeps the URL synchronized with the screen the application must display.

Within the project, Vue Router is not an abstract idea or a backend function. It is installed as a frontend dependency, configured in src/core/router/index.ts, registered globally in main.ts, and then used by components or composables to read the current route or initiate navigation.

Part	Location/function
Dependency	frontend/package.json → vue-router
Central configuration	frontend/src/core/router/index.ts
Global registration	frontend/src/main.ts → app.use(router)
Where the view is displayed	frontend/src/App.vue → RouterView
Use from components	useRouter(), useRoute(), router.push()
Query-string update	router.replace() in useMoviesFilters.ts

13.1 Where Vue Router is located and how it enters the application

package.json declares vue-router as a frontend dependency. npm install installs the package inside node_modules, and TypeScript can import it by package name. CineVerse's own configuration does not live in node_modules: it is in src/core/router/index.ts.

```
"vue-router": "^5.0.3"
```

main.ts imports the already configured instance and registers it with app.use(router). That registration connects Vue Router to the entire application; from that point on, RouterView, useRouter(), useRoute(), and navigation components can work with the same instance.

```
import router from "../core/router";

app
  .use(router)
  .use(VueQueryPlugin)
  .use(i18n)
  .mount("#app");
```

13.2 createRouter(), createWebHistory(), and the central instance

src/core/router/index.ts imports createRouter and createWebHistory from vue-router and creates a single central instance. createRouter() receives the complete router configuration. createWebHistory(import.meta.env.BASE_URL) enables normal URLs without # and respects the root configured by Vite. scrollBehavior() returns { top: 0 } so each navigation starts at the top of the new view.

```
import { createRouter, createWebHistory } from "vue-router";

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),

  scrollBehavior() {
    return { top: 0 };
  },

  routes: [
    // ... CineVerse routes
  ],
});

export default router;
```

Key idea

createRouter() creates and configures the router; it does not navigate by itself. Navigation occurs later, when the URL changes, the user uses a RouterLink, or the code calls router.push() or router.replace().

13.3 How a route is declared: path, name, component, params, and meta

Each object inside routes maps a URL to a view. path defines the pattern that must match, name allows the route to be referenced without manually writing the URL, and component indicates which Vue component RouterView should display. Dynamic parameters begin with :, and meta stores additional information that the guard can inspect.

```

{
  path: "[:lang(en|es)/movies",
  name: "Movies",
  component: MoviesView,
},
{
  path: "[:lang(en|es)/movies/:id(\\d+)",
  name: "MovieDetail",
  component: MovieDetailView
},
{
  path: "[:lang(en|es)/profile",
  name: "Profile",
  component: ProfileView,
  meta: { requiresAuth: true }
}

```

`[:lang(en|es)` restricts the language to `en` or `es`. In the detail route, `:id(\\d+)` accepts numeric ids only. `meta: { requiresAuth: true }` does not block the route by itself: it marks the route so the global guard applies the authentication check before navigation completes.

Route	View / purpose
<code>[:lang</code>	HomeView
<code>[:lang/movies</code>	Movie explorer
<code>[:lang/movies/:id</code>	Movie details; numeric id only
<code>[:lang/person/:id</code>	Actor details; numeric id only
<code>[:lang/favorites</code>	Local favorites
<code>[:lang/profile</code>	Profile; <code>requiresAuth</code>
<code>[:lang/profile/reviews</code>	My reviews; <code>requiresAuth</code>
<code>[:lang/login</code>	Login; <code>requiresGuest</code>
<code>[:lang/register</code>	Registration; <code>requiresGuest</code>
<code>[:lang/forgot-password</code>	Recovery; <code>requiresGuest</code>
<code>[:lang/reset-password</code>	New password

The root route behaves differently: when the user opens `/`, the redirect function checks `navigator.language`. If the browser language begins with `es`, it redirects to `/es`; otherwise it uses `/en`.

```

{
  path: "/",
  redirect: () => {
    const browserLang = navigator.language.toLowerCase();

    if (browserLang.startsWith("es")) {
      return "/es";
    }

    return "/en";
  },
}

```

13.4 Guards: how `requiresAuth` and `requiresGuest` become a decision

A guard is a function that runs before navigation is completed. CineVerse uses a global guard with `router.beforeEach()`. It first synchronizes `:lang` with `vue-i18n`. It then waits for session reconstruction only when the route needs to know the authentication state: `requiresAuth` protects `Profile` and `MyReviews`; `requiresGuest` prevents a user who already has a session from opening `Login`, `Register`, or `ForgotPassword`.

```

router.beforeEach(async (to) => {
  const lang = to.params.lang as string;

  if (lang === "en" || lang === "es") {
    i18n.global.locale.value = lang;
  }

  if (to.meta.requiresAuth || to.meta.requiresGuest) {
    const authStore = useAuthStore();

    if (!authStore.isInitialized) {
      await authStore.initialize();
    }

    if (to.meta.requiresAuth && !authStore.isAuthenticated) {
      return {
        name: "Login",
        params: { lang: lang || i18n.global.locale.value }
      };
    }

    if (to.meta.requiresGuest && authStore.isAuthenticated) {
      return {
        name: "Home",
        params: { lang: lang || i18n.global.locale.value }
      };
    }
  }

  return true;
});

```

to represents the route that is being opened. to.meta contains the flags defined in the route declaration. If the session has not yet been reconstructed, the guard waits for authStore.initialize(); if the decision allows navigation, it returns true, and otherwise it returns a navigation object to Login or Home.

```

route with meta.requiresAuth
  ↓
router.beforeEach()
  ↓
useAuthStore()
  ↓
authStore.initialize() if needed
  ↓
authStore.isAuthenticated
  ↓
allow navigation or redirect to Login

```

13.5 404 routes and invalid parameters

Catch-all routes appear after the valid routes. `/:lang(en|es)/:pathMatch(.*)*` preserves the language when an address beginning with `/es` or `/en` does not match any screen; `/:pathMatch(.*)*` catches everything else. Because movie and person routes accept only numeric ids, an address such as `/es/movies/abc` reaches `NotFoundView` without sending an unnecessary query to TMDB.

13.6 How a component invokes Vue Router: `useRouter()`, `useRoute()`, and `router.push()`

The central configuration only defines the rules. Components use those rules through Vue Router composables. `MovieCard.vue` imports `useRouter()` for navigation and `useRoute()` to read the language from the current URL. When the user clicks a card, `goToMovie()` navigates by route name and fills in the lang and id parameters.

```
import { useRoute, useRouter } from "vue-router";

const router = useRouter();
const route = useRoute();

function goToMovie(id: number) {
  router.push({
    name: "MovieDetail",
    params: {
      lang: route.params.lang,
      id,
    },
  });
}
}
```

`useRouter()` returns the instance used to initiate navigation. `useRoute()` exposes the active route and its parameters. `router.push()` adds a new entry to the browser history. Using `name: "MovieDetail"` avoids constructing a URL manually and lets Vue Router apply the pattern declared in `router/index.ts`.

Filters use a different operation: `router.replace({ query })`. `replace` changes the current URL without adding a new entry to browser history. As a result, moving a slider several times does not force the user to press Back once for every intermediate value.

```
// useMoviesFilters.ts
router.replace({ query });
```

13.7 RouterView and routeKey: where the selected view appears

`App.vue` contains `RouterView`, which is the point where Vue Router provides the component corresponding to the current URL. CineVerse uses the `RouterView` slot to obtain Component and route, and applies a calculated key. `routeKey()` excludes `lang`: changing only the language reuses the current view; changing an `id` produces a different key and forces the component to mount again.

```
function routeKey(route: RouteLocationNormalizedLoaded) {
  const { lang, ...rest } = route.params;
  return `${route.name as string}-${JSON.stringify(rest)}`;
}

<RouterView v-slot="{ Component, route }">
  <component
    :is="Component"
    :key="routeKey(route)"
  />
</RouterView>
```

This prevents movie data from being accidentally preserved when moving, for example, from `/es/movies/550` to `/es/movies/272`. By contrast, `/es/movies/550 → /en/movies/550` keeps the same key and allows `vue-i18n` and reactive queries to update the language without destroying the entire view.

13.8 Complete example: from MovieCard to MovieDetailView

```
User clicks MovieCard with id 550
-> @click executes goToMovie(550)
-> router.push({ name: "MovieDetail", params: { lang, id: 550 } })
-> beforeEach synchronizes the language and applies guards if required
-> Router resolves /es/movies/550
-> RouterView receives MovieDetailView
-> routeKey contains id=550
-> Vue mounts the view
-> useMovieDetail(550, locale)
-> service / Axios / TMDB
```

In this flow, Vue Router does not retrieve the movie data. Its responsibility ends with resolving the navigation and displaying `MovieDetailView`. Information loading is then delegated to the composable, Vue Query, the services, and Axios.

14. Languages: vue-i18n and language switching

i18n is an abbreviation used in development for internationalization. CineVerse can display its interface in Spanish or English, but switching languages affects more than visible words such as “Inicio” or “Películas.” It also changes the lang parameter in the URL, changes the language used in certain TMDB queries, adapts the format of some dates, and determines the language of verification and recovery emails sent by the backend.

CineVerse uses vue-i18n for its own interface text. Spanish translations are in frontend/src/core/i18n/es.json and English translations are in frontend/src/core/i18n/en.json. The following examples follow the complete path from what Carlos does to the file and function that produce the change.

14.1 Guided example: Carlos changes the menu from Spanish to English

Carlos is using CineVerse in Spanish. In the header he sees Inicio, Películas, and Favoritos; the search box shows “Buscar una película,” and ES is active in the language selector.



Spanish interface before the language change: navigation, search box, and ES / EN selector.

Carlos clicks EN. The first piece that receives that action is AppHeader.vue: the button does not translate text by itself; instead, it calls a function that asks Vue Router to navigate to the same screen using a different value for lang.

Project location: frontend/src/layouts/components/AppHeader.vue · Component · ES / EN selector and start of the language switch

```
<button
  @click="switchLang('en')"
  :class="{ active: locale === 'en' }"
  :disabled="locale === 'en'"
>
  EN
</button>
```

The @click directive indicates what Vue should execute when the user clicks the button. In this case, switchLang("en") is invoked. The button is also marked active when locale equals "en" and remains disabled while that language is already selected.

Project location: frontend/src/layouts/components/AppHeader.vue · Component · switchLang() function

```
function switchLang(lang: string) {
  router.push({
    name: route.name as string,
    params: { ...route.params, lang },
  });
}
```

switchLang() preserves the route name and current parameters - for example, the id of a movie or person - and replaces lang. Thus, an address such as /es/person/1892 becomes /en/person/1892 without sending Carlos to a different feature.

```
Carlos clicks EN
-> AppHeader executes switchLang("en")
-> router.push(...) preserves the route and changes lang
-> /es/... becomes /en/...
-> the Router's global guard reads lang="en"
-> i18n.global.locale.value becomes "en"
-> vue-i18n resolves the t(...) keys again
-> Vue updates the visible text
```

Project location: frontend/src/core/router/index.ts · Router · synchronization between the URL's lang parameter and vue-i18n

```
router.beforeEach(async (to) => {
  const lang = to.params.lang as string;

  if (lang === "en" || lang === "es") {
    i18n.global.locale.value = lang;
  }

  // ... authentication guards
});
```

beforeEach runs before every navigation. If the route parameter is es or en, the Router copies that value to vue-i18n's global locale. Because locale is reactive, components that use translations are automatically reevaluated.



Result after clicking EN: Home, Movies, Favorites, and Search for a movie appear in English.

Carlos remains inside CineVerse and on the same type of screen. No new complete HTML page is downloaded: the SPA preserves navigation, changes the active language, and Vue rerenders the parts that depend on that value.

14.2 How vue-i18n obtains “Películas” or “Movies”

Components do not write two versions of every phrase. Instead of manually deciding whether they should display “Películas” or “Movies,” they request a key through the t() function. vue-i18n looks up that key in the dictionary corresponding to the active locale.

Project location: frontend/src/layouts/components/AppHeader.vue · Component · use of translation keys inside the header

```
{{ t("nav.home") }}
{{ t("nav.movies") }}
{{ t("nav.favorites") }}

:placeholder="t('header.searchPlaceholder')"
```

t("nav.movies") does not contain the visible word. It only identifies which text the component needs. The actual value is in the language files.

Project location: frontend/src/core/i18n/es.json · Dictionary · CineVerse interface text in Spanish

```
"nav": {
  "home": "Inicio",
  "movies": "Películas",
  "favorites": "Favoritos"
},
"header": {
  "searchPlaceholder": "Buscar una película"
}
```

Project location: frontend/src/core/i18n/en.json · Dictionary · CineVerse interface text in English

```
"nav": {
  "home": "Home",
  "movies": "Movies",
  "favorites": "Favorites"
},
"header": {
  "searchPlaceholder": "Search for a movie"
}

t("nav.movies")
-> locale = "es" -> es.json -> "Películas"
-> locale = "en" -> en.json -> "Movies"
```

Project location: frontend/src/core/i18n/index.ts · Configuration · creation of the global vue-i18n instance

```
const i18n = createI18n({
  legacy: false,
  locale: "es",
  fallbackLocale: "en",
  messages: {
    en,
    es,
  },
});
```

locale indicates the active language. messages registers the two available dictionaries. fallbackLocale: "en" sets English as the fallback language if a key does not exist in the active language. legacy: false enables the modern vue-i18n API used with Vue 3 and useI18n().

In the current code review, es.json contains 210 terminal keys and en.json contains 208. The English file is missing auth.success.resendSuccessTitle and auth.success.resendSuccessSubtitle, used by the successful verification - resend flow. The fallback helps when an alternative translation exists, but it does not replace the need to keep both dictionaries synchronized.

14.3 Guided example: Carlos changes the language of Matt Damon's profile

The language change can also modify content that does not come from CineVerse's dictionaries. Carlos opens Matt Damon's profile with ES active. Navigation is in Spanish, and the biography received from TMDB appears in Spanish.



Matt Damon profile with ES active: navigation and biography displayed in Spanish.

Carlos clicks EN without leaving the profile. The route changes language and PersonDetailView continues using the same person id, but now receives locale="en". That locale affects more than t(...): it is also part of the person - data query.

The biography does not come from en.json. The dictionaries translate CineVerse's own text; the biography belongs to external TMDB data and is requested again with a different language value.

Project location: frontend/src/features/person/views/PersonDetailView.vue · View · obtaining the locale and calling the person-detail composable

```
const { t, locale } = useI18n();

const { data, isError } = usePersonDetail(id, locale);
```

PersonDetailView does not translate the biography. Its responsibility is to obtain the id from the route, access the active locale, and pass both values to usePersonDetail().

Project location: frontend/src/features/person/composables/usePersonDetail.ts · Composable · reactive query identified by person and language

```
return useQuery<PersonDetail>({
  queryKey: ["person-detail", id, language],
  enabled: Number.isFinite(id) && id > 0,
  retry: false,
  refetchOnWindowFocus: false,
  queryFn: async () => {
    return await getFullPersonDetail(id, language.value);
  },
});
```

The queryKey includes the id and language. When Carlos switches from ES to EN, the reactive language value changes and Vue Query treats the English details as a different query. queryFn then runs getFullPersonDetail() again with the new language.

Project location: frontend/src/features/person/services/person.service.ts · Frontend service · HTTP request for person details to TMDB

```
const { data } = await api.get<PersonDetail>(`/person/${personId}`, {
  params: {
    language,
    append_to_response: "movie_credits",
  },
});
```

person.service.ts sends language as a parameter in the request to TMDB. append_to_response="movie_credits" also requests that the filmography be included in the same response. When TMDB responds, PersonHero receives the new person object and displays person.biography.

Carlos clicks EN

- > locale changes from "es" to "en"
- > PersonDetailView passes the new locale to usePersonDetail
- > queryKey changes to ["person-detail", id, language]
- > Vue Query executes getFullPersonDetail(id, "en")
- > person.service.ts sends GET /person/{id}?language=en
- > TMDB returns the localized data again
- > PersonHero displays person.biography
- > Carlos sees the biography in English



The same profile after clicking EN: the menu changes through vue-i18n and the biography is returned by TMDB in English.

14.4 Translating the interface and localizing data are not the same thing

The previous examples use a single active language, but not all text comes from the same place. CineVerse's own interface is resolved with vue-i18n; external content is requested from TMDB with language; dates are formatted with Intl.DateTimeFormat; and emails use PHP templates in the backend.

What changes	Where it comes from	Mechanism
Inicio → Home	es.json / en.json	vue-i18n resolves t("nav.home") according to locale.
Películas → Movies	es.json / en.json	vue-i18n resolves t("nav.movies").
Buscar una película → Search for a movie	es.json / en.json	The placeholder uses t("header.searchPlaceholder").
Matt Damon biography	TMDB	PersonDetail is queried again with language="es" or "en".
Localized movie and person data	TMDB	The services send the active language as the language parameter.
Visible date format	Intl.DateTimeFormat	useLocalizedDate maps es → es-AR and en → en-US.
Verification or recovery email	PHP backend	MailService selects the .es.php or .en.php template according to language.

This separation is important for reading the project. Changing en.json does not modify the biography returned by TMDB, and changing language in a TMDB request does not by itself translate buttons written by CineVerse.

14.5 The language also changes date formatting

A person's visible date is not manually written in a fixed order either. PersonHero uses useLocalizedDate(), a shared composable that watches the current locale and creates a regional formatter.

Project location: frontend/src/features/person/components/PersonHero.vue · Component · displaying the date of birth

```
{{ formatDate(person.birthday) }}
```

Project location: frontend/src/core/composables/useLocalizedDate.ts · Composable · mapping the CineVerse language to an Intl.DateTimeFormat locale

```
const LOCALE_MAP: Record<string, string> = {
  es: "es-AR",
  en: "en-US"
};

const dateFormatter = computed(() => {
  const resolvedLocale = LOCALE_MAP[locale.value] ?? "es-AR";

  return new Intl.DateTimeFormat(resolvedLocale, {
    day: "numeric",
    month: "numeric",
    year: "numeric"
  });
});
```

With ES active, CineVerse uses es-AR; with EN active, it uses en-US. The same date can therefore be displayed in day/month/year or month/day/year order. This shows that internationalizing an interface is not simply replacing words: it can also adapt the representation of values to regional conventions.

Current implementation detail. formatDate() builds a Date object from the received string. When TMDB returns a date without a time, JavaScript may interpret it in UTC and then convert it to the local time zone. The Matt Damon screenshots show the effect: the biography mentions October 8, while the date label appears as 7/10/1970 in Spanish and 10/7/1970 in English. The ordering difference between the labels correctly reflects es-AR versus en-US; the one-day shift is an independent effect of date parsing in the current implementation.

14.6 The language also reaches the backend and emails

The same locale is reused in frontend operations that may generate an email. Suppose Carlos has CineVerse set to English and creates an account: RegisterView includes `language="en"` in the JSON sent to `POST /register`.

Project location: `frontend/src/features/auth/views/RegisterView.vue` · View · sending the active language during registration

```
const lang = locale.value as "es" | "en";

await registerUser({
  username,
  email,
  password,
  password_confirmation,
  language: lang
});
```

```
Carlos uses CineVerse in English
-> locale.value = "en"
-> RegisterView includes language: "en"
-> POST /register
-> AuthController
-> AuthValidator checks that it is "es" or "en"
-> AuthService
-> MailService
-> verify-account.en.php
-> Carlos receives the verification email in English
```

The backend does not try to infer the language from the browser, email, or user location. It uses the value explicitly received from the frontend. The same approach applies to resending verification and password recovery, where MailService chooses between the `.es.php` and `.en.php` templates. Section 25 develops the complete authentication and email process.

14.7 Current implementation details

Internationalization works in the main areas, but the current code retains some behaviors that should be distinguished from the general architecture.

- `switchLang()` preserves `route.params` - for example, a movie or person id - but does not copy `route.query`. If Carlos changes language from an explorer URL containing search text, genres, years, or other filters, those query parameters may be lost. To preserve them, navigation would also need to include `query: route.query`.

Project location: `frontend/src/features/person/components/PersonHero.vue` · Component · biography expansion-button text

```
{{ expanded ? 'Leer menos' : 'Leer más' }}
```

- “Leer más” and “Leer menos” are written directly in Spanish inside `PersonHero.vue`. For that reason, the English screenshot still shows “Leer más.” By contrast, the missing-biography message does use `t("personDetail.biographyNotAvailable")`. To fully internationalize the button, both texts would need to be moved to `es.json` and `en.json` and resolved through `t(...)`.
- The regional date formatting and the one-day shift described in 14.5 are separate issues: `Intl.DateTimeFormat` changes the visible ordering according to the locale, while the possible shift comes from how `Date` is constructed from a date without a time.

15. Communication between the frontend, TMDB, and the backend

CineVerse keeps three communications separate: the Vue frontend queries TMDB for catalog and people data; the same frontend queries the PHP API for account and review features; and the PHP backend uses PDO to read from or write to MySQL.

The browser chooses the path based on the user's action. Movies do not pass through PHP, while a session, a review, or the profile does pass through the application's own backend.

15.1 From Vue to TMDB: summarized flow

When a screen needs movies or actors, a frontend service uses `tmdbApi.ts` to query TMDB directly. That client adds the base address and Bearer token; the response returns to the frontend for Vue to render. Section 3 explains from the beginning what TMDB is, how a developer obtains access, and how an operation travels from a View to its servers.

```
Vue screen
-> movies or person service
-> tmdbApi.ts
-> TMDB
-> JSON response
-> Vue Query / view
```

PHP and MySQL do not participate in this path: the Bearer token identifies CineVerse to TMDB, and the response returns directly to the frontend.

15.2 From Vue to the PHP backend: login example

When the user submits the login form, `LoginView` calls the authentication service. `backendApi` sends `POST /login` with the credentials and allows the browser to manage the session cookie. The backend validates the input, queries users in MySQL, and returns the user and CSRF token if access is successful.

```
User clicks "Sign In"
-> LoginView.vue
-> auth.service.ts
-> backendApi.ts
-> POST /login
-> Router / AuthController
-> AuthService / User
-> PDO / MySQL
-> LOGIN_SUCCESS { user, csrf_token }
-> useAuthStore / Pinia
-> authenticated interface
```

Here the application's own API, MySQL, and the PHP session do participate. Subsequent operations that modify data combine the session cookie with X-CSRF-Token.

15.3 Axios, HTTP clients, and the services layer

This section brings together pieces that were presented separately in earlier sections. Axios is the library that executes HTTP requests; `tmdbApi.ts` and `backendApi.ts` are CineVerse files that create configured Axios clients; and `*.service.ts` files use those clients to perform specific operations.

```
View / Component / Store / Composable
|
v
service
|
v
tmdbApi / backendApi
|
v
Axios
|
v
TMDB / API PHP
```

15.3.1 Axios: what it is, where it is installed, and what responsibility it has

Axios is a JavaScript/TypeScript library for making HTTP requests. In CineVerse, it is declared as a dependency in `frontend/package.json`. When `npm install` is run, `npm` installs the package inside `frontend/node_modules/axios`. The project's own code can then import it with `import axios from "axios"`.

```
frontend/package.json
"axios": "^1.13.6"
|
```

```

      | npm install
      v
frontend/node_modules/axios/
      |
      v
import axios from "axios";

```

Axios does not decide which movie to search for, does not store Vue Query's cache, and does not know the backend's business rules. Its responsibility is to build and send HTTP requests and provide the code with the response or an error.

15.3.2 tmdbApi.ts: CineVerse Axios client for TMDB

tmdbApi.ts is a TypeScript file belonging to the CineVerse frontend; it is not part of Axios or TMDB. It is located at frontend/src/core/api/tmdbApi.ts. Its purpose is to create a single Axios instance prepared to communicate with TMDB's external API and export it so other modules can reuse it.

The file reads VITE_TMDB_BASE_URL and VITE_TMDB_TOKEN from import.meta.env. This centralizes the base URL and Bearer authentication in one place. movies.service.ts and person.service.ts import the api instance and do not need to repeat those options in every function.

```

// frontend/src/core/api/tmdbApi.ts (actual executable excerpt)
import axios from "axios";

const baseUrl = import.meta.env.VITE_TMDB_BASE_URL;
const token = import.meta.env.VITE_TMDB_TOKEN;

const api = axios.create({
  baseUrl: baseUrl,
  headers: {
    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json",
  },
});

export default api;

```

axios.create() does not send any request yet. It creates a reusable instance. baseUrl is prepended to relative endpoints, and Authorization is added by default to requests made with api. Content-Type: application/json establishes JSON as the content type for requests that send a body; it does not by itself validate the response structure.

```

tmdbApi.ts
-> export default api
    |
    +--> movies.service.ts
    +--> person.service.ts
    |
    v
  TMDB

```

15.3.3 backendApi.ts: CineVerse Axios client for the PHP backend

backendApi.ts is also a CineVerse TypeScript file. It is located at frontend/src/core/api/backendApi.ts. Its purpose is to create and export the Axios instance used for the project's PHP API. Unlike tmdbApi.ts, it does not use TMDB's Bearer token: it takes VITE_BACKEND_URL as the base address and enables withCredentials so the browser can manage the session cookie.

```

// frontend/src/core/api/backendApi.ts (actual executable excerpt)
import axios from "axios";

const backendApi = axios.create({
  baseUrl: import.meta.env.VITE_BACKEND_URL,
  headers: {
    "Content-Type": "application/json"
  },
  withCredentials: true
});

```

```
});

export default backendApi;
```

withCredentials: true allows requests made by this client to include browser-managed credentials, such as the PHP session cookie. The CSRF token is a separate piece: protected operations add it explicitly as X-CSRF-Token in the corresponding request.

```
backendApi.ts
-> export default backendApi
    |
    +--> auth.service.ts
    +--> reviews.service.ts
    +--> profile.service.ts
    |
    v
    backend PHP
```

15.3.4 The *.service.ts layer: why it exists between Vue and Axios

A service is a CineVerse TypeScript module that groups operations related to a data source. The service does not always import axios directly: it normally imports api or backendApi, meaning an instance that already contains the shared configuration. This avoids repeating baseURL, the Bearer token, or withCredentials in every operation.

Service	Location	Client used	Destination
movies.service.ts	src/features/movies/services/	api from tmdbApi.ts	TMDB
person.service.ts	src/features/person/services/	api from tmdbApi.ts	TMDB
auth.service.ts	src/features/auth/services/	backendApi	PHP API
reviews.service.ts	src/features/movies/services/	backendApi	PHP API
profile.service.ts	src/features/profile/services/	backendApi	PHP API

The usual pattern is: receive arguments -> build the endpoint, params, body, or headers -> await the Axios Promise -> extract data -> return the result. The code that calls the service can then focus on the interface, state, or cache.

15.3.5 movies.service.ts: movie catalog, search, and details

movies.service.ts is one of the central frontend files. It is located at frontend/src/features/movies/services/movies.service.ts. It groups operations that obtain movies from TMDB: trending, predefined lists, search, filters, genres, and detail data. It imports api from tmdbApi.ts, so it works with a client that already knows the base URL and Bearer token.

```
import api from "../../core/api/tmdbApi";
```

Exported function	Responsibility	Who uses it
getTrendingMovies()	Daily trending movies.	useHomesMovies.ts
getMovieList()	popular, now_playing, upcoming, or top_rated lists.	useHomesMovies.ts and useMoviesQuery.ts
searchByText()	Searches titles through /search/movie.	Used internally by searchByTextAndFilters().
searchByFilters()	Builds /discover/movie with genre, dates, rating, and sorting.	useMoviesQuery.ts
searchByTextAndFilters()	Searches by text and then applies filters/sorting locally to the received page.	useMoviesQuery.ts
getMovieGenres()	Obtains /genre/movie/list.	useMoviesGenresQuery.ts
getMovieDetail()	Obtains details and applies image fallback between languages.	useMovieDetail.ts
getMovieCredits()	Obtains cast and crew with fallback.	useMovieDetail.ts
getMovieVideos()	Obtains videos and retries without a language if no results are found.	useMovieDetail.ts
getSimilarMovies()	Obtains similar movies.	useMovieDetail.ts

Example 1: a GET with a dynamic endpoint and parameters

```
export const getMovieList = async (
  listType: MovieListType,
  page: number,
  language: string
): Promise<PaginatedResponse<Movie>> => {
  const { data } = await api.get<PaginatedResponse<Movie>>`(
    `/movie/${listType}`,
    {
      params: {
        page,
        language,
      },
    }
  );

  if (listType === "upcoming") {
    const today = new Date().toISOString().split("T")[0]!;
    data.results = data.results.filter(
      (movie) => movie.release_date >= today
    );
  }

  return data;
};
```

api is the instance created by tmdbApi.ts. .get() selects HTTP GET. /movie/\${listType} allows the endpoint to change according to the requested list. params is transformed into a query string. The service also applies a local rule for upcoming: it removes results whose date has already passed.

Example 2: converting interface filters into parameters understood by TMDB

```
export const searchByFilters = async (
  page: number,
  language: string,
  filters: DiscoverFilters = {}
): Promise<PaginatedResponse<Movie>> => {
  const params: Record<string, any> = { page, language };

  if (filters.genres?.length) {
    params.with_genres = filters.genres.join(",");
  }

  if (filters.yearFrom) {
    params["primary_release_date.gte"] = `${filters.yearFrom}-01-01`;
  }

  if (filters.minRating !== undefined) {
    params["vote_average.gte"] = filters.minRating;
  }

  const { data } = await api.get<PaginatedResponse<Movie>>`(
    "/discover/movie",
    { params }
  );

  return data;
};
```

This example shows why the service is more than a single Axios line: it translates interface values into the names and formats expected by TMDB. Genres go from an array to a comma-separated string, and the year is converted into a start date.

Example 3: a service can also transform the response

```
const data = await searchText(
  filters.query,
  page,
  language
);
```

```
let results = data.results;

// ... filters and sorting applied on the frontend ...

return {
  ...data,
  results,
  total_results: results.length,
};
```

`searchByTextAndFilters()` exists because `/search/movie` does not allow all of the explorer's advanced options to be combined. It first obtains one page from TMDb and then applies the additional filters to that page. As a result, `total_results` becomes the filtered count for that page, while `total_pages` still comes from TMDb; Section 16 explains this limitation in greater detail.

Who invokes `movies.service.ts`

```
MoviesView
  -> useMoviesQuery.ts
    -> getMovieList()
    -> searchByFilters()
    -> searchByTextAndFilters()
  -> movies.service.ts
  -> api from tmdbApi.ts
  -> Axios
  -> TMDb
```

`useMoviesQuery.ts` decides which of these operations it needs based on the text, list type, and filters. Vue Query manages `queryKey`, pagination, loading, and cache; `movies.service.ts` handles the specific data operation.

15.3.6 auth.service.ts: authentication, account, and password recovery

`auth.service.ts` is located at `frontend/src/features/auth/services/auth.service.ts`. It is the service that centralizes account-lifecycle and authentication calls to CineVerse's own API. It imports `backendApi` and does not communicate with TMDb.

```
import backendApi from "@/core/api/backendApi";
```

Function	Operation	Primary consumer
<code>registerUser()</code>	POST <code>/register</code>	<code>RegisterView.vue</code>
<code>verifyEmail()</code>	GET <code>/verify-email?token=...</code>	<code>VerifyEmailView.vue</code>
<code>loginUser()</code>	POST <code>/login</code>	<code>LoginView.vue</code>
<code>getAuthenticatedUser()</code>	GET <code>/user</code>	<code>useAuthStore.ts</code>
<code>resendVerificationEmail()</code>	POST <code>/resend-verification-email</code>	<code>LoginView.vue</code>
<code>forgotPassword()</code>	POST <code>/forgot-password</code>	<code>ForgotPasswordView.vue</code>
<code>validateResetToken()</code>	GET <code>/reset-password/validate</code>	<code>ResetPasswordView.vue</code>
<code>resetPassword()</code>	POST <code>/reset-password</code>	<code>ResetPasswordView.vue</code>
<code>logoutUser()</code>	POST <code>/logout</code> + X-CSRF-Token	<code>UserMenu.vue</code>

Example: login

```
export const loginUser = async (
  payload: LoginPayload
): Promise<LoginResponse> => {
  const { data } = await backendApi.post<LoginResponse>(
    "/login",
    payload
  );
  return data;
};
```

`backendApi.post()` receives the endpoint and payload. Axios serializes the object as a JSON body. The session cookie remains under browser management thanks to `withCredentials`, while the JSON response can contain user and `csrf_token`.

Example: rebuilding the session when the SPA starts

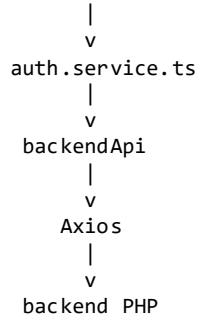
```
export const getAuthenticatedUser = async (
): Promise<AuthenticatedUserResponse> => {
  const { data } = await backendApi.get<AuthenticatedUserResponse>(
    "/user"
  );
  return data;
};
```

useAuthStore.initialize() calls getAuthenticatedUser(). The service only sends GET /user and returns the response; the store is what updates user, csrfToken, and isAuthenticated. This separates HTTP communication from global state.

Example: CSRF-protected logout

```
export const logoutUser = async (
  csrfToken: string
): Promise<LogoutResponse> => {
  const { data } = await backendApi.post<LogoutResponse>(
    "/logout",
    {},
    { headers: { "X-CSRF-Token": csrfToken } }
  );
  return data;
};
```

LoginView / RegisterView / ResetPasswordView / useAuthStore / UserMenu



15.3.7 reviews.service.ts: reading and writing reviews

reviews.service.ts is located at frontend/src/features/movies/services/reviews.service.ts. Although it belongs to the movies feature, it does not query TMDB: it uses backendApi because reviews are CineVerse-owned data stored in MySQL. This service handles review-related operations, such as obtaining the list, creating a new review, updating it, or deleting it.

Function	HTTP	Use
createReview()	POST /reviews	Create a review from MovieReviews.vue.
updateReview()	PUT /reviews/{id}	Edit one of the user's own reviews.
deleteReview()	DELETE /reviews/{id}	Delete one of the user's own reviews.
getUserReviews()	GET /reviews?page=...	Load the history in MyReviewsView.vue.
getMovieReviews()	GET /movies/{movieId}/reviews?page=...	Load reviews and statistics in MovieReviews.vue.

```
export const createReview = async (
  payload: CreateReviewPayload,
  csrfToken: string
) => {
  const { data } = await backendApi.post(
    "/reviews",
    payload,
    {
      headers: {
        "X-CSRF-Token": csrfToken,
      },
    }
  );
}
```

```
);
return data;
};
```

In `post(url, body, config)`, the first argument identifies the endpoint, the second is the JSON body, and the third allows request-specific options to be added. CineVerse adds X-CSRF-Token only to protected operations that modify data.

```
// update
backendApi.put(`/reviews/${reviewId}`, payload, {
  headers: { "X-CSRF-Token": csrfToken }
});

// delete
backendApi.delete(`/reviews/${reviewId}`, {
  headers: { "X-CSRF-Token": csrfToken }
});
```

Do not confuse frontend and backend

`reviews.service.ts` is a frontend TypeScript module that sends HTTP requests. `ReviewService.php` is a backend class that applies business rules after the request reaches the server. They share the word `service`, but they live in different layers and have different responsibilities.

15.3.8 person.service.ts: actors and filmography from TMDB

`person.service.ts` is located at `frontend/src/features/person/services/person.service.ts`. It uses the `api` instance from `tmdbApi.ts` and exports one main operation: `getFullPersonDetail()`. The `usePersonDetail.ts` composable invokes it through Vue Query.

```
import api from "@core/api/tmdbApi";

export const getFullPersonDetail = async (
  personId: number,
  language: string
): Promise<PersonDetail> => {
  const { data } = await api.get<PersonDetail>(
    `/person/${personId}`,
    {
      params: {
        language,
        append_to_response: "movie_credits",
      },
    }
  );

  return data;
};
```

`append_to_response=movie_credits` asks TMDB to include the filmography in the same person-detail response. This avoids a second separate call for movie credits.

```
PersonDetailView
-> usePersonDetail.ts
-> getFullPersonDetail()
-> person.service.ts
-> api / tmdbApi.ts
-> Axios
-> TMDB
```

15.3.9 profile.service.ts: authenticated-profile summary

`profile.service.ts` is located at `frontend/src/features/profile/services/profile.service.ts`. It is a small service dedicated to obtaining from the backend the summary needed by `ProfileView`: total review count and latest reviews. It uses `backendApi`, so the session cookie travels with the request.

```
import backendApi from "@core/api/backendApi";
```

```
import type {
  ProfileResponse
} from "../types/profile";

export const getProfile = async (
): Promise<ProfileResponse> => {
  const { data } = await backendApi.get<ProfileResponse>(
    "/profile"
  );
  return data;
};
```

ProfileView.vue imports getProfile() and executes it inside loadProfile(). The service returns the response; the view decides how to copy reviews_count and latest_reviews into its refs, how to display loading, and what to do if it receives AUTH_REQUIRED.

```
ProfileView.vue
-> loadProfile()
-> getProfile()
-> profile.service.ts
-> backendApi.get("/profile")
-> Axios
-> backend PHP
-> ProfileResponse
```

15.3.10 How to read an Axios call in the services

Once the pieces above are understood, service calls can be read systematically. The Axios method indicates the HTTP verb; the first argument is the relative endpoint; params produces a query string; in POST or PUT, the second argument is usually the body; and config allows headers or other options to be added.

Code	Meaning
api.get(url, { params })	GET to TMDB; params becomes a query string.
backendApi.get(url, { params })	GET to the application's own backend.
backendApi.post(url, body, config)	POST with a JSON body and optional configuration.
backendApi.put(url, body, config)	PUT to update resources.
backendApi.delete(url, config)	DELETE; the config may include X-CSRF-Token.
const { data } = await ...	Awaits the Promise and extracts response.data from the Axios response object.
api.get<T>(…)	T is the type TypeScript expects; it does not validate the actual JSON at runtime.

Services usually return data rather than the full Axios object. As a result, the view or composable works with the business response and is less coupled to details such as status or headers. When Axios receives an HTTP error response, it rejects the Promise; Section 15.7 shows how that error reaches a view's catch block.

15.4 Vite proxy during development

During development, VITE_BACKEND_URL=/api causes backendApi to build relative routes under /api. The variable is defined and followed step by step in 5.4.4. The Vite development server then applies the proxy rule from frontend/vite.config.ts: it forwards those routes to http://cineverse.local and removes the /api prefix before passing them to Apache.

Current state

The two Axios clients do not have a global timeout or a global error interceptor. An interceptor is a shared function that can inspect all requests or responses before they reach the rest of the code. Currently, each service or composable receives the error and decides how to handle it within its own flow.

15.5 Parts of a request in CineVerse

A request can carry information in different places. CineVerse uses the following three mechanisms, each with a different purpose.

- URL parameters: in GET /movies/550/reviews?page=2, page=2 travels after the ? character. Request::query() reads it through \$_GET.
- JSON body: POST /reviews sends movie_id, movie_title, rating, and comment inside the body. Request::json() reads php://input and runs json_decode().
- Headers: Authorization carries the Bearer token to TMDB; X-CSRF-Token carries the CSRF token in protected write operations.

The session cookie is different: it is not part of the JSON or the CSRF token. The browser manages it as an HTTP cookie, and backendApi has withCredentials: true to allow it to be sent to the backend.

15.6 Cookie, CSRF, and Bearer are not the same thing

TMDB

Authorization: Bearer <TMDB token>
-> identifies the application to TMDB

CineVerse backend

PHP session cookie
-> associates the browser with \$_SESSION

Protected backend operation

Session cookie + X-CSRF-Token
-> authentication + additional proof against CSRF

For example, GET /profile requires a session but not CSRF because it only reads information. POST /reviews requires both because it creates data. GET /movies/{movieId}/reviews is public: it requires neither a session nor CSRF, although if a cookie exists the backend can identify the user and return their own review separately.

15.7 How an Axios error reaches a view

When the backend responds with an HTTP error status, Axios rejects the Promise and control passes to catch. CineVerse views inspect error.response?.data?.code to retrieve the technical code sent by the API and convert it into a message or form error.

```
try {
  const response = await loginUser(form);

  if (response.success) {
    authStore.setUser(response.data.user);
    authStore.setCsrToken(response.data.csrf_token);
    await router.push({
      name: "Home",
      params: { lang: locale.value }
    });
  }
}
catch (error: any) {
  const code = error.response?.data?.code as string;

  const fieldError =
    authFieldErrorMap[
      code as keyof typeof authFieldErrorMap
    ];

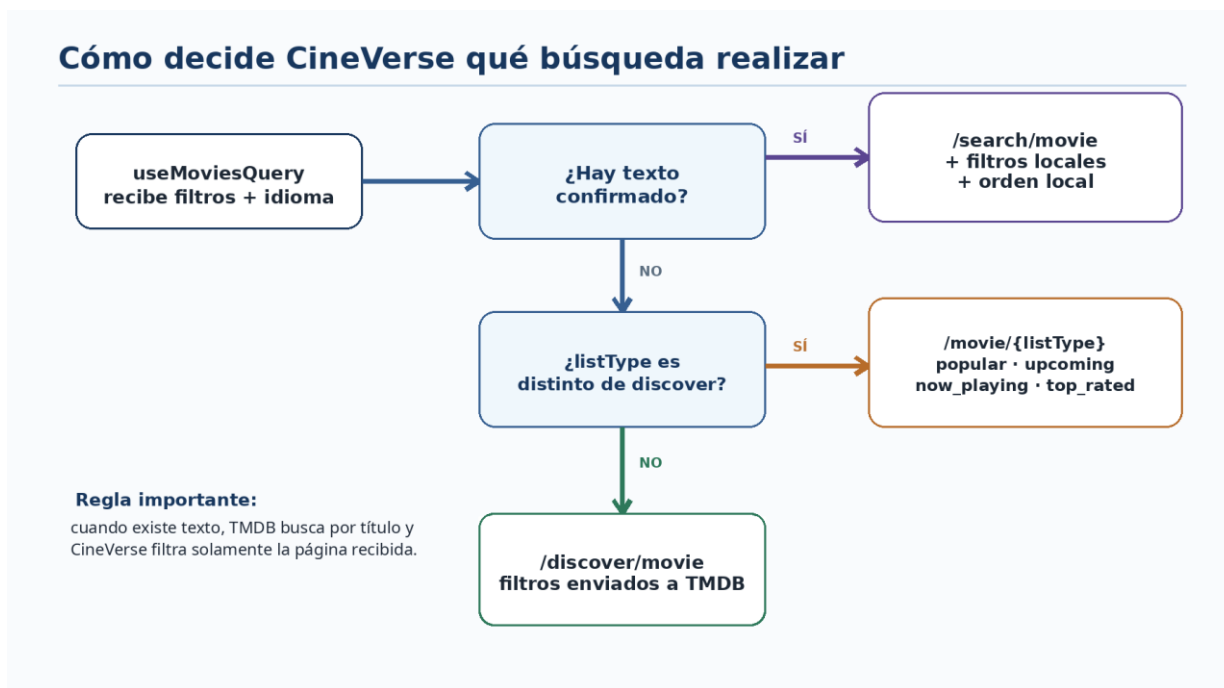
  // ... the code is converted into an interface error
}
```

error.response exists when Axios received an HTTP response representing an error. data contains that response's JSON, and code is the stable identifier produced by the backend, for example INVALID_CREDENTIALS or REVIEW_ALREADY_EXISTS. Section 28 develops the complete classification of expected and unexpected errors.

16. Movie explorer and search system

The explorer combines the filter panel, SPA navigation, and TMDB queries. The important idea is not to memorize isolated functions, but to follow what happens from the moment the user changes an option until new cards appear in the grid.

Key idea. The explorer has three query paths. Confirmed text takes priority; when there is no text, a predefined list or Discover is used. useMoviesQuery decides which path applies, and movies.service.ts executes the specific request.



Internal useMoviesQuery decision: text search, predefined catalog, or Discover.

16.1 Pieces involved

Project location: frontend/src/features/movies/views/MoviesView.vue · View · coordinates filters and results

Project location: frontend/src/features/movies/composables/useMoviesFilters.ts · Composable · maintains search and filter state

Project location: frontend/src/features/movies/composables/useMoviesQuery.ts · Composable · decides which query to run and manages pagination

Project location: frontend/src/features/movies/services/movies.service.ts · Service · builds requests to TMDB

Project location: frontend/src/features/movies/components/MovieResults.vue · Component · converts received pages into visible cards

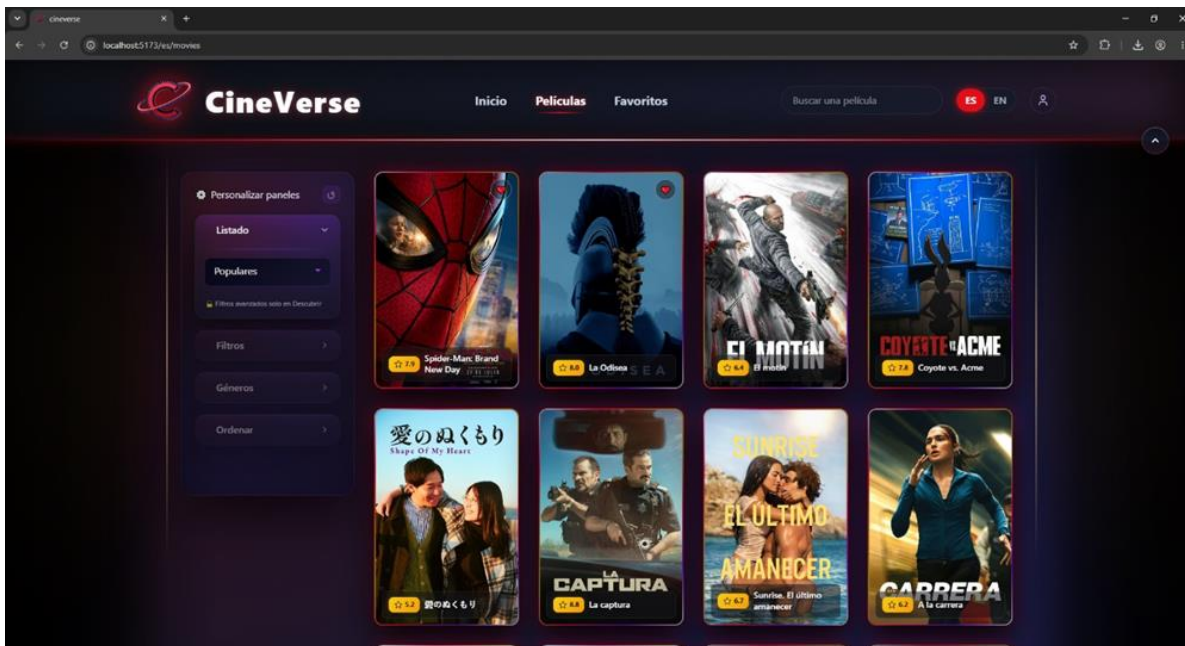
Project location: frontend/src/features/movies/components/MovieCard.vue · Component · renders each movie and contains the hover overlay

Project location: frontend/src/layouts/components/AppHeader.vue · Component · starts global search from the header

MoviesView does not know TMDB endpoints, and MovieResults does not decide what to search for. Each piece receives the output of the previous one and fulfills a specific responsibility.

16.2 Guided example: Carlos opens Movies and the first grid is built

Carlos enters from the menu at `/es/movies`. To follow the flow from scratch, in this example we assume the URL has no filters and the tab has no previous search stored in `sessionStorage`. In that case, `useMoviesFilters` uses its default values and the initial list is popular.



First explorer grid: sidebar panel and cards created from the results received from TMDB.

Project location: `frontend/src/features/movies/composables/useMoviesFilters.ts` · Composable · determines the initial filter state

```
function parseListType(value: unknown): MovieListType {
  const valid: MovieListType[] = [
    "discover", "popular", "now_playing", "upcoming", "top_rated",
  ];
  return valid.includes(value as MovieListType)
    ? (value as MovieListType)
    : "popular";
}

const hasUrlParams = Object.keys(route.query).length > 0;
const initialQuery = hasUrlParams ? route.query : readStoredQuery();
```

If the URL contains parameters, those values take priority. If it is clean, the composable attempts to restore the most recent state saved in `sessionStorage`. Only when that previous state does not exist does it fall back to the default values. `parseListType()` returns "popular" when it does not receive a valid list type.

Project location: `frontend/src/features/movies/composables/useMoviesQuery.ts` · Composable · starts the paginated query

```
initialPageParam: 1,

if (filters.listType.value !== "discover") {
  return getMovieList(
    filters.listType.value as MovieListType,
    page,
    language.value,
  );
}
```

Because this example has no confirmed text and `listType` equals "popular", the first execution uses `page=1` and enters the `getMovieList()` branch. Vue Query also maintains `isLoading`, `data`, `error`, and the information needed to request subsequent pages.

Project location: frontend/src/features/movies/services/movies.service.ts · Service · builds the popular-list request

```
const { data } = await api.get<PaginatedResponse<Movie>>(
  `/movie/${listType}`,
  { params: { page, language } }
);
```

With listType="popular", page=1, and Spanish selected, the effective endpoint is /movie/popular with page=1 and language=es. tmdbApi.ts supplies the https://api.themoviedb.org/3 base and the Authorization header. TMDB returns a paginated object with page, results, total_pages, and total_results.

Project location: frontend/src/features/movies/components/MovieResults.vue · Component · loading state and data.pages processing

```
<div v-if="isLoading" class="cinema-loading">
  ...
  {{ t('common.searchingMovies') }}
</div>
```

While the first request is pending, MovieResults does not try to draw cards: it displays the loading state. When Vue Query provides data.pages, the component processes the results and replaces that state with the grid.

```
const pages = data?.pages;
const allMovies = pages.flatMap(p => p.results);

validMovies.value = allMovies.filter(
  movie => movie.poster_path && !movie.poster_path.includes("null")
);
```

flatMap() turns the results from all loaded pages into a single list. Movies without a usable poster_path are then removed because the card needs a valid image to be part of this grid.

Important detail. MovieResults defines MIN_MOVIES = 20. If fewer than 20 remain after movies without posters are discarded and TMDB reports that another page exists, the component can automatically emit load-more to try to fill a reasonable first grid. This happens before and independently of the infinite scrolling Carlos later activates with “Show more.”

Project location: frontend/src/features/movies/components/MovieResults.vue · Component · creation of MovieCard and distribution with CSS Grid

```
<div class="movies-grid">
  <MovieCard
    v-for="(movie, index) in validMovies"
    :key="movie.id"
    :movie="movie"
    :delay="index * 30"
  />
</div>
```

v-for iterates through validMovies. On each iteration, Vue creates a MovieCard instance and passes the movie object through the :movie prop. If validMovies contains 20 movies, the component attempts to render 20 cards; the number of columns is a separate visual decision.

```
.movies-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 32px;
}
```

On desktop, the grid uses four columns. Media queries in the same component switch to three columns up to 1024 px, two up to 768 px, and one up to 480 px. Therefore, “how many movies exist” and “how many appear per row” are different concepts.

Project location: frontend/src/features/movies/components/MovieCard.vue · Component · visual poster loading and entry animation for each card

```
<div
  class="movie-card"
  :style="{ animationDelay: delay + 'ms' }"
  @click="goToMovie(movie.id)"
>
  ...
  <div v-if="!imageLoaded" class="poster-skeleton"></div>

  
</div>
```

The fact that the Movie object has already arrived does not mean the poster has been downloaded. Each MovieCard starts with imageLoaded=false and displays poster-skeleton while the browser fetches the image from image.tmdb.org. loading="lazy" allows images that are still far outside the visible area to be deferred. When the element fires @load, onPosterLoad() changes imageLoaded to true and the loaded class makes the poster appear through an opacity transition.

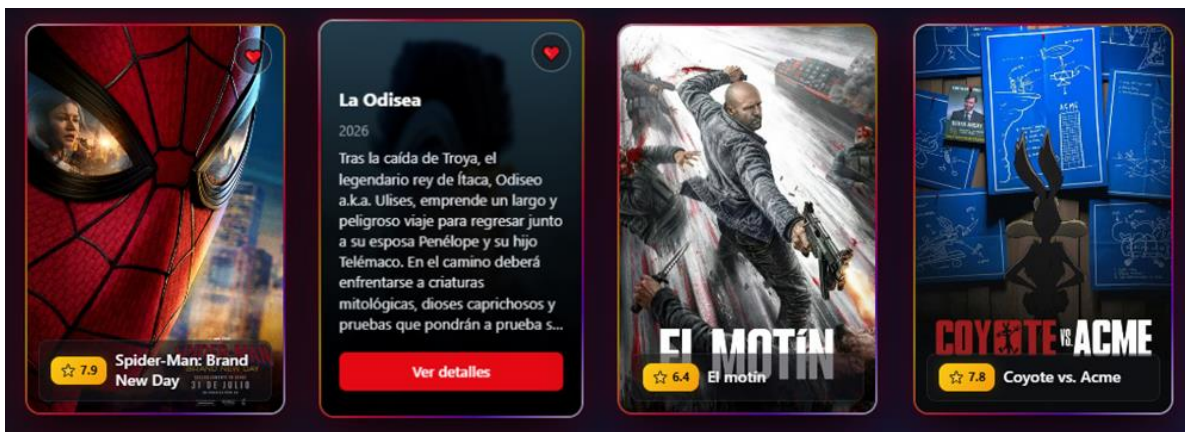
The delay prop also has a purely visual effect: MovieResults passes index * 30 milliseconds, and MovieCard applies it as animationDelay. The cards therefore do not enter at exactly the same time, but in a slightly staggered sequence. This delay does not change the data order or delay the TMDB query.

```
TMDB has already returned the Movie object
-> MovieCard mounts with imageLoaded = false
-> poster-skeleton appears
-> the browser requests the poster (loading="lazy")
-> <img> finishes loading and triggers @load
-> onPosterLoad() sets imageLoaded = true
-> the loaded class makes the poster visible
-> animationDelay = index x 30 ms staggers the visual entrance
```

```
Carlos enters Movies
-> MoviesView
-> useMoviesFilters
-> useMoviesQuery(page 1)
-> getMovieList("popular")
-> movies.service.ts
-> TMDB /movie/popular
-> Vue Query stores data.pages
-> MovieResults performs flatMap + filters poster_path
-> validMovies
-> v-for creates MovieCard x N
-> each MovieCard loads its poster and hides the skeleton when finished
-> CSS Grid lays out the cards
-> Carlos sees the grid
```

16.3 Guided example: Carlos hovers over a movie

With the grid already loaded, Carlos places the pointer over The Odyssey. The card stops showing only the poster and reveals the title, year, synopsis, and View details button.



Hover over a MovieCard: The Odyssey reveals information that was already inside the movie object received by the card.

Project location: frontend/src/features/movies/components/MovieCard.vue · Component · data displayed inside the overlay

```
<h3 class="overlay-title">{{ movie.title }}</h3>
<p class="year">{{ movie.release_date?.slice(0, 4) }}</p>
<p class="overview">
  {{ movie.overview || t("movies.noDescription") }}
</p>
<button class="details-btn">
  {{ t("movies.viewDetails") }}
</button>
```

The description is not requested from the server when the hover begins. `movie.overview` arrived with the rest of the Movie object in the response that already built the grid. MovieCard only decides when to make it visible; if `overview` is empty, it uses the `movies.noDescription` translation.

Project location: frontend/src/features/movies/components/MovieCard.vue · Component · CSS that reveals the overlay

```
.poster-overlay {
  opacity: 0;
}

.movie-card:hover .poster-overlay {
  opacity: 1;
}

.overlay-content {
  opacity: 0;
  transform: translateY(12px);
}

.movie-card:hover .overlay-content {
  opacity: 1;
  transform: translateY(0);
}
```

The `:hover` selector belongs to CSS. When the pointer enters `.movie-card`, the overlay changes from transparent to visible and its content moves smoothly upward from 12 px. This visual effect does not modify the movie object or trigger a new query.

Project location: frontend/src/features/movies/components/MovieCard.vue · Component · clicking “View details” and navigating to the detail page

```
<div
  class="movie-card"
  @click="goToMovie(movie.id)"
>
  ...
<button class="details-btn">
```

```

    {{ t("movies.viewDetails") }}
  </button>

  <button
    class="btn-favorite"
    @click.stop="onFavoriteClick(movie)"
  >
    ...
  </button>
</div>

```

The View details button does not have its own @click. When Carlos clicks it, the click event bubbles to the .movie-card container, whose @click executes goToMovie(movie.id). That function uses Vue Router to open MovieDetail while preserving the current language. The complete MovieCard → Router → MovieDetailView flow is developed in Section 13.

The favorite button provides the contrast: it uses @click.stop, so Vue stops event propagation. Carlos can click the heart without that same click accidentally opening the movie details.

Project location: frontend/src/features/movies/components/MovieResults.vue · Component · mouse tracking for 3D tilt and glare

```

<div class="movies-grid"
  @mousemove="handleGridMouseMove"
  @mouseleave="resetTiltedCard">

```

Alongside the CSS overlay, MovieResults listens for mousemove across the entire grid. handleGridMouseMove() uses closest(".movie-card") to identify the active card, getBoundingClientRect() to determine its position, and requestAnimationFrame() to limit calculations to roughly one update per frame. With that information, it modifies rotateX, rotateY, and the glare position on the .tilt-layer.

```

Carlos moves the mouse over The Odyssey
-> CSS :hover reveals poster-overlay
-> MovieCard displays title + release_date + overview
-> at the same time MovieResults receives mousemove
-> handleGridMouseMove() identifies the card
-> requestAnimationFrame() calculates tilt/glare
-> Carlos sees information + 3D effect

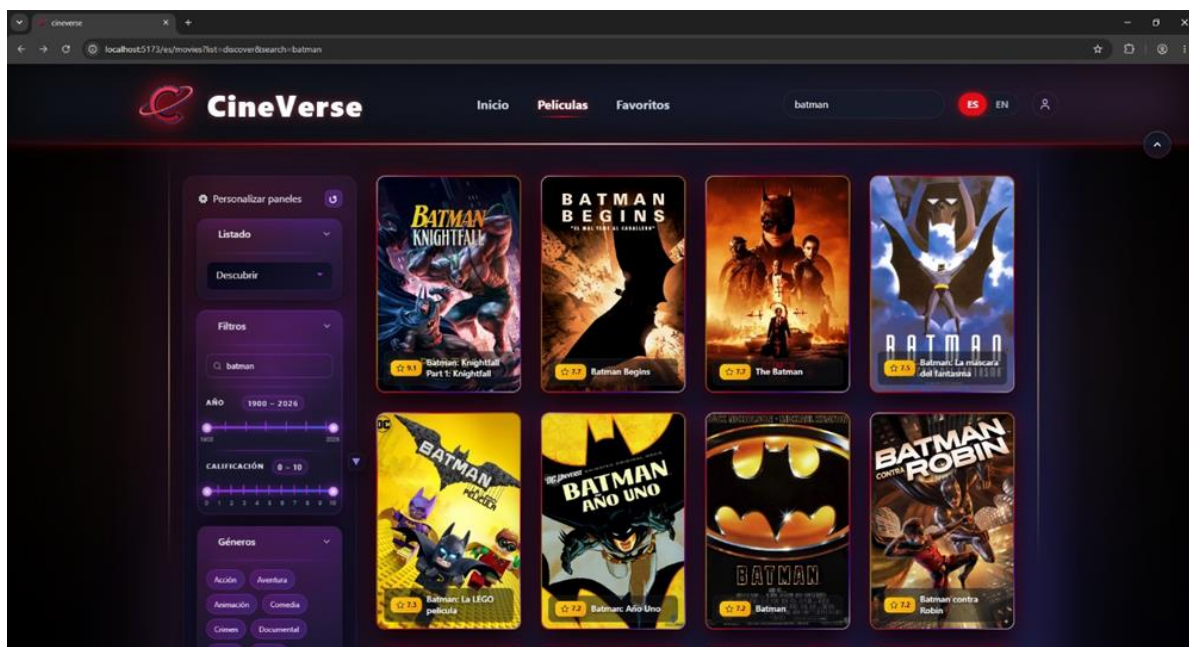
```

No new request is sent to TMDB.

Responsive behavior. On screens up to 768 px, MovieCard's CSS forces the overlay and its content to remain visible because a touch device does not have the same mouse hover behavior.

16.4 Guided example: Carlos searches for “Batman” from the header

Carlos types Batman in the top search box and submits the form. This path does not begin in MovieFilters: AppHeader first changes the URL through Vue Router, and then the explorer rebuilds its state from that URL.



Result of searching for "batman" from the header: the URL carries the criterion and the sidebar reflects it when reconstructing its filters.

Project location: frontend/src/layouts/components/AppHeader.vue · Component · handleSearch() in the top search box

```
function handleSearch() {
  const query = searchQuery.value.trim();
  const current = route.query.search;

  if (!query) return;
  if (query.length < 3) return;

  if (current === query) {
    router.go(0);
    return;
  }

  router.push({
    name: "Movies",
    params: { lang: route.params.lang },
    query: {
      search: query,
      list: "discover",
      sort: "vote_average.desc",
    },
  });
}
```

handleSearch() trims whitespace and requires at least three characters. If the search is new, Vue Router navigates to Movies with search, list="discover", and rating sorting. If the same search is already in the URL, the current behavior uses router.go(0) to force a full reload because the URL would otherwise not change.

Project location: frontend/src/features/movies/composables/useMoviesFilters.ts · Composable · reading route.query after navigation

```
const search = typeof query.search === "string"
  ? query.search.trim()
  : "";

filters.txtMovie = search;
filters.debounceQuery = search.length >= 3 ? search : "";
```

Because Batman came from the URL, `useMoviesFilters` treats it as an already-confirmed value: it updates `txtMovie` and `debouncedQuery` directly. It does not wait for the 500 ms debounce used when Carlos types in the sidebar field. That is why the sidebar in the screenshot also shows batman.

They are not the same input. AppHeader maintains its own local searchQuery, and the sidebar uses the useMoviesFilters state. In this screenshot both display “batman” because the header initiated navigation and the panel read search from route.query; there is no single shared, bidirectional ref between the two fields.

Project location: `frontend/src/features/movies/composables/useMoviesQuery.ts` · Composable · the new `queryKey` activates text search

```
if (filters.debouncedQuery.value) {
  return searchByTextAndFilters(page, language.value, {
    query: filters.debouncedQuery.value,
    ...filterOptions,
  });
}
```

`debouncedQuery` is part of the `queryKey`. When it changes to "batman", TanStack Vue Query treats that combination as a different query and runs `searchByTextAndFilters()` starting from page 1.

Project location: `frontend/src/features/movies/services/movies.service.ts` · Service · text search in TMDB

```
const { data } = await api.get<PaginatedResponse<Movie>>(
  "/search/movie",
  { params: { query, page, language } }
);
```

TMDB receives `query=batman`, `page=1`, and the active language. `searchByTextAndFilters()` can then apply on the frontend any additional filters that `/search/movie` does not support directly.

Project location: `frontend/src/features/movies/views/MoviesView.vue` · View · resetting the grid when `queryKey` changes

```
watch(
  () => filters.queryKey.value,
  () => {
    (document.activeElement as HTMLElement)?.blur();
    resetSignal.value++;
  }
);
```

`MoviesView` increments `resetSignal` when the filter identity changes. `MovieResults` watches that signal, clears the previous movies, temporarily disables infinite scrolling, and returns to the beginning. This prevents the popular cards from the first screenshot from being mixed with the Batman results.

```
Carlos types Batman in AppHeader
-> handleSearch()
-> router.push() writes search into the URL
-> useMoviesFilters reads route.query
-> txtMovie and debouncedQuery = "batman"
-> queryKey changes
-> MoviesView increments resetSignal
-> useMoviesQuery uses searchByTextAndFilters()
-> movies.service.ts calls /search/movie
-> TMDB returns results[]
-> MovieResults builds a new validMovies
-> v-for creates new MovieCards
-> Carlos sees the Batman grid
```

Two entry points, one result engine. The top search box navigates first through the URL; the sidebar field changes reactive state first and waits 500 ms. Both paths ultimately feed debouncedQuery, useMoviesQuery, movies.service.ts, and MovieResults.

16.5 The three search paths

Project location: frontend/src/features/movies/composables/useMoviesQuery.ts · Composable ·

useInfiniteQuery queryFn

```
if (filters.debounceQuery.value) {
  return searchByTextAndFilters(page, language.value, {
    query: filters.debounceQuery.value,
    ...filterOptions,
  });
}

if (filters.listType.value !== "discover") {
  return getMovieList(
    filters.listType.value as MovieListType,
    page,
    language.value,
  );
}

return searchByFilters(page, language.value, filterOptions);
```

This code is read from top to bottom. If confirmed text exists, the other branches are not reached. Without text, a list such as popular or top_rated uses TMDB's prepared catalog. Discover is the third path and sends the structured filters to /discover/movie.

User situation	Function used	What TMDB receives
Enters text	searchByTextAndFilters()	/search/movie with text, page, and language
Selects Popular / Now Playing / Upcoming / Top Rated	getMovieList()	/movie/{listType}
Uses Discover without text	searchByFilters()	/discover/movie with genre, dates, rating, and sorting

Special case for Upcoming. When listType is "upcoming", getMovieList() receives the TMDB page and removes movies whose release_date is earlier than today. This is a local project correction because TMDB may include titles that have already been released in that list.

16.6 Guided example: Carlos searches for “Batman” from the sidebar

Carlos is already in Movies and uses the sidebar search field. Unlike the top search box, this path begins in MovieFilters' reactive state and applies a debounce before confirming the text. The internal flow is as follows:

The typing debounce does apply here: Carlos is typing inside the panel rather than arriving from an already-confirmed query parameter.

- MovieFilters updates the reactive text maintained by useMoviesFilters.
- The search watcher waits 500 ms after the last keystroke. Because “Batman” has at least three characters, it copies it to debounceQuery.
- debounceQuery is part of queryKey. When that identity changes, TanStack Vue Query reruns the explorer query.
- useMoviesQuery detects that confirmed text exists and calls searchByTextAndFilters().
- movies.service.ts first executes GET /search/movie with query=Batman, page=1, and the active language.
- tmdbApi.ts supplies the base URL and Authorization header. TMDB returns a PaginatedResponse<Movie>.
- If Carlos had selected a genre, year, rating, or sorting option, searchByTextAndFilters() processes only the results from that page in the browser.
- Vue Query stores the processed page. MoviesView passes data to MovieResults, and MovieResults creates the visible cards.

Project location: frontend/src/features/movies/services/movies.service.ts · Service · searchTextAndFilters()

```
const data = await searchText(filters.query, page, language);
let results = data.results;

if (filters.genres?.length) {
  results = results.filter((movie) =>
    movie.genre_ids?.some((id) => filters.genres!.includes(id))
  );
}
```

This fragment shows why “text + filters” is different from Discover. A page first arrives from /search/movie, and then JavaScript removes from that page any movies that do not satisfy the filter. CineVerse is not filtering the entire universe of TMDB matches.

Current limitation. In a text search, total_results is replaced by the number that survived filtering on that page, while total_pages keeps the original pagination reported by TMDB. A page can therefore end up short even though more pages still exist.

16.7 Genres: Discover and text search do not mean exactly the same thing

In Discover, movies.service.ts converts [28, 12] into with_genres="28,12", and TMDB interprets that combination. In text search, the local code uses some(...): it is enough for a movie to have any of the selected IDs for it to be kept. The same visual panel therefore leads to two different mechanisms depending on whether search text exists.

17. How filters are saved and reflected in the URL

useMoviesFilters does not request movies. Its purpose is to maintain the state that represents what the user selected and synchronize it with the URL. This allows the Back/Forward buttons, F5, and a shared URL to reconstruct the same explorer state.

17.1 Three locations with different responsibilities

Location	What it keeps	What it is for
Vue reactive state	text, catalog, genres, years, rating, and sorting	Represent what is currently selected
route.query / URL	filters that differ from the default values	Share, reload, and navigate with a reproducible state
sessionStorage	moviesFilters:lastQuery	Restore the most recent query when returning to Movies with a clean URL

17.2 Guided example: Carlos selects filters and the URL changes

Carlos selects Discover, Action and Adventure, a starting year of 2000, and a minimum rating of 7. The state first changes inside Vue. A watcher waits 300 ms so it does not write a URL for every small slider movement and builds only the necessary parameters.

Project location: frontend/src/features/movies/composables/useMoviesFilters.ts · Composable · watcher that writes state → URL

```
if (filters.listType !== "popular") query.list = filters.listType;
if (filters.selectedGenres.length) query.genres = filters.selectedGenres.join(",");
if (filters.yearFrom !== null) query.yearFrom = String(filters.yearFrom);
if (filters.ratingRange[0] > 0) query.ratingMin = String(filters.ratingRange[0]);
if (filters.debouncedQuery) query.search = filters.debouncedQuery;

writeStoredQuery(query);
router.replace({ query });
```

The result can look like this:

```
/es/movies?list=discover&genres=28,12&yearFrom=2000&ratingMin=7
```

`router.replace` is used instead of `router.push`. This prevents the user from accumulating a history entry for every intermediate slider movement.

17.3 What happens if Carlos copies that URL and opens it again

In the reverse direction, the `route.query` watcher reconstructs the state: `genres="28,12"` becomes `[28, 12]`, `yearFrom` becomes a number, and `search` is treated as already confirmed. Because the value came from a URL, it does not wait for the 500 ms debounce used while a person is typing.

Project location: `frontend/src/features/movies/composables/useMoviesFilters.ts` · Composable · watcher that reads URL → state

```
const newGenres = parseGenresParam(query.genres);
if (!arraysEqual(filters.selectedGenres, newGenres)) {
  filters.selectedGenres = newGenres;
}

filters.yearFrom = parseIntParam(query.yearFrom);
const search = typeof query.search === "string" ? query.search.trim() : "";
filters.debouncedQuery = search.length >= 3 ? search : "";
```

The `syncingFromRoute` flag prevents a loop: read URL → modify filters → filter watcher writes URL again → read again. During synchronization that originated from the route, the write watcher exits and allows the flow to finish.

17.4 What takes priority when entering the explorer

- If the URL contains query parameters, the URL is the source of truth.
- If the URL is clean, `sessionStorage["moviesFilters:lastQuery"]` is checked.
- If there is no saved query either, the default values are used.

Do not confuse persistence mechanisms. The search query is stored in `sessionStorage`. Panel order or open/closed state belongs to the filter's visual preferences and is handled separately. Not all screen state is stored in the same place.

18. How more movies are loaded

TMDB returns paginated results. CineVerse does not replace the first page when the second arrives: TanStack Vue Query retains the pages already obtained, and `MovieResults` joins them to form one continuous grid.

18.1 Guided example: Carlos clicks “Show more”

- `MovieResults` executes `enableScroll()`: it enables automatic scrolling and emits `load-more`.
- `MoviesView` listens for `@load-more` and calls `fetchNextPage()` directly, a function provided by `useInfiniteQuery`.
- `getNextPageParam` examines the last response. If `lastPage.page` is less than `lastPage.total_pages`, it returns `page + 1`.
- The query runs the same search path again, now with `pageParam=2`.
- When page 2 arrives, `data.pages` contains page 1 and page 2; the first page is not lost.

Project location: `frontend/src/features/movies/views/MoviesView.vue` · View · connection between the component event and Vue Query

```
<MovieResults
  :data="data"
  :isLoading="isLoading"
  :isFetching="isFetching"
  :error="error"
  :hasNextPage="hasNextPage"
  :isFetchingNextPage="isFetchingNextPage"
  :resetSignal="resetSignal"
```

```
@load-more="fetchNextPage"
/>
```

Project location: frontend/src/features/movies/composables/useMoviesQuery.ts · Composable · getNextPageParam

```
getNextPageParam: (lastPage) => {
  if (lastPage.page < lastPage.total_pages) {
    return lastPage.page + 1;
  }
  return undefined;
}
```

18.2 How two pages become one grid

Project location: frontend/src/features/movies/components/MovieResults.vue · Component · processing data.pages

```
const allMovies = pages.flatMap(p => p.results);

validMovies.value = allMovies.filter(
  movie => movie.poster_path && !movie.poster_path.includes("null")
);
```

Conceptual example: if the first page contains 20 movies and the second contains another 20, flatMap produces a list of 40. If three lack a usable poster_path, validMovies contains 37, and that is the collection iterated by the cards' v-for.

MIN_MOVIES equals 20. If fewer than 20 usable cards remain after processing the loaded pages and another page still exists, MovieResults can emit another load to complete a reasonable initial grid.

18.3 From the button to infinite scrolling

Infinite scrolling does not begin automatically. Carlos first clicks Show more; from that point, autoScrollEnabled remains active. An IntersectionObserver watches an invisible element at the end of the grid.

Project location: frontend/src/features/movies/components/MovieResults.vue · Component · IntersectionObserver

```
if (
  entry.isIntersecting &&
  autoScrollEnabled.value &&
  !props.isFetchingNextPage &&
  !loadingMore.value &&
  props.hasNextPage
) {
  loadingMore.value = true;
  emit("load-more");
}
```

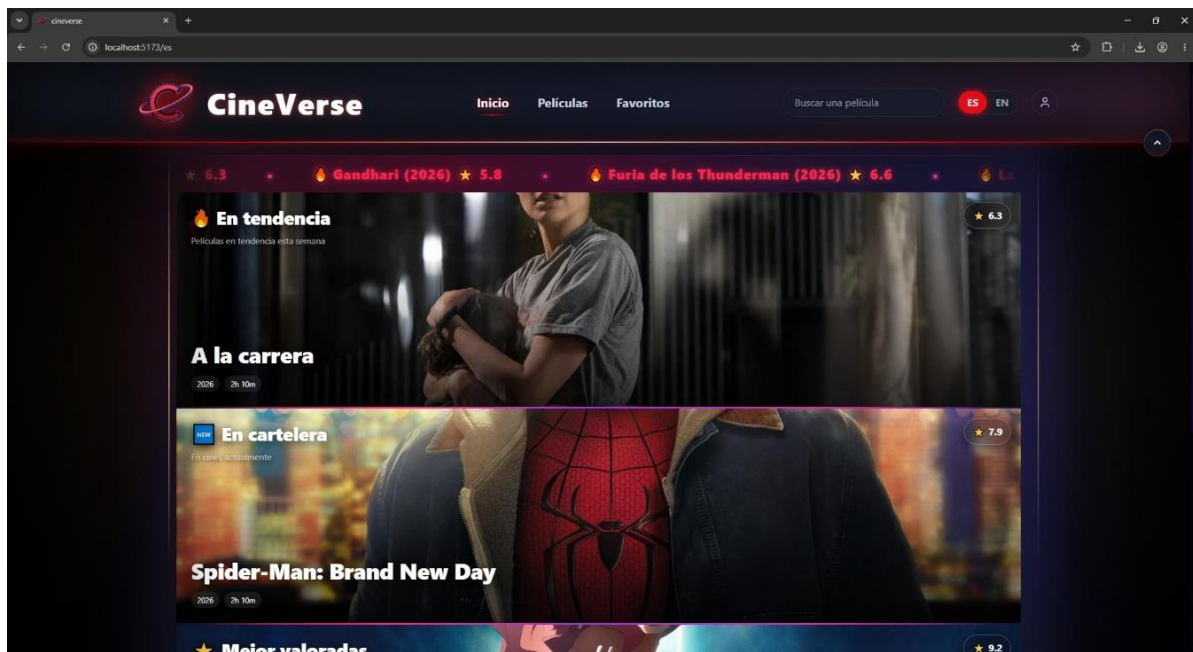
The observer uses rootMargin: "2500px". This allows the next load to begin before the user physically reaches the end, reducing the chance of encountering an empty gap while waiting.

18.4 What happens when the search changes

MoviesView increments resetSignal every time queryKey changes. MovieResults watches that signal, clears validMovies, temporarily disables infinite scrolling, and returns to the beginning. The pages from the new query then repopulate the grid. This prevents results from two different searches from being mixed.

19. Home page

The Home page uses three independent queries for trending, now playing, and top-rated movies. Their independence matters: one row can be ready and displayed while the others are still waiting for TMDB.



Home page: trending, now-playing, and top-rated movies are displayed in independent carousels.

19.1 Pieces involved

Project location: `frontend/src/features/home/views/HomeView.vue` · View · composes the rows on the page

Project location: `frontend/src/features/home/composables/useHomesMovies.ts` · Composable · creates the three Home queries; the exported function is named `useHomeMovies()`

Project location: `frontend/src/features/home/components/HomeRow.vue` · Component · manages a row's title, overlay, and ready state

Project location: `frontend/src/features/home/components/HomeCarousel.vue` · Component · initializes Swiper and reports when the first image is ready

Project location: `frontend/src/features/home/components/MovieCardCarousel.vue` · Component · renders each movie in the carousel

19.2 Guided example: Carlos opens Home

When `HomeView` mounts, `useHomeMovies(locale)` runs. The composable creates three different `useQuery` calls; none waits for the previous one to finish.

Project location: `frontend/src/features/home/composables/useHomesMovies.ts` · Composable · three independent queries

```
const trending = useQuery({
  queryKey: ["home-movies", "trending", language],
  queryFn: async () => (await getTrendingMovies(1, language.value)).results,
  ...HOME_QUERY_OPTIONS,
});

const topRated = useQuery({
  queryKey: ["home-movies", "top Rated", language],
  queryFn: async () => (await getMovieList("top Rated", 1, language.value)).results,
  ...HOME_QUERY_OPTIONS,
});

const nowPlaying = useQuery({
  queryKey: ["home-movies", "now_playing", language],
  queryFn: async () => (await getMovieList("now_playing", 1, language.value)).results,
  ...HOME_QUERY_OPTIONS,
});
```

For example, if Trending responds first, that data can already feed its HomeRow while Top Rated continues loading. This concurrency does not depend on a Promise.all: it results from three independent queries mounting at the same time.

19.3 Temporary data reuse

Project location: frontend/src/features/home/composables/useHomesMovies.ts · Composable · HOME_QUERY_OPTIONS

```
const HOME_QUERY_OPTIONS = {
  staleTime: 1000 * 60 * 5,
  refetchOnWindowFocus: false,
  placeholderData: keepPreviousData,
} as const;
```

For five minutes, results are considered fresh. Changing the language changes the queryKey; keepPreviousData temporarily preserves the previous result while the new localized version arrives instead of abruptly emptying the row.

19.4 When a row's loader disappears

Receiving the JSON is not enough for HomeRow to remove the overlay. HomeCarousel waits for two conditions: Swiper must be initialized and the first visible image must have loaded. That image load is confirmed after two requestAnimationFrame calls to give the browser an opportunity to paint it before ready is emitted.

Project location: frontend/src/features/home/components/HomeRow.vue · Component · handleReady()

```
function handleReady() {
  isReady.value = true

  if (!alreadyLoaded) {
    sessionStorage.setItem("home-loaded", "true")
  }

  setTimeout(() => {
    showOverlay.value = false
  }, 50)
}
```

sessionStorage["home-loaded"] prevents the same initial loader from being repeated if Home has already been shown during that tab session.

Current implementation detail. MovieCardCarousel displays "2h 10m" as fixed text. The Home rows do not request /movie/{id} to obtain the actual runtime; the real duration is available on the movie detail page.

19.5 How the top ticker is built

Project location: frontend/src/features/home/views/HomeView.vue · View · computed tickerItems

```
for (const movie of trend.slice(0, 5)) {
  if (!movie) continue;

  const title = movie.title;
  const year = movie.release_date?.slice(0, 4) || "----";
  const rating = movie.vote_average?.toFixed(1) || "0.0";

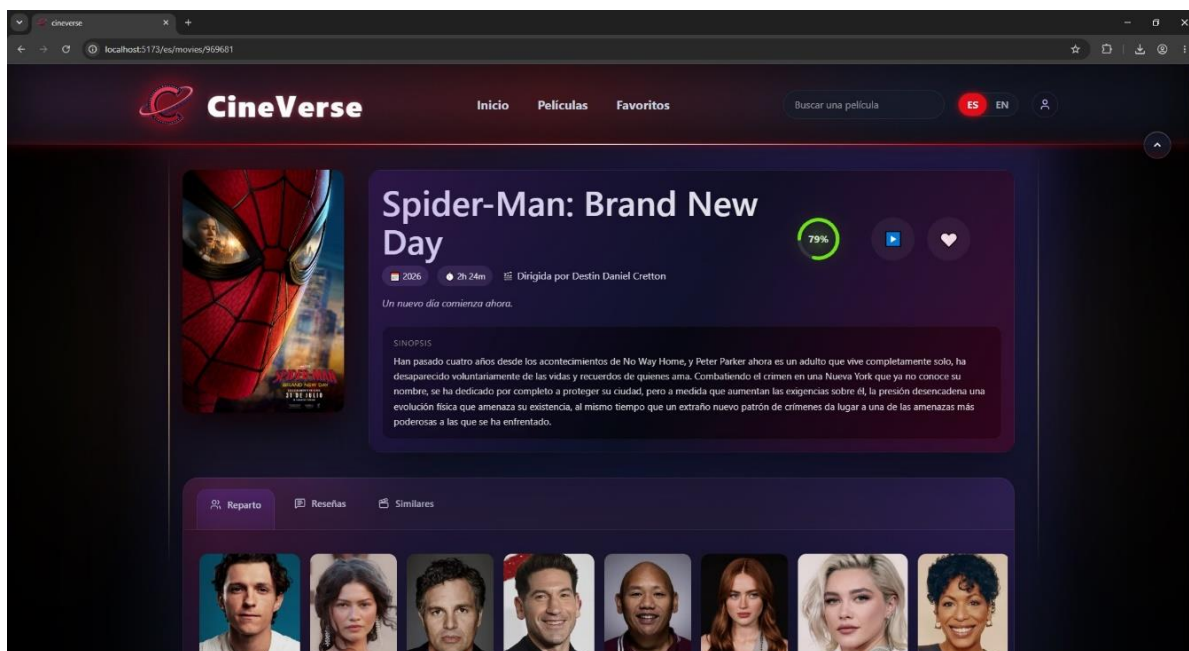
  result.push(`🔥 ${title} (${year}) ☆ ${rating}`);
}

return [...result, ...result];
```

The ticker takes the first five Trending movies and builds a phrase with title, year, and rating. Before returning, HomeView also adds three translated interface messages. Finally, it duplicates the entire array so the animation can scroll continuously without leaving a gap between the end and the beginning.

20. Movie details

The detail page is a good example of coordination among Router, Vue Query, TMDB, and components. First the main movie is confirmed; credits, videos, and similar movies are then obtained in parallel; finally, the view distributes the information between MovieHero and MovieTabs.



Movie details: main information, trailer, favorite, and cast, reviews, and similar-movie tabs.

20.1 Guided example: Carlos opens /es/movies/550

- Vue Router recognizes the detail route and mounts MovieDetailView.vue.
- MovieDetailView takes route.params.id, converts it to a number, and calls useMovieDetail(550, locale).
- useMovieDetail first requests getMovieDetail(550, language). If that request fails, the query enters an error state and the secondary queries are not launched.
- If the movie exists, the composable launches credits, videos, and similar movies inside Promise.all.
- Using the responses, it limits the cast to ten, finds crew.job === "Director", selects a YouTube trailer, and keeps the similar results.
- The processed result returns to MovieDetailView. The view extracts movie, cast, director, trailer, and similar using computed values.
- MovieHero receives the main information. MovieTabs receives cast, similar movies, movieId, and movieTitle.

Project location: frontend/src/features/movies/composables/movie-detail/useMovieDetail.ts · Composable · queryFn

```
const movieData = await getMovieDetail(id, language.value);

if (!movieData) {
  throw new Error("Movie not found");
}

const [creditsData, videosData, similarData] =
  await Promise.all([
    getMovieCredits(id, language.value),
    getMovieVideos(id, language.value),
    getSimilarMovies(id, 1),
  ]);
```

The first request is not part of Promise.all: CineVerse waits for the main details before launching the secondary requests. Once the movie is confirmed, the other three are awaited together rather than one after another.

20.2 What useMovieDetail returns to the screen

```
return {
  movie: movieData,
  cast: creditsData.cast.slice(0, 10),
  director: director ? { name: director.name } : null,
  trailer: trailerData ? trailerData.key : null,
  similar: similarData.results,
};
```

This `MovieDetailResult` object is a structure prepared for the interface. `MovieDetailView` does not need to search crew for the director again or select the trailer; those transformations have already been performed in the composable.

20.3 Why “the JSON arrived” does not mean “the details are already visible”

`MovieDetailView` maintains `heroReady` separately from `isLoading`. The query may have finished, but the view keeps the overlay until `MovieHero` confirms that the poster is ready.

Project location: `frontend/src/features/movies/components/movie-detail/MovieHero.vue` · Component · `checkReady()`

```
function checkReady() {
  if (posterLoaded.value) {
    setTimeout(() => {
      isReady.value = true;
      intro.value = true;
      emit("ready");
    }, 150);
  }
}
```

`MovieDetailView` receives that event with `onHeroReady()`. `MovieTabs` is mounted only when `heroReady` is true. This prevents the full content from appearing while the hero is still entering visually.

20.4 Cast, Reviews, and Similar are not loaded in the same way

Project location: `frontend/src/features/movies/components/movie-detail/MovieTabs.vue` · Component · tab selection with `KeepAlive`

```
<KeepAlive>
  <MovieCast v-if="activeTab === TABS.CAST" :cast="cast" />
  <SimilarMovies v-else-if="activeTab === TABS.SIMILAR" :movies="similar" />
  <MovieReviews v-else :movie-id="movieId" :movie-title="movieTitle" />
</KeepAlive>
```

Cast is the initial tab. Similar movies have already been obtained by `useMovieDetail`. Reviews is different: `MovieReviews` is created when Carlos enters that tab for the first time; its `onMounted` loads reviews from the PHP API and MySQL. `KeepAlive` preserves the instance after it has been created.

21. Trailers, similar movies, and images

This section applies the concepts defined at the beginning of the document to three secondary pieces of detail data: the trailer key, similar movies, and image paths. In each case, the flow is followed from what CineVerse receives to what Carlos ultimately sees.

21.1 Guided example: Carlos opens the trailer

Carlos is already viewing a movie's details. `useMovieDetail` previously obtained the videos from TMDB and must decide which item to use as the trailer before `MovieHero` can offer the corresponding button.

Project location: frontend/src/features/movies/composables/movie-detail/useMovieDetail.ts · Composable · trailer selection

```
const trailerData = videosData.results.find(
  (v) => v.type === "Trailer" && v.site === "YouTube"
);

trailer: trailerData ? trailerData.key : null
```

TMDB does not return the video file inside that response: it returns metadata and a key. The code takes the first item whose type is Trailer and whose site is YouTube. It does not check the official property; therefore, the accurate description is “first YouTube trailer found.”

```
Carlos opens the detail page
-> useMovieDetail receives videosData.results[]
-> searches for type="Trailer" + site="YouTube"
-> keeps trailerData.key
-> MovieHero receives that a trailer exists
-> Carlos clicks "Trailer"
-> MovieHero emits openTrailer
-> MovieDetailView changes isTrailerOpen = true
-> Vue mounts the modal and its iframe
-> Carlos sees the trailer play
```

Project location: frontend/src/features/movies/views/MovieDetailView.vue · View · modal iframe

```
<iframe
  :src="`https://www.youtube.com/embed/${trailer}?autoplay=1`"
  title="Trailer"
  allow="autoplay; encrypted-media"
  allowfullscreen
></iframe>
```

MovieHero does not open the modal directly: it emits the openTrailer event, and MovieDetailView, the parent component that controls the modal, reacts to the event. When the modal is closed, the iframe is unmounted and playback stops. The Escape key also closes it; while the modal is open, body scrolling remains locked.

21.2 Guided example: Carlos opens the Similar tab

The request for similar movies does not begin when Carlos clicks the tab. useMovieDetail already obtains that data while loading the movie details. The later click only changes which MovieTabs component is displayed on screen.

Project location: frontend/src/features/movies/services/movies.service.ts · Frontend service · getSimilarMovies()

```
const { data } = await api.get(`/movie/${movieId}/similar`, {
  params: {
    page,
    language: "en-US",
  },
});
```

Step-by-step flow:

```
Carlos opens the detail page
-> useMovieDetail calls getSimilarMovies(movieId, 1)
-> movies.service.ts
-> GET /movie/{id}/similar
-> TMDB returns results[]
-> useMovieDetail keeps the similar array
-> MovieDetailView passes it to MovieTabs
-> Carlos clicks the "Similar" tab
-> MovieTabs activates TABS.SIMILAR
-> SimilarMovies receives :movies="similar"
-> Vue creates the cards
-> Carlos sees the similar movies
```

The call currently uses a fixed language="en-US" instead of the active language. Carlos can therefore be using CineVerse in Spanish and still encounter English titles or text for similar movies. This is a characteristic of the current implementation, not of the visual component.

21.3 Guided example: how Carlos ultimately sees a poster

TMDB does not normally send image bytes inside the JSON. It returns a path such as /abc123.jpg. CineVerse combines that path with TMDB's image server and a size appropriate to each context; the browser then makes another request to download the image.

```
poster_path = "/abc123.jpg"

https://image.tmdb.org/t/p/w500/abc123.jpg
https://image.tmdb.org/t/p/original/abc123.jpg
```

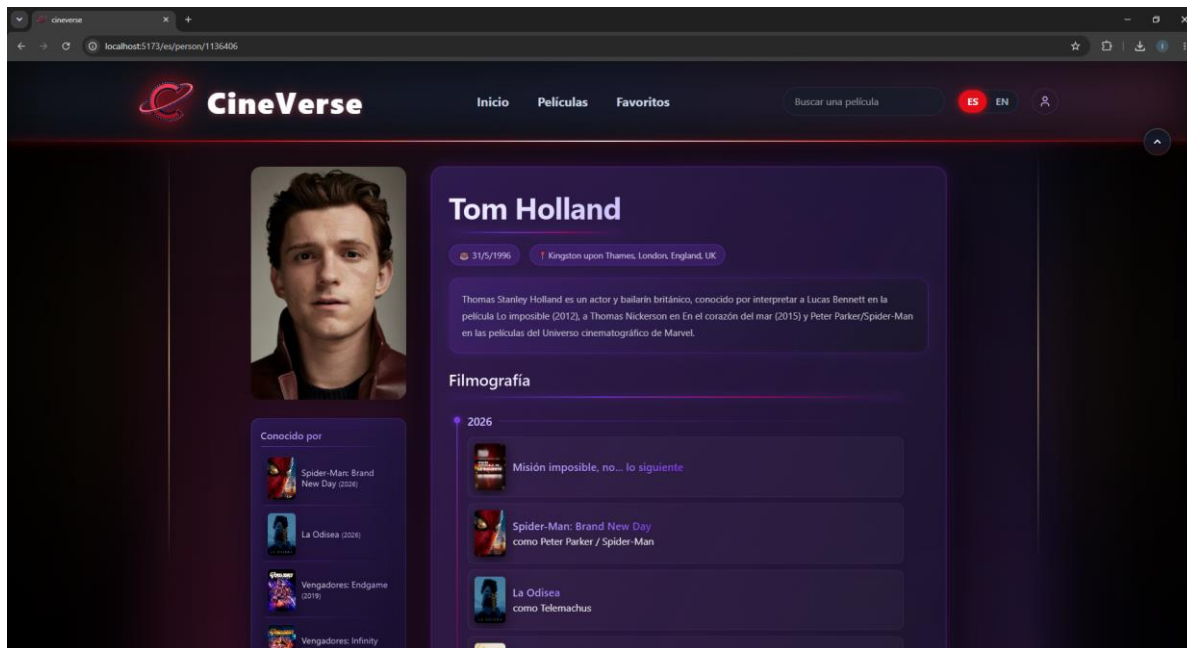
Step-by-step flow:

```
TMDB responds with JSON containing poster_path
-> the component receives the path
-> combines host + size + poster_path
-> obtains a complete image.tmdb.org URL
-> the browser requests that image
-> <img> renders it
-> Carlos sees the poster
```

The size depends on the context: MovieHero uses original for a large image; SimilarMovies uses w300 for its cards; MovieCast uses w185 for cast portraits. The fallbacks are not identical either: MovieHero can display no-image-poster.jpg; MovieCast and the person profile use no-image-avatar.jpg; MovieResults, by contrast, discards from the grid movies that do not have a usable poster_path.

22. Actors and filmography

A person-detail flow begins in a movie's cast. From that click, Vue Router changes views and a single request to TMDB returns both the person's data and their movie_credits.



Person details in CineVerse: hero, "Known for," and filmography.

22.1 Guided example: Carlos clicks an actor in the cast

- MovieCast.vue displays the cast cards. Clicking one executes goToPerson(actor.id).

- `goToPerson` uses `router.push` with `name="Person"`, preserves `lang`, and places the person's `id` in the new route.
- `PersonDetailView.vue` obtains `route.params.id` and calls `usePersonDetail(id, locale)`.
- `usePersonDetail` creates a query identified by the person `id` and language.
- `person.service.ts` executes `GET /person/{id}` with language and `append_to_response=movie_credits`.
- TMDB returns `PersonDetail` and `movie_credits` in the same response. Vue Query retains the result, and `PersonDetailView` passes it to `PersonHero`.

Project location: `frontend/src/features/movies/components/movie-detail/MovieCast.vue` · Component · `goToPerson()`

```
function goToPerson(id: number) {
  router.push({
    name: "Person",
    params: { lang: route.params.lang, id },
  });
}
```

Project location: `frontend/src/features/person/services/person.service.ts` · Service · `getFullPersonDetail()`

```
const { data } = await api.get<PersonDetail>(`/person/${personId}`, {
  params: {
    language,
    append_to_response: "movie_credits",
  },
});
```

`append_to_response` avoids making a second, separate request only for the filmography: `movie_credits` arrives embedded in the person's response.

22.2 What `PersonHero` builds from the response

`PersonHero` displays photo, name, birthday, place of birth, and biography. If `profile_path` is not valid, it uses `no-image-avatar.jpg`. The biography is shown in shortened form up to `MAX_CHARS = 600` and, when longer, a Read more / Read less control appears.

Two relevant uses in the source code: `frontend/src/features/person/components/PersonHero.vue` · Component · distribution of `movie_credits`

```
<PersonKnownFor :credits="person.movie_credits?.cast || []" />
<PersonFilmography :credits="person.movie_credits?.cast || []" />
```

Implementation detail. Both sections currently receive `movie_credits.cast`. The visual filmography is therefore built from acting credits; it does not automatically mix in crew credits.

22.3 How “Known for” is built

Project location: `frontend/src/features/person/components/PersonKnownFor.vue` · Component · `knownFor`

```
const knownFor = computed(() =>
  [...props.credits]
    .sort((a, b) => b.popularity - a.popularity)
    .slice(0, 8)
);
```

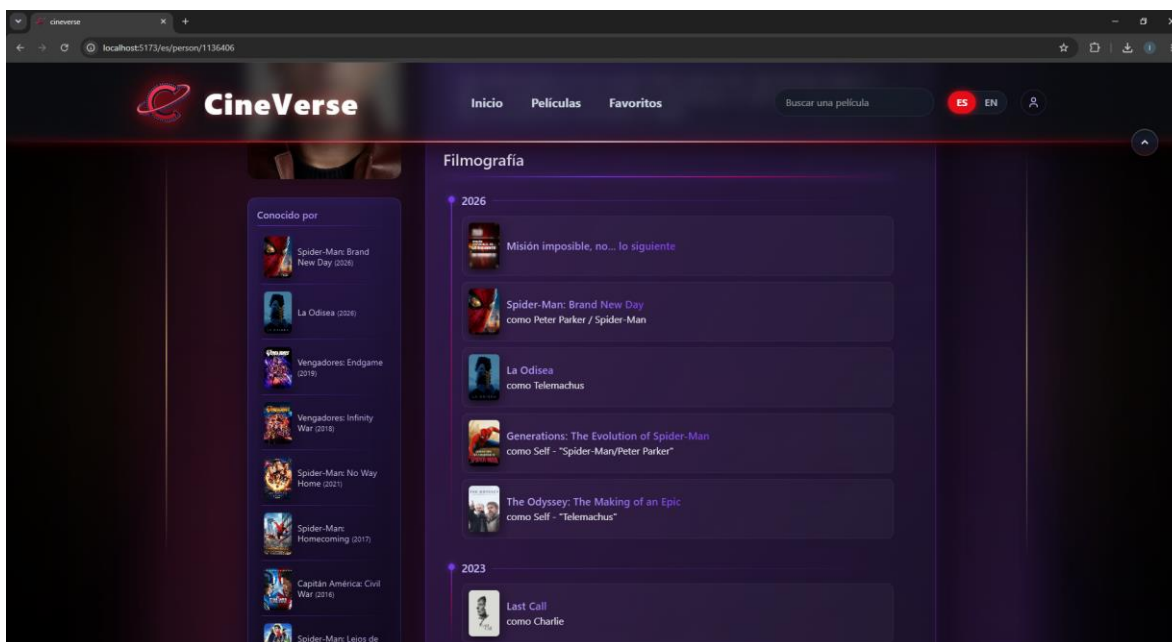
The array is copied before sorting so the received prop is not modified directly. It is then sorted by popularity from highest to lowest, and eight movies are kept. Clicking one of those cards navigates back to `MovieDetail` while preserving the current language.

22.4 How the filmography is organized

Project location: `frontend/src/features/person/components/PersonFilmography.vue` · Component · `sortedCredits` and `groupedByYear`

```
const sortedCredits = computed(() =>
  [...props.credits]
    .filter(m => m.release_date)
    .sort((a, b) =>
      new Date(b.release_date).getTime() -
      new Date(a.release_date).getTime()
    )
);
```

`PersonFilmography` discards credits without `release_date`, sorts from newest to oldest, and then groups movies by the year extracted from YYYY-MM-DD. The template iterates through those groups and builds a timeline; clicking a movie returns to that title's detail page.



Filmography in CineVerse: acting credits ordered by date and grouped by year.

23. Favorites

Favorites are stored only in the browser. They do not require an account, are not sent to the backend, and do not create records in MySQL. The Pinia store keeps Movie objects, and `pinia-plugin-persistedstate` saves them to `localStorage`.

Rule	Implementation
No duplicates	<code>isFavorite()</code> and <code>addFavorite()</code> compare <code>movie.id</code> .
Maximum	<code>MAX_FAVORITES = 20</code> .
Remove	<code>removeFavorite(id)</code> .
Clear	<code>clearFavorites()</code> .
Persistence	<code>persist: true</code> in the store.

Practical implication

Favorites belong to the current browser, not to the CineVerse account. If the user changes browsers, uses another profile, or clears local storage, the list may disappear. Reviews are unaffected because they are stored in MySQL and associated with the account.

23.1 Pinia applied to favorites

The general Pinia architecture is explained in Section 12. For Favorites, what matters is its concrete application: MovieCard, MovieHero, AppHeader, and MoviesFavoritesView query the same useFavoritesStore(), so they all observe a single reactive list during the SPA's lifetime.

The "favorites" store keeps the list of favorite movies and allows items to be added, removed, or queried. Persistence is used only to preserve this local state in the browser; it does not replace the PHP session or store data in MySQL.

23.2 How useFavoritesStore is built

```
const favorites = ref<Movie[]>([]);

const favoritesCount = computed(() =>
  favorites.value.length
);

isFavorite(id)
addFavorite(movie)
removeFavorite(id)
clearFavorites()

persist: true
```

favorites is the main state and contains complete Movie objects, not just ids. favoritesCount is derived: it is recalculated from favorites.length. isFavorite() uses some() to find a movie by id. addFavorite() returns true if it could add the movie and false if it already existed or the maximum of 20 had been reached.

removeFavorite(id) builds a new array excluding that id. clearFavorites() replaces the array with an empty one. Because favorites is reactive, every change is immediately reflected in all components that consume the store.

23.3 Complete example: Carlos saves Fight Club as a favorite

Carlos finds Fight Club and clicks the heart icon. The movie is already loaded in the card, so CineVerse does not need to query another API to save it as a favorite.

MovieCard receives a Movie object:

```
{
  id: 550,
  title: "Fight Club",
  poster_path: "/...",
  release_date: "1999-10-15",
  vote_average: ...,
  ...
}
```

Carlos clicks the heart

```
-> onFavoriteClick(movie)
-> isFavorite(550) == false
-> addFavorite(movie)
-> favorites.value.push(movie)
-> returns true
-> Vue updates the heart and favoritesCount
-> the persistence plugin stores the new state in localStorage
```

No POST is sent to the backend, and no row is created in MySQL. The movie details are not requested again for storage either: the Movie object already available to the component is persisted.

23.4 Persistence after reloading

After F5, pinia-plugin-persistedstate restores the state of the favorites store from localStorage. The general serialization and rehydration mechanism is developed in Section 12.3; here the functional consequence is that the list becomes available again without querying the backend.

Persistence belongs to the browser, not the authenticated user. Signing in with another account in the same browser profile does not create a separate list. Using another browser, another profile, or clearing localStorage can make those favorites disappear.

23.5 Limit of 20 and button behavior

MAX_FAVORITES equals 20. addFavorite() first checks for duplicates and then favorites.length >= MAX_FAVORITES. MovieCard uses the returned boolean: if addFavorite() returns false because of the limit, the card temporarily displays a tooltip for 1500 ms.

MovieHero also toggles favorites, but it currently does not use the false value returned by addFavorite(). The limit is still enforced, although the hero does not display the same visual notification that the limit was reached. This is a current difference between the two interfaces.

23.6 Removing a favorite

```

user clicks the heart on an already saved movie
-> isFavorite(id) == true
-> removeFavorite(id)
-> filter(m => m.id !== id)
-> favorites changes
-> Vue updates the interface
-> the persisted state is updated as well

```

23.7 Favorites page and “Clear all”

MoviesFavoritesView obtains favorites and favoritesCount directly from the store. It displays the current count out of 20 and creates one MovieCard for each stored object. If the list is empty, it displays an empty state with a link back to the explorer instead of the grid.

The Clear all button does not delete immediately. openConfirm() opens a modal, a watch locks body scrolling, and confirmClear() calls clearFavorites(). When the store is emptied, the grid disappears, favoritesCount becomes 0, and the plugin persists the empty state.

23.8 Consequence of storing the complete Movie object

MoviesFavoritesView reuses the stored Movie objects to build the cards. The view does not iterate over favorites and make a new TMDB request for every movie. This makes opening Favorites immediate and allows it to work with persisted data, but it also means that the data is a local copy from the moment each movie was saved.

24. How the PHP backend is organized

The CineVerse backend is written in plain PHP, without Laravel, Symfony, or another framework. The project therefore implements the complete request flow itself: it prepares the environment, recognizes the route, applies middleware, reads HTTP data, validates the input, executes business rules, queries MySQL, and builds the JSON response.

The separation into classes prevents all this work from being concentrated in a single file. Each layer answers a different question: the Router decides which endpoint applies; middleware decides whether the request may continue; the Controller coordinates HTTP; Validators check the input; the Service applies CineVerse rules; the Model communicates with MySQL through PDO; and Response builds the output.

```

HTTP request
-> public/index.php
-> bootstrap.php / PDO / Session / CORS
-> routes/api.php
-> Router
-> Middleware
-> Controller
    -> Request (reads body / query string)
    -> Validator
-> Service
-> Model
-> PDO / MySQL
-> Response JSON

```

Map of backend-owned classes

The current backend version contains 23 classes programmed specifically for CineVerse, excluding libraries installed inside vendor. The following map makes it possible to locate each class before studying a request's flow. Not all require the same depth: some are developed here and others, such as MailService and AppLogger, have dedicated sections later.

Layer	Class and location within backend/	Primary responsibility
Controller	AuthController controllers/AuthController.php	Receives HTTP operations for registration, login, verification, recovery, logout, and retrieval of the authenticated user.
Controller	ProfileController controllers/ProfileController.php	Handles GET /profile and coordinates Session, ProfileService, and Response.
Controller	ReviewController controllers/ReviewController.php	Handles creation, editing, deletion, and reading of reviews.
Core / auth	Session core/auth/Session.php	Groups PHP session handling, authenticated user, regeneration, logout, and the CSRF token.
Core / HTTP	Router core/http/Router.php	Resolves method + route, runs middleware, and calls the configured Controller.
Core / HTTP	Request core/http/Request.php	Reads the JSON body and query-string parameters.
Core / HTTP	Response core/http/Response.php	Builds uniform JSON responses and sets the HTTP status.
Core / logging	AppLogger core/logging/AppLogger.php	Handles logging of backend technical errors and events through Monolog, storing the information in log files configured by the application.
Middleware	AuthMiddleware core/middleware/AuthMiddleware.php	Prevents continuation when a route requires an authenticated user and there is no valid session.
Middleware	CsrfMiddleware core/middleware/CsrfMiddleware.php	Validates the X-CSRF-Token header in protected operations.
Middleware	CorsMiddleware core/middleware/CorsMiddleware.php	Configures CORS for the frontend and responds to OPTIONS preflight requests.
Base validation	Validator core/validation/Validator.php	Converts and validates basic types received from HTTP.
Database	Database database/Database.php	Creates and returns the PDO connection configured with backend variables.
Model	User models/User.php	Groups SQL queries for accounts, verification, and password recovery.
Model	LoginAttempt models/LoginAttempt.php	Reads and modifies the login_attempts table used for the login-attempt limit.
Model	Review models/Review.php	Executes the SQL queries related to reviews and their statistics.
Model	Profile models/Profile.php	Executes the queries that build the profile activity summary.
Service	AuthService services/AuthService.php	Applies the rules for authentication, verification, recovery, and login-attempt handling.
Service	MailService services/MailService.php	Builds and sends verification and recovery emails through PHPMailer.

Layer	Class and location within backend/	Primary responsibility
Service	ReviewService services/ReviewService.php	Applies review business rules and delegates persistence to Review.
Service	ProfileService services/ProfileService.php	Defines the six-recent-reviews rule and delegates queries to Profile.
Specific validator	AuthValidator validators/AuthValidator.php	Applies rules specific to username, email, password, and language.
Specific validator	ReviewValidator validators/ReviewValidator.php	Applies rules specific to movie_id, title, rating, and comment.

The following sections use the same separation: HTTP infrastructure first, authentication and account next, and finally reviews and profile. The goal is to make each class easy to locate both by its file and by its responsibility within the flow.

24.1 public/index.php: API entry point

backend/public/index.php is the main entry point for CineVerse's own API. Apache routes backend requests to this file, so paths such as /login, /reviews, or /reviews/15 end up there even though no PHP file with that name exists. Requests sent directly from the frontend to TMDB do not pass through index.php. Its role is not to implement every use case: it loads the shared environment, opens PDO, starts the session, applies CORS, loads the routes, and finally passes the HTTP method and path to the Router, which decides which Controller must run.

```
require_once __DIR__ . '/../core/bootstrap.php';

try {
    $pdo = Database::connect();
} catch (\PDOException $exception) {
    AppLogger::error($exception->getMessage(), [
        'controller' => 'Application',
        'method' => 'bootstrap',
        'status' => 503,
        'pdo_code' => $exception->getCode(),
    ]);

    Response::error('DATABASE_UNAVAILABLE', 503);
    exit;
}

Session::start();
CorsMiddleware::handle();
```

require_once runs the bootstrap before the request is processed. Database::connect() creates the PDO connection; if MySQL is unavailable, the exception is caught at this level because no Controller exists yet that could handle it. The session is then started and CORS is applied. This order explains why an early MySQL outage can produce a 503 response before the Router is involved.

```
$routes = require __DIR__ . '/../routes/api.php';

$method = $_SERVER['REQUEST_METHOD'];
$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

$basePath = '/cineverse/backend/public';
$path = str_replace($basePath, '', $path);

$router = new Router($routes);
$router->dispatch($method, $path, $pdo);
```

The HTTP method and path are obtained from the actual request. parse_url(..., PHP_URL_PATH) removes the query string so the Router compares only the path. The current basePath makes it possible to strip a specific installation prefix and, finally, dispatch() receives the method, path, and PDO connection.

24.2 bootstrap.php: preparing PHP before handling the route

bootstrap.php does not decide which endpoint will run. Its responsibility is to prepare the shared environment that any endpoint may need. This section presents it within the flow; .env and Composer configuration are covered in section 6.

```
require_once __DIR__ . '/../vendor/autoload.php';

date_default_timezone_set('America/Argentina/Buenos_Aires');

$dotenv = Dotenv\Dotenv::createImmutable(__DIR__ . '/../');
$dotenv->load();

$dotenv->required([
    'DB_HOST', 'DB_PORT', 'DB_NAME', 'DB_USER',
    'DB_CHARSET', 'APP_ENV', 'APP_DEBUG',
    'CINEVERSE_LOG_PATH'
]);
```

vendor/autoload.php makes it possible to use both project classes configured through PSR-4 and dependencies installed by Composer. phpdotenv reads backend/.env and places the values in \$_ENV. required() checks that the essential variables exist before the backend continues.

24.3 routes/api.php: declaring which endpoints exist

routes/api.php returns an array organized by HTTP method. Each entry describes a route, the Controller method that must run, and two security requirements. Declaring these rules in a single file allows the Router to remain generic: it does not need to know the specific logic for login, profile, or reviews.

```
"POST" => [
    "/reviews" => [
        "handler" => [ReviewController::class, "create"],
        "auth" => true,
        "csrf" => true
    ]
],

"GET" => [
    "/user" => [
        "handler" => [AuthController::class, "getAuthenticatedUser"],
        "auth" => true,
        "csrf" => false
    ]
],

"PUT" => [
    "/reviews/{id}" => [
        "handler" => [ReviewController::class, "update"],
        "auth" => true,
        "csrf" => true
    ]
]
```

handler contains the class and method that will resolve the operation. auth indicates whether user_id must already exist in the session. csrf adds the token check for protected operations that modify state. Thus, POST /reviews and PUT /reviews/{id} require both a session and CSRF, while GET /user requires a session but not CSRF because it only reads information.

24.4 Router: finding the route and executing the handler

Router.php is located at backend/core/http/Router.php. public/index.php creates this class with the route table loaded from routes/api.php and then calls Router::dispatch(). The method receives the HTTP method, the path, and the PDO connection; it selects only the routes declared for that method, checks the alternatives, and transforms dynamic parameters such as {id} into numeric capture groups in a regular expression.

```

$methodRoutes = $this->routes[$method] ?? [];

foreach ($methodRoutes as $route => $routeData) {
    $pattern = preg_replace(
        '#\[a-zA-Z_]+\#',
        '([0-9]+)',
        $route
    );

    $pattern = '#^' . $pattern . '$#';

    if (preg_match($pattern, $path, $matches)) {
        ...
    }
}

```

For example, `/reviews/{id}` is converted into a pattern that accepts `/reviews/15` but not `/reviews/15/edit`. The `^` and `$` symbols require the match to cover the entire route. The `([0-9]+)` group captures the number so it can later be passed to the Controller method.

24.4.1 How the Router invokes middleware

```

if ($routeData["auth"]) {
    if (!$authMiddleware->requireAuth()) {
        return;
    }
}

if ($routeData["csrf"]) {
    if (!$csrfMiddleware->validateCsrf()) {
        return;
    }
}

```

At this point, the Router's responsibility is what matters: it reads the `auth` and `csrf` flags declared in `routes/api.php` and, when appropriate, invokes the middleware before creating the Controller. If either one returns `false`, `dispatch()` stops immediately. This is why a request without a session or with an invalid CSRF token never enters `ReviewController::create()` or reaches business rules or SQL. The internal behavior of `AuthMiddleware` and `CsrfMiddleware` is covered in section 24.6.

24.4.2 Dynamic parameters and the Controller call

```

array_shift($matches);

$matches = array_map(
    static fn (string $value) => ctype_digit($value)
        ? (int) $value
        : $value,
    $matches
);

[$controllerClass, $method] = $routeData["handler"];
$controller = new $controllerClass($pdo);
$controller->$method(...$matches);

```

`preg_match()` stores the full URL in `$matches[0]`, so `array_shift()` removes it. Captured values arrive as strings and are converted to `int` when they contain digits only. The Router then takes the handler declared in `routes/api.php`, creates the Controller with the PDO connection, and uses `...$matches` to pass the URL parameters. For `PUT /reviews/15`, the final call is equivalent to `$controller->update(15)`. If no route matches, the Router responds with `ROUTE_NOT_FOUND` and HTTP 404.

24.5 Request: reading JSON and URL parameters

`Request.php` is located at `backend/core/http/Request.php`. It is a project-specific class that encapsulates incoming HTTP data and gives Controllers a controlled way to access the JSON body and query-string parameters.

It does not apply business rules, nor is it a stage that the Router executes before the Controller: in the actual flow, the Controller itself uses Request when it needs to read that data.

```
private ?array $body = null;

private function getJson(): array
{
    $rawBody = file_get_contents("php://input");
    $data = json_decode($rawBody, true);

    if (!is_array($data)) {
        throw new RuntimeException("INVALID_JSON_BODY");
    }

    return $data;
}

public function json(): array
{
    if ($this->body === null) {
        $this->body = $this->getJson();
    }
    return $this->body;
}

public function query(string $key, mixed $default = null): mixed
{
    return $_GET[$key] ?? $default;
}
```

php://input contains the raw body sent by the client. json_decode(..., true) converts it into an associative array. The \$body property acts as a cache inside the Request object: the JSON is processed the first time json() is called and is then reused. query() does not read the body; it reads \$_GET, for example to obtain page from /reviews?page=2.

24.6 Middleware: authentication, CSRF, and CORS

The backend's custom middleware is located in backend/core/middleware/. AuthMiddleware.php requires a valid session when a route has auth=true; CsrfMiddleware.php validates the token for operations with csrf=true; and CorsMiddleware.php configures which origin, methods, and headers the frontend may use. These layers run before the Controller: if a check fails, the requested functionality is not executed.

```
public function requireAuth(): bool
{
    if (!Session::has("user_id")) {
        Response::error("AUTH_REQUIRED", 401);
        return false;
    }
    return true;
}

public function validateCsrf(): bool
{
    $token = $_SERVER["HTTP_X_CSRF_TOKEN"] ?? "";

    if (!Session::validateCsrfToken($token)) {
        Response::error("INVALID_CSRF_TOKEN", 403);
        return false;
    }
    return true;
}
```

AuthMiddleware checks the identity associated with the session. CsrfMiddleware reads the X-CSRF-Token header and delegates the comparison to Session. Both can build the error response and return false, causing the Router to stop the flow before executing the handler. The security rationale for these checks is covered in section 27.

CorsMiddleware.php: allowing frontend communication

CorsMiddleware.php, located at backend/core/middleware/CorsMiddleware.php, is a static class that public/index.php runs before creating the Router. In the documented environment it allows the origin http://localhost:5173, enables credentials, and declares the methods and headers accepted by the API.

```
public static function handle(): void
{
    header("Access-Control-Allow-Origin: http://localhost:5173");
    header("Access-Control-Allow-Credentials: true");
    header("Access-Control-Allow-Headers: Content-Type, X-CSRF-Token");
    header("Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS");

    if ($_SERVER["REQUEST_METHOD"] === "OPTIONS") {
        http_response_code(204);
        exit;
    }
}
```

OPTIONS requests are preflight checks that the browser may send before the actual operation. The middleware responds with 204 and ends execution because there is no Controller functionality to handle in that case. The allowed origin is hard-coded in the current implementation; when CineVerse is deployed on another domain, it must be reviewed together with the production configuration.

24.7 Controller: coordinating the HTTP request

Controller classes are located in backend/controllers/. The current implementation contains AuthController.php, ProfileController.php, and ReviewController.php. The Router creates the class specified by the route handler and provides the PDO connection through its constructor. A Controller works at the boundary between HTTP and application logic: it reads input, consults the session when necessary, calls validators and Services, and uses Response to return JSON. It should not execute SQL queries or concentrate business rules. The following fragment uses ReviewController::create() as a complete example of that responsibility.

```
public function create(): void
{
    $userId = Session::getUserId();
    $data = $this->request->json();
    $movieId = Validator::int($data["movie_id"]);
    $movieTitle = Validator::stringTrim(
        $data["movie_title"] ?? ""
    );
    $rating = Validator::int($data["rating"]);
    $comment = ReviewValidator::normalizeComment(
        Validator::stringTrim($data["comment"] ?? "")
    );
    ReviewValidator::validateCreate(
        $movieId, $movieTitle, $rating, $comment
    );
    $this->reviewService->create(
        $userId, $movieId, $movieTitle, $rating, $comment
    );
    Response::success("REVIEW_CREATED");
}
```

The fragment shows the actual core of create(), with catch blocks omitted to focus on responsibilities. movie_id and rating are currently read through direct access to the received array; if a manual request omits either key, PHP may raise a notice before Validator::int() can apply the controlled flow. Using \$data["movie_id"] ?? null and \$data["rating"] ?? null would be a hardening improvement. The Controller knows the HTTP input and the final response, but the INSERT and the one-review-per-user-and-movie rule belong to later layers.

24.8 Validator and ReviewValidator: checking input

CineVerse separates general validation from rules specific to a feature. Validator converts and checks basic types; ReviewValidator knows the concrete limits for a review.

```

public static function int(mixed $value): int
{
    if (is_string($value) && ctype_digit($value)) {
        return (int) $value;
    }

    if (!is_int($value)) {
        throw new InvalidArgumentException("MUST_BE_INTEGER");
    }

    return $value;
}

public static function validateCreate(
    int $movieId,
    string $movieTitle,
    int $rating,
    string $comment
): void {
    if ($movieId <= 0) {
        throw new InvalidArgumentException("INVALID_MOVIE_ID");
    }

    self::validateMovieTitle($movieTitle);
    self::validateContent($rating, $comment);
}

```

`Validator::int()` accepts either a PHP integer or a string containing digits only and always returns an int; if it cannot do so, it throws `MUST_BE_INTEGER`. `ReviewValidator` then works with already typed values and applies domain rules, such as `movie_id` greater than zero, a rating between 1 and 5, and a comment between 20 and 1000 characters.

24.9 Service: applying business rules

The Service answers questions specific to CineVerse. `ReviewService::create()` does not read JSON or choose an HTTP status code. It checks the `one-review-per-user-and-movie` rule and, if the rule is satisfied, delegates the write to the Model.

```

public function create(
    int $userId,
    int $movieId,
    string $movieTitle,
    int $rating,
    string $comment
): void {
    $existingReview = Review::existsReviewByUserIdAndMovieId(
        $this->pdo, $userId, $movieId
    );

    if ($existingReview) {
        throw new RuntimeException("REVIEW_ALREADY_EXISTS");
    }

    Review::create(
        $this->pdo, $userId, $movieId,
        $movieTitle, $rating, $comment
    );
}

```

If the rule fails, the Service throws `REVIEW_ALREADY_EXISTS`. The Controller then decides that this case corresponds to HTTP 409. This separation allows the business rule to exist in the business layer without coupling it to the exact shape of an HTTP response.

24.10 Model and PDO: executing SQL against MySQL

The Model contains data access. `Review::create()` receives the PDO connection and executes a prepared INSERT. This is where table names, columns, and SQL parameters appear.

```

$stmt = $pdo->prepare("
    INSERT INTO reviews (
        user_id, movie_id, movie_title, rating, comment
    )
    VALUES (
        :user_id, :movie_id, :movie_title, :rating, :comment
    )
");

$stmt->execute([
    ":user_id" => $userId,
    ":movie_id" => $movieId,
    ":movie_title" => $movieTitle,
    ":rating" => $rating,
    ":comment" => $comment
]);

```

`prepare()` separates the SQL structure from the values. The `:user_id`, `:movie_id`, and other placeholders are bound when the query executes. The Model does not decide whether the user was authenticated or which HTTP status the browser should receive; it focuses on reading or modifying MySQL.

24.11 Response: building a consistent JSON response

`Response.php` is located at `backend/core/http/Response.php`. It builds HTTP responses in JSON format so Controllers do not have to repeat `http_response_code()` and `json_encode()` in every method. It performs the opposite role of `Request`: it takes an internal backend result and transforms it into a uniform response through `success()` or `error()`.

```

public static function success(
    string $code,
    array $data = [],
    int $statusCode = 200
): void {
    self::json($statusCode, true, $code, $data);
}

public static function error(
    string $code,
    int $statusCode
): void {
    self::json($statusCode, false, $code);
}

// resulting JSON output
{
    "success": true,
    "code": "REVIEW_CREATED"
}

```

`Response::json()` sets the HTTP status, creates the `success` and `code` keys, adds data only when it contains information, and finally serializes the response with `json_encode(..., JSON_UNESCAPED_UNICODE)`. This gives the frontend JSON responses with the same basic structure whether an operation succeeds or an error occurs. In the current implementation, the `Content-Type: application/json; charset=utf-8` header is set by the Controllers before the response is sent.

24.12 Complete example: PUT /reviews/15 from Apache to MySQL

Now that each piece is known, the complete flow can be read without introducing new concepts. Suppose an authenticated user edits review number 15.

```

PUT /reviews/15
-> Apache / .htaccess
-> public/index.php
-> bootstrap.php
-> Database::connect() / Session::start() / CORS
-> routes/api.php finds PUT /reviews/{id}
-> Router converts {id} into a numeric pattern
-> AuthMiddleware::requireAuth()
-> CsrfMiddleware::validateCsrf()
-> Router extracts id=15 and creates ReviewController
-> ReviewController::update(15)
-> Request::json()
-> Validator + ReviewValidator
-> ReviewService::update()
-> Review::findReviewById(15)
-> check review.user_id == session user
-> Review::update()
-> PDO UPDATE reviews
-> MySQL
-> Response::success("REVIEW_UPDATED")
-> JSON returns to the frontend

```

The value of separating layers is clear in this flow. The Router does not know the ownership rule; ReviewService does not know how {id} was extracted from the URL; Review does not know about CSRF; and Response does not execute SQL. Each piece can be read and tested with one specific responsibility. Sections 25 and 26 apply this architecture to authentication, account, profile, and reviews.

25. Authentication and account

Authentication means verifying who the user is and preserving that identity across subsequent requests.

CineVerse combines validation, hashed passwords, PHP sessions, cookies, random tokens, and email. A new account must verify its email address before it can sign in.

A password hash is an irreversible result generated from a password; the backend stores that hash instead of the original text. A token is a hard-to-guess random value used, for example, to verify an email address or reset a password. A session stores on the server that a browser has signed in, while the cookie allows later requests from that browser to be associated with the correct session.

How authentication classes divide responsibilities

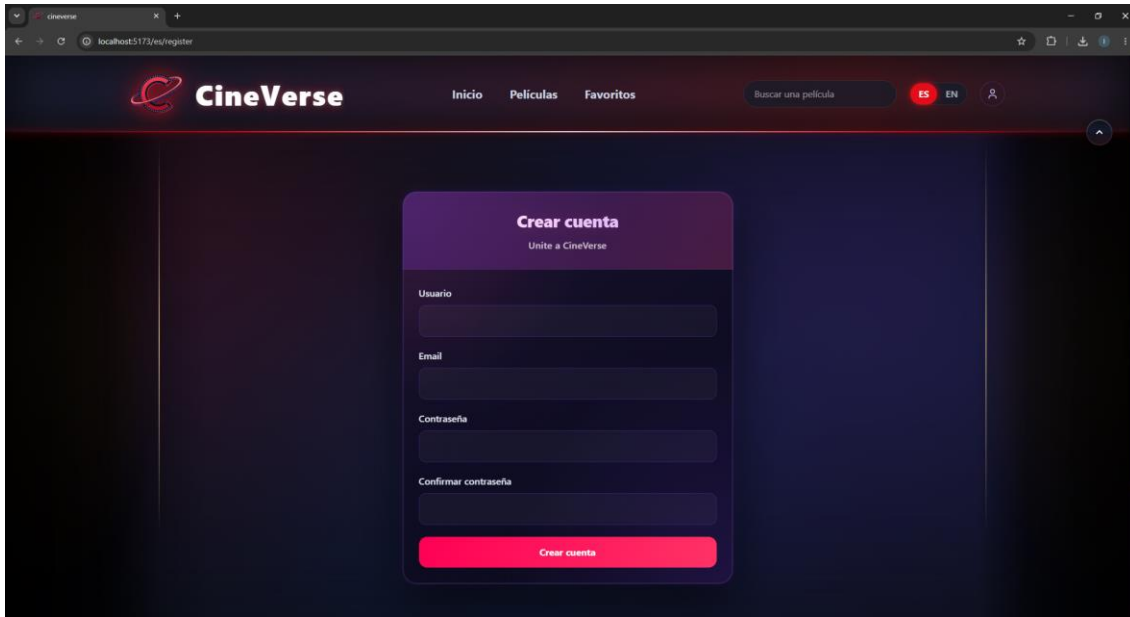
Authentication does not live in a single class. AuthController receives the HTTP operation; AuthValidator validates input data; AuthService applies process rules; User and LoginAttempt access MySQL; Session maintains the authenticated identity and CSRF token; and MailService participates whenever the flow needs to send a verification or recovery email.

```

AuthController
-> AuthValidator
-> AuthService
    -> User
    -> LoginAttempt
    -> Session
    -> MailService when email is required
-> Response

```

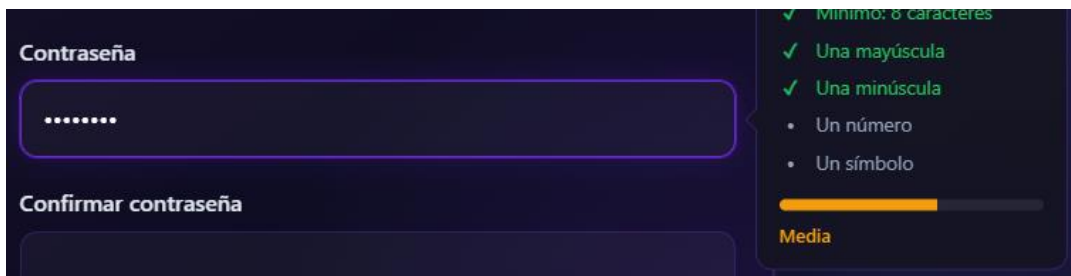
25.1 Registration



CineVerse registration screen. This is where the process of creating and verifying a new account begins.

Practical example: Carlos creates an account

Carlos does not have an account yet. He enters a username, email address, password, and confirmation. RegisterView.vue validates the form in the browser before calling the backend; if it finds an error, it stops submission and displays the error on the screen itself.



Visual password validation during registration. The interface marks in real time which requirements have already been met and summarizes the current strength.

The frontend and backend validate for different reasons. RegisterView.vue uses the functions in register.validation.ts to provide immediate feedback: at least 8 characters, one uppercase letter, one lowercase letter, one number, and one symbol; it also enforces the 32-character maximum and checks that the confirmation matches. When the user clicks Create Account, AuthValidator.php checks on the server the rules that must actually be enforced, without trusting what the browser sent.

Main components

Frontend: RegisterView.vue -> auth.service.ts

Backend: AuthController.php -> AuthService.php -> User.php -> MailService.php

```

Carlos clicks "Create Account"
-> RegisterView.register()
-> validateRegisterForm()
-> registerUser()
-> POST /register through backendApi
-> Router
-> AuthController::register()
-> AuthValidator validates username, email, password, and language
-> AuthService checks for duplicate username/email
-> password_hash(PASSWORD_DEFAULT)
  
```

```

-> generates verification_token with a 24-hour expiration
-> beginTransaction()
-> User::create() inserts the account with is_verified = 0
-> MailService builds the email
-> PHPMailer hands it to the SMTP server
-> COMMIT
-> RegisterView tells the user to check their email

```

If the email cannot be sent before COMMIT, AuthService executes ROLLBACK and the registration is not committed. If delivery succeeds, Carlos receives the verification link and continues with section 25.2.

Field	Backend rules
username	3-20 characters; letters, numbers, and _.
email	Valid format; maximum 80 characters.
password	8-32 characters; uppercase, lowercase, number, and special character.
language	Only es or en.

AuthValidator.php: authentication-specific rules

AuthValidator.php is located at backend/validators/AuthValidator.php. It is an authentication-specific validation class: it checks username, email, password, and language, but does not query MySQL or decide whether an account already exists. When a value violates a rule, it throws `InvalidArgumentException` with a stable code that the Controller can convert into an HTTP response.

```

public static function validateRegister(
    string $username,
    string $email,
    string $password
): void {
    self::validateUsername($username);
    self::validateEmail($email);
    self::validatePassword($password);
}

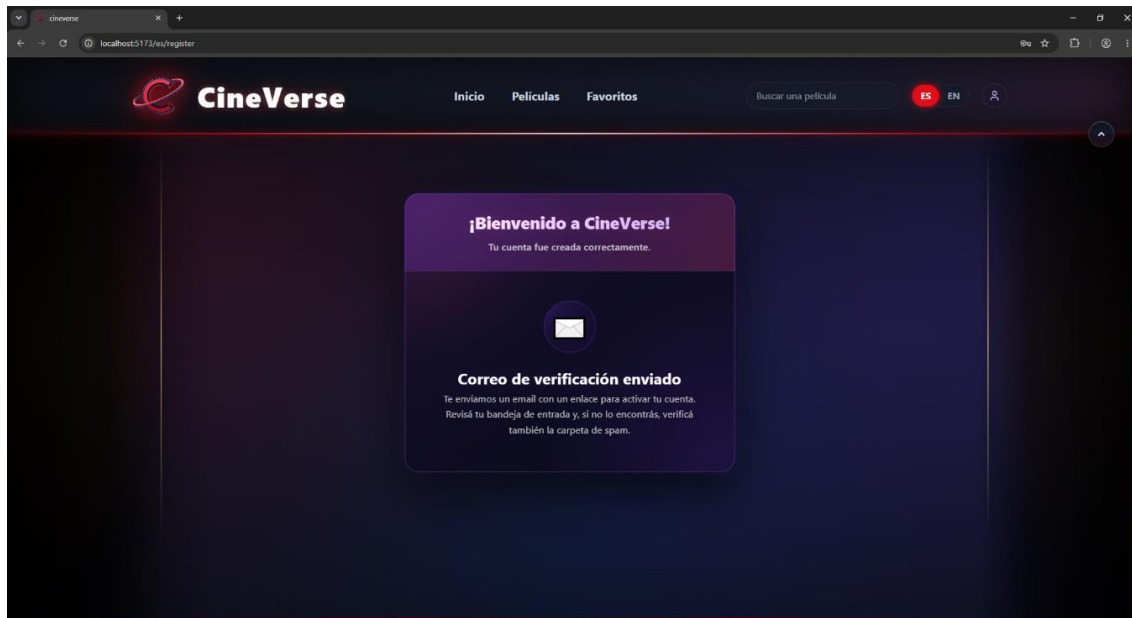
public static function validateLanguage(string $language): void
{
    if (!in_array($language, ["es", "en"], true)) {
        throw new InvalidArgumentException("INVALID_LANGUAGE");
    }
}

```

The first method chains private format and complexity rules. `validateLanguage()` limits emails to the two languages that actually have templates. Duplicate checks, password hashing, and user creation happen later, in `AuthService` and `User`.

1. `AuthController` receives the JSON sent by the form and checks that the basic fields have the expected type.
2. `AuthValidator` checks the username, email, password, and language rules.
3. `AuthService` queries MySQL to prevent duplicate usernames or email addresses.
4. `password_hash(PASSWORD_DEFAULT)` transforms the password into the hash that will be stored in users.
5. `random_bytes(32)` generates a hard-to-predict verification token, and a 24-hour expiration time is stored.
6. User creation and email delivery take place inside a transaction. The user is inserted as unverified and `MailService` sends the confirmation link.
7. If everything completes successfully, COMMIT is executed. If an exception occurs before the process completes, ROLLBACK is executed to undo the changes in that transaction.

Legacy frontend detail. `RegisterView.vue` still uses the name `laravelErrors` and handles `password_confirmation`; `auth.ts` also retains a comment that mentions Laravel. The current CineVerse backend does not use Laravel, so these names are remnants of an earlier implementation and should be removed or renamed to avoid confusion.



Post-registration screen: the user must verify the email address before signing in.

25.1.1 How the email language is chosen

The backend does not automatically detect whether the user wants an email in Spanish or English. The language is already active in vue-i18n, and the frontend explicitly includes it in operations that may send a message.

```

RegisterView.vue
  locale.value = "es"
    |
    v
  POST /register
  {
    "username": "user",
    "email": "user@example.com",
    "password": "...",
    "language": "es"
  }
    |
    v
  AuthController -> AuthValidator -> AuthService -> MailService

```

AuthController reads language from the JSON with `Validator::stringTrim()`, and `AuthValidator::validateLanguage()` accepts only `es` or `en`. `AuthService` then receives that value and passes it without reinterpretation to `generateVerificationEmail()` or `generatePasswordResetEmail()`.

25.1.2 How MailService builds the verification email

`MailService` is a CineVerse-specific class. `PHPMailer` is the external library that performs delivery. `MailService` first builds the content and only then delegates transmission to `PHPMailer`.

`generateVerificationEmail(email, token, language)` follows this flow:

1. It builds `verificationUrl` from `FRONTEND_URL`, the language, and the token. The token passes through `urlencode()` before being incorporated into the URL.
2. It forms the template path `templates/emails/verify-account.<language>.php`. For `es` it uses `verify-account.es.php`; for `en`, `verify-account.en.php`.
3. `ob_start()` opens an output buffer. `require` executes the PHP template, which uses `$verificationUrl` to create the email's button and visible link.
4. `ob_get_clean()` retrieves all HTML generated by the template and stores it in `$html`.

5. The subject also depends on language: en uses "CineVerse Account Verification" and es uses "Verificación de cuenta | CineVerse".

6. Finally, it calls the private method `send(email, subject, html)`.

```

FRONTEND_URL + "/" + language + "/verify-email?token=" + token
    |
    v
verify-account.es.php / verify-account.en.php
    |
    v
HTML completo
    |
    v
send(destinatario, asunto, html)

```

If the calculated template does not exist, MailService throws `RuntimeException("EMAIL_TEMPLATE_NOT_FOUND")`. Because AuthValidator already limits language to es or en, the normal flow should use only the four templates included in the project.

25.1.3 How PHPMailer actually sends the message

`send()` creates new `PHPMailer(true)`. `true` tells PHPMailer to throw exceptions when delivery fails. It then configures an SMTP connection with values from `backend/.env`.

```

$mail = new PHPMailer(true);
$mail->isSMTP();
$mail->Host = $_ENV["MAIL_HOST"];
$mail->Port = (int) $_ENV["MAIL_PORT"];
$mail->SMTPAuth = true;
$mail->Username = $_ENV["MAIL_USERNAME"];
$mail->Password = $_ENV["MAIL_PASSWORD"];
$mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
$mail->CharSet = "UTF-8";

$mail->setFrom($_ENV["MAIL_FROM"], $_ENV["MAIL_FROM_NAME"]);
$mail->addAddress($to);
$mail->isHTML(true);
$mail->Subject = $subject;
$mail->Body = $html;
$mail->send();

```

`isSMTP()` indicates that the message will be sent via SMTP. `Host` and `Port` identify the server. `SMTPAuth` enables authentication, and `Username/Password` provide the credentials. `STARTTLS` encrypts the SMTP connection after it has been initiated. `setFrom()` sets the sender, `addAddress()` the recipient, `isHTML(true)` indicates that `Body` contains HTML, and `Subject/Body` set the final content.

When `$mail->send()` completes without an exception, PHPMailer has handed the message to the configured SMTP server and MailService returns to AuthService. This does not by itself guarantee that the recipient has already read it; it means that the SMTP delivery requested by the application completed successfully.

25.1.4 What happens if email delivery fails

The try/catch block in `send()` specifically wraps the call to `$mail->send()`. If that call throws a `Throwable`, MailService transforms it into a `RuntimeException` with the internal code `EMAIL_DELIVERY_FAILED`. The original exception is stored as `previous`, allowing AuthController to log the actual technical message without sending it to the browser. An error occurring earlier, during PHPMailer configuration, would still follow the backend's general exception handling.

```

$mail->send() throws a Throwable
-> the try/catch in send() catches it
-> RuntimeException("EMAIL_DELIVERY_FAILED", previous = SMTP error)
-> AuthController
-> AppLogger records the actual SMTP detail
-> Response::error("EMAIL_DELIVERY_FAILED", 422)
-> frontend

```

During registration, AuthService had already opened a transaction before inserting the user. If the email fails, the catch executes rollBack() and rethrows the exception; therefore, the registration is not committed. commit() runs only after the email has been sent successfully. Verification resend uses the same pattern so the new token is not committed if the message could not be sent.

The current AuthController code logs EMAIL_DELIVERY_FAILED failures through AppLogger and responds with HTTP 422. This classification is documented as current behavior; a 5xx code would better represent an SMTP service outage. Section 28 also details which message the user ultimately sees on each screen.

25.1.5 What is stored in users during registration

User::create() executes an INSERT into users with username, email, password_hash, is_verified=0, verification_token, and verification_expires_at. The original password never enters the table: AuthService first transforms it with password_hash(PASSWORD_DEFAULT).

```
Successful registration, before email verification:
username           = "user"
email              = "user@example.com"
password_hash      = "<hash generated by PHP>"
is_verified        = 0
verification_token  = "<random token>"
verification_expires_at = "<24 hours>"
```

That INSERT and email delivery are inside the same transaction. If the email fails before commit(), rollBack() undoes the registration and the row is not committed.

25.2 Verification and resend

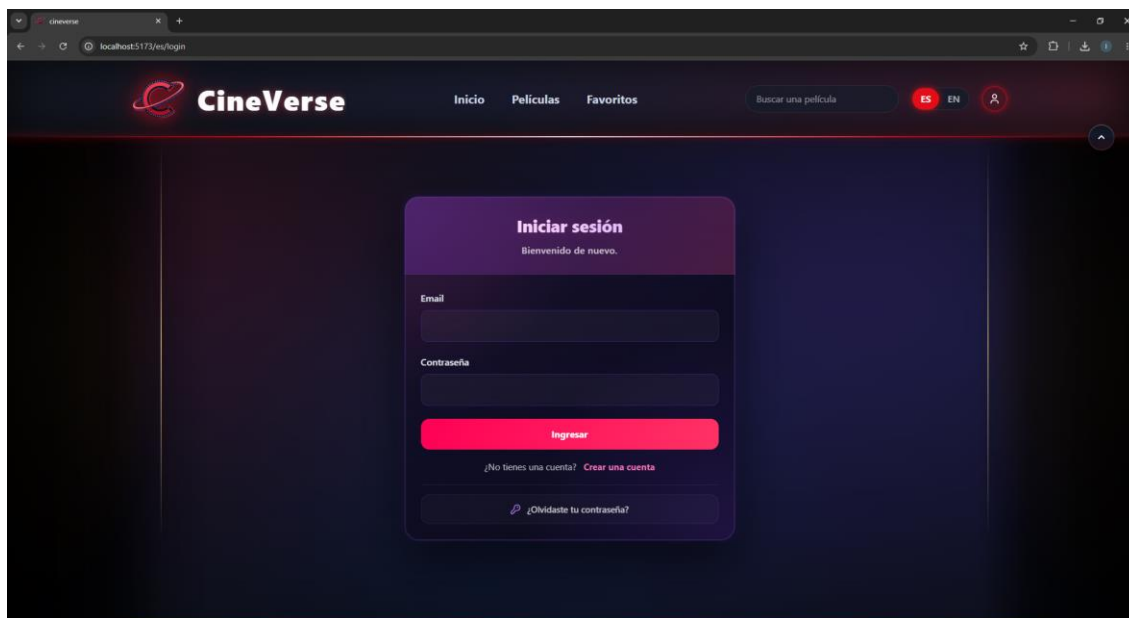
Carlos opens the email received during registration and clicks the verification link. The URL contains the token generated during registration. GET /verify-email looks up that token, checks that it has not expired, and activates the account. The token and its expiration time are then cleared so the same link cannot be reused. Resend uses neutral responses when the email address does not exist or is already verified, so registered accounts are not disclosed.

25.2.1 What happens when the verification link is opened

```
email -> /es/verify-email?token=...
-> VerifyEmailView reads token from the URL
-> GET /verify-email?token=...
-> AuthController
-> AuthService::verifyEmail(token)
-> User::findByVerificationToken()
-> check expiration
-> User::activateEmail()
-> UPDATE users:
    is_verified = 1
    verification_token = NULL
    verification_expires_at = NULL
-> EMAIL_VERIFIED
```

The token is invalidated when the account is activated. Resend follows a different branch: if the account exists and remains unverified, AuthService generates a new token, updates the two verification columns, and commits the transaction only if MailService was able to send the new message.

25.3 Sign-in and attempt limiting



Sign-in screen. This is where the process that creates the PHP session and updates the frontend authentication state begins.

25.3.1 Practical example: Carlos signs in

Carlos has already verified his account. He enters his email address and password and clicks Sign In. LoginView.vue validates the form first; only then does auth.service.ts send the credentials to CineVerse's own API.

Main components

Frontend: LoginView.vue -> auth.service.ts

Backend: AuthController.php -> AuthService.php -> User.php / LoginAttempt.php -> Session.php

```

Carlos clicks "Sign In"
-> LoginView.login()
-> loginUser()
-> POST /login
-> AuthController::login()
-> AuthValidator::validateLogin()
-> AuthService::login()
-> checkLoginAttempts(IP + email)
-> User::findByEmail()
-> password_verify()
-> check_is_verified
-> clear failed attempts if any existed
-> Session::regenerate()
-> Session::set("user_id", id)
-> Session::generateCsrfToken()
-> LOGIN_SUCCESS { user, csrf_token }
-> authStore.setUser(...)
-> authStore.setCsrfToken(...)
-> router.push(Home)
-> Carlos is authenticated

```

Before checking the password, AuthService reviews failed attempts associated with the IP + email combination. The first three incorrect attempts are recorded as failures. Once that combination has accumulated three attempts, the next attempt is blocked before credentials are checked and the backend responds with LOGIN_TEMPORARILY_BLOCKED. Old records are deleted after 15 minutes. This control is known as rate limiting or attempt limiting.

If the user does not exist or password_verify() determines that the password does not match the stored hash, the failure counter is incremented. If the password is correct but the account is still unverified, the backend returns

EMAIL_NOT_VERIFIED. On a successful sign-in, the counter is cleared, the session ID is regenerated, and a CSRF token is generated.

AuthController.php: receiving POST /login

AuthController.php is located at backend/controllers/AuthController.php. Among other operations, it handles registration, sign-in, verification, recovery, logout, and retrieval of the authenticated user. During sign-in, it obtains the JSON through Request, normalizes basic types, calls AuthValidator, delegates the actual process to AuthService, and finally builds the response with Response.

```
$data = $this->request->json();

$email = Validator::stringTrim($data["email"] ?? "");
$password = Validator::string($data["password"] ?? "");

AuthValidator::validateLogin(
    $email,
    $password
);

$data = $this->authService->login(
    $email,
    $password
);

Response::success(
    "LOGIN_SUCCESS",
    $data
);
```

The fragment is the actual core of login(), with catch blocks and comments omitted to focus on coordination. The Controller knows the HTTP input and final response, but it does not execute password_verify(), query users directly, or modify the login_attempts table.

AuthService.php: applying sign-in rules

AuthService.php is located at backend/services/AuthService.php. It is the layer that coordinates authentication rules: it controls failed attempts, queries the user, verifies the password and account status, clears previous attempts, and prepares the authenticated session. It does not decide the HTTP status code the browser will receive.

```
$ip = $_SERVER["REMOTE_ADDR"] ?? "";
$attempt = $this->checkLoginAttempts($ip, $email);
$user = User::findByEmail($this->pdo, $email);

if (!$user || !password_verify($password, $user["password_hash"])) {
    if ($attempt) {
        LoginAttempt::increment($this->pdo, $attempt["id"]);
    } else {
        LoginAttempt::create($this->pdo, $ip, $email);
    }

    throw new RuntimeException("INVALID_CREDENTIALS");
}

if (!$user["is_verified"]) {
    throw new RuntimeException("EMAIL_NOT_VERIFIED");
}

if ($attempt) {
    LoginAttempt::reset($this->pdo, $attempt["id"]);
}

Session::regenerate();
Session::set("user_id", $user["id"]);
$csrfToken = Session::generateCsrfToken();
```

This shows why AuthService is a business layer: it combines several components to fulfill a complete CineVerse rule. Attempt limiting is checked before the password is validated; an unverified account is rejected; and only a successful sign-in regenerates the session, stores user_id, and generates the CSRF token.

User.php: Model for the users table

User.php is located at backend/models/User.php. It is the Model that groups SQL queries for the users table: duplicate checks, creation, searches by id/email/tokens, account activation, and token or password updates. During sign-in, AuthService uses findByEmail() to retrieve exactly the data it needs to authenticate.

```
public static function findByEmail(
    PDO $pdo,
    string $email
): array|false {
    $stmt = $pdo->prepare("
        SELECT
            id,
            username,
            email,
            password_hash,
            is_verified,
            created_at
        FROM users
        WHERE email = :email
        LIMIT 1
    ");

    $stmt->execute([
        ":email" => $email
    ]);

    return $stmt->fetch();
}
```

The Model knows the table and the SQL, but it does not decide whether a password is valid or which HTTP response should be returned. This separation allows User::findByEmail() to be reused in sign-in and recovery without moving authentication rules into data access.

LoginAttempt.php: Model for attempt limiting

LoginAttempt.php is located at backend/models/LoginAttempt.php and works only with the login_attempts table. It can find a record by IP + email, create the first failure, increment the counter, delete it after a successful sign-in, and remove records whose last attempt was more than 15 minutes ago. The Model stores that data; the decision to block after three attempts belongs to AuthService.

```
public static function increment(PDO $pdo, int $id): void
{
    $stmt = $pdo->prepare("
        UPDATE login_attempts
        SET attempts = attempts + 1
        WHERE id = :id
    ");

    $stmt->execute([
        ":id" => $id
    ]);
}

public static function deleteExpired(PDO $pdo): void
{
    $stmt = $pdo->prepare("
        DELETE FROM login_attempts
        WHERE last_attempt < DATE_SUB(NOW(), INTERVAL 15 MINUTE)
    ");

    $stmt->execute();
}
```

AuthService::checkLoginAttempts() first calls deleteExpired(), finds the record for the IP + email combination, and throws LOGIN_TEMPORARILY_BLOCKED when the existing counter is already 3 or higher. In this way, the Model persists the control state while the Service retains the business rule.

The session identifier remains in the PHP cookie; the frontend does not need to know its value. The CSRF token is returned in the JSON and stored in Pinia because protected operations must copy it to X-CSRF-Token.

Attempt limiting is applied before the password is validated. The first three failures are recorded. If three failures already exist for that IP+email combination when an attempt begins, the next attempt is blocked and returns LOGIN_TEMPORARILY_BLOCKED before credentials are checked.

25.4 User session and cookie

Session.php: PHP session management

Session.php is located at backend/core/auth/Session.php. It is a static class responsible for working with the application session so Controllers, Services, and middleware do not have to manipulate \$_SESSION directly in different parts of the code. Its responsibilities include starting and configuring the session, storing and retrieving values, identifying the authenticated user, regenerating the session identifier after sign-in, destroying the session during logout, and generating or validating the CSRF token.

```
public static function getUserId(): ?int
{
    return self::has("user_id")
        ? (int) self::get("user_id")
        : null;
}

public static function generateCsrfToken(): string
{
    $token = bin2hex(
        random_bytes(32)
    );

    self::set("csrf_token", $token);

    return $token;
}

public static function validateCsrfToken(string $token): bool
{
    return hash_equals(
        self::getCsrfToken() ?? "",
        $token
    );
}
```

getUserId() provides a single way to retrieve the identity associated with the session. The CSRF token is generated with random_bytes(32), stored in the same session, and later compared with the value received by CsrfMiddleware using hash_equals(). The following table also summarizes the concrete cookie configuration and inactivity expiration.

Protection	Behavior
lifetime	0: session cookie.
path	/
HttpOnly	true
SameSite	Lax
Secure	Enabled when \$_SERVER["HTTPS"] is not empty.
Inactivity	1800 seconds / 30 minutes.

Protection	Behavior
Session fixation	session_regenerate_id(true) after sign-in.

25.4.1 What happens after F5 if the user was already signed in

Carlos presses F5 after signing in. Pinia starts again in memory, but the browser keeps the PHP cookie; therefore, the SPA can reconstruct the identity by requesting /user without asking for the password again.

```
new frontend load
-> the browser keeps the PHP cookie
-> main.ts calls authStore.initialize()
-> GET /user with withCredentials=true
-> AuthMiddleware confirms user_id in $_SESSION
-> AuthController obtains user_id + csrf_token from Session
-> User::findById(user_id)
-> AUTHENTICATED_USER { user, csrf_token }
-> Pinia reconstructs user and csrfToken
```

This is why the authentication store does not need to be persisted in localStorage. PHP is the actual source of the session; Pinia only reconstructs a representation that is useful to the interface.

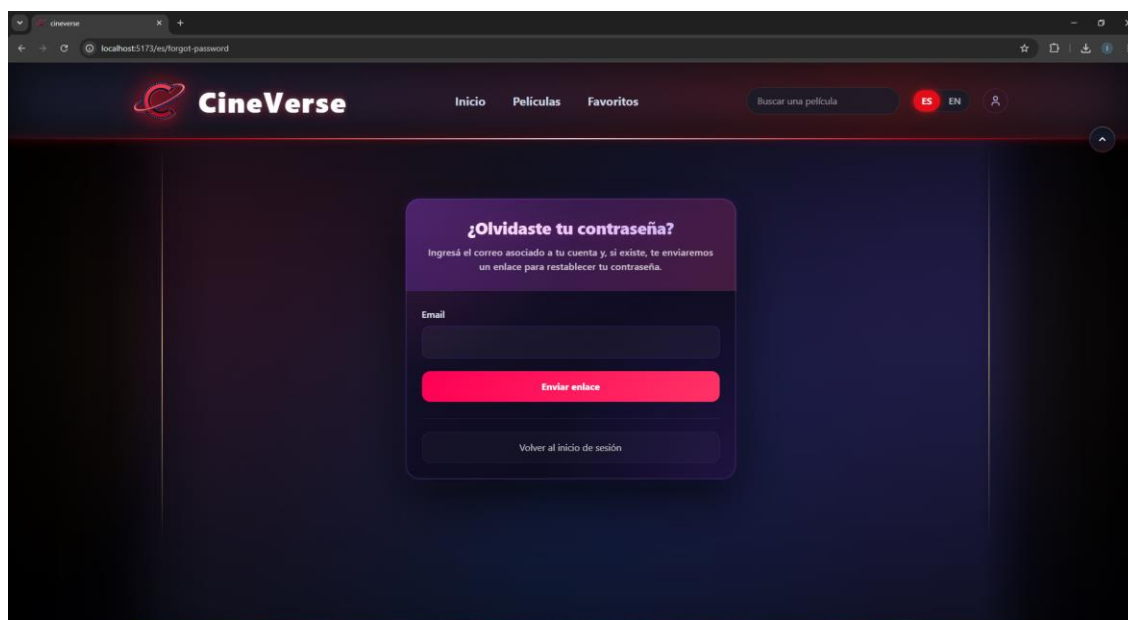
25.4.2 Logging out internally

Logout does modify state, so POST /logout requires both a session and CSRF. AuthService calls Session::destroy(): it clears \$_SESSION, expires the cookie if PHP uses cookies, and destroys the server session. The frontend must then clear its store so the interface no longer considers the user authenticated.

25.5 CSRF

CSRF protects data-changing operations against requests initiated from another site that exploit an already active session. Session::generateCsrfToken() creates a random value and stores it in the session. The frontend sends that same value in X-CSRF-Token, and CsrfMiddleware compares it with hash_equals before allowing the operation.

25.6 Password recovery



"Forgot your password?" screen. This is the first step of the flow: the user requests a link without CineVerse revealing whether the email address exists.

Recovery is not a single request. CineVerse separates the process into three stages: request the email, validate the received token, and only if that token is valid, save a new password.

25.6.1 Practical example: Carlos recovers his password

Step 1. Request the link

Carlos does not remember his password. From Login, he clicks "Forgot your password?", Vue Router opens /es/forgot-password, he enters his email address and clicks "Send link".

Main components

Frontend: ForgotPasswordView.vue / ResetPasswordView.vue -> auth.service.ts

Backend: AuthController.php -> AuthService.php -> User.php -> MailService.php

```

Carlos clicks "Send link"
-> ForgotPasswordView.sendResetEmail()
-> validateEmail()
-> forgotPassword()
-> POST /forgot-password { email, language }
-> AuthController::forgotPassword()
-> AuthService::forgotPassword()
-> if the email does not exist: finish without revealing that fact
-> if it exists: generate a token with a 1-hour expiration
-> beginTransaction()
-> User::updatePasswordResetToken()
-> MailService -> PHPMailer -> SMTP
-> COMMIT
-> the frontend displays a neutral response

```

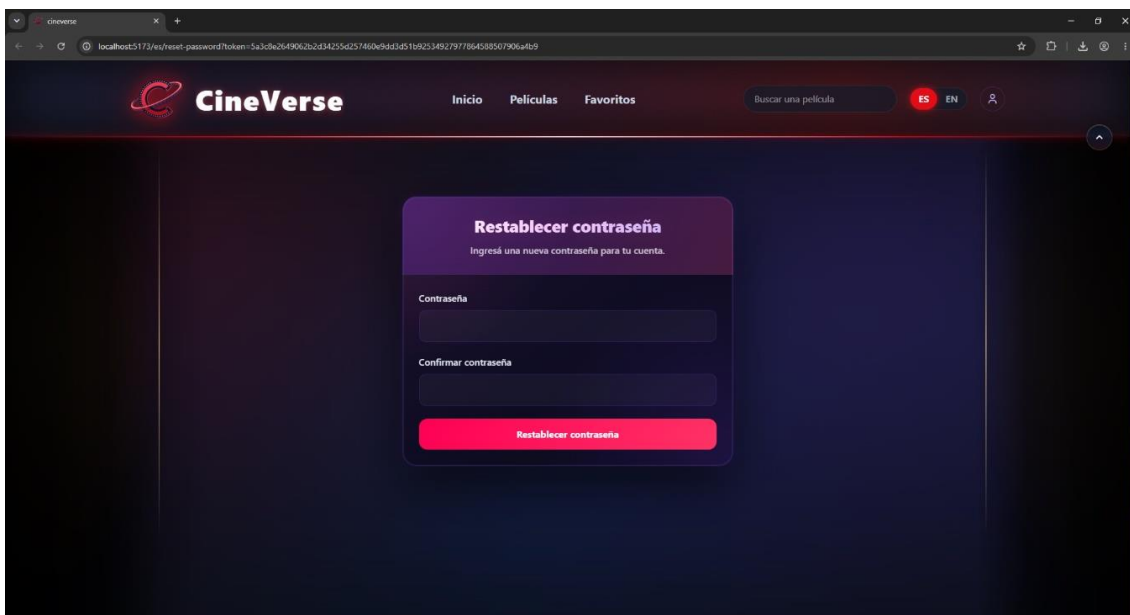
The neutral response is deliberate. The screen says "if it exists" because CineVerse prevents someone from discovering which addresses are registered by comparing different responses.

Step 2. Open the received link

```

Carlos receives the email and clicks the link
-> /es/reset-password?token=...
-> Vue Router mounts ResetPasswordView.vue
-> onMounted() reads token from route.query
-> validateResetToken(token)
-> GET /reset-password/validate
-> backend checks that the token exists and has not expired
-> valid token: tokenStatus = "valid"
-> only then does the form appear

```



Form for setting a new password. ResetPasswordView displays it only after the backend accepts the token received by email.

Step 3. Set a new password

Carlos enters and confirms the new password. The view reuses the same password rules as registration; if the form is valid, it sends the token together with the new password to the backend.

```

Carlos clicks "Reset Password"
-> ResetPasswordView.sendResetPassword()
-> validatePassword() + confirm match
-> resetPassword({ token, password })
-> POST /reset-password
-> AuthController::resetPassword()
-> AuthService::resetPassword()
-> validate token and expiration again
-> password_hash(new password)
-> User::updatePassword()
-> UPDATE users
    password_hash = new hash
    password_reset_token = NULL
    password_reset_expires_at = NULL
-> PASSWORD_RESET_SUCCESS
-> Carlos can return to Login with the new password

```

25.6.2 How the recovery email is built and sent

POST /forgot-password receives email and language. If the email address does not exist, AuthService finishes without doing anything and the Controller still returns PASSWORD_RESET_EMAIL_SENT; this avoids revealing which addresses are registered. If the account exists, a random 32-byte token is generated and converted to hexadecimal, with a one-hour expiration time.

AuthService starts a transaction, stores the token, and calls generatePasswordResetEmail(email, token, language). MailService builds FRONTEND_URL/<language>/reset-password?token=<token>, selects reset-password.es.php or reset-password.en.php, captures the HTML with ob_start()/ob_get_clean(), chooses the subject, and reuses exactly the same PHPMailer send() method described for verification.

26. Reviews and profile

Reviews link a CineVerse account to a movie obtained from TMDb. MySQL stores user_id, movie_id, movie_title, rating, and comment. For public reviews, the backend combines reviews with users to also obtain the username. That combination between tables is performed through an SQL JOIN.

26.1 Review rules

Rule	Actual value
One per movie	Enforced in ReviewService + UNIQUE(user_id,movie_id) in MySQL.
Rating	Integer 1-5.
Comment	20-1000 characters after normalization.
Movie title	Required; maximum 255 characters.
Ownership	Only the owner can edit/delete.
Pagination	12 reviews per page.

Current interface detail. The actual comment limit is 1000 characters in MovieReviews.vue, ReviewValidator.php, and MySQL. However, the commentTooLong keys in es.json and en.json still display a maximum of 2000. Those texts should say 1000 to match the actual validation.

26.2 Cleaning the comment before saving it

Before validating the comment, `ReviewValidator::normalizeComment()` removes extra whitespace at the beginning and end, normalizes line breaks, and prevents excessive blocks of blank lines. It then checks the length, and only after that can the text be stored.

26.3 Creating a review

```
POST /reviews
X-CSRF-Token: <token>

{
  "movie_id": 550,
  "movie_title": "Fight Club",
  "rating": 5,
  "comment": "A valid review of at least twenty characters."
}
```

`ReviewService` first checks whether a review already exists with the same `user_id` and `movie_id`. If it does, it returns `REVIEW_ALREADY_EXISTS`. If it does not, `Review::create()` executes the `INSERT`. MySQL's `UNIQUE` constraint acts as a second safeguard against two concurrent requests.

26.3.1 What happens in the interface when the user writes a review

`MovieReviews` displays the Write Review button only when `authStore.isAuthenticated` is true, no `userReview` exists, and the form is closed. When it opens, `formMode` becomes "create", `rating` returns to 0, and `comment` is reset to an empty string.

`StarRating` updates `formRating` through the `update:rating` event. The textarea uses `v-model` for `formComment`, and `handleCommentInput` normalizes line breaks while the user types, prevents more than one consecutive blank line, and truncates the text at 1000 characters. `canSubmitForm` is enabled only with `rating > 0` and a normalized comment between 20 and 1000 characters.

Figure. Review creation form with rating, comment, and character counter.

26.3.2 Complete example: Carlos reviews Fight Club

Carlos is viewing Fight Club. He opens the Reviews tab, selects a rating, and writes his comment. When he clicks Save, the flow begins that ultimately persists the review in MySQL and then reads the state confirmed by the backend again.

```
Current movie:
  movieId = 550
  movieTitle = "Fight Club"

User enters:
  rating = 5
  comment = "A valid review ..."

MovieReviews.submitForm()
  -> takes csrfToken from useAuthStore
  -> normalizeLineBreaks(comment)
  -> createReview(payload, csrfToken)

POST /reviews
Cookie: PHP session
X-CSRF-Token: <token>
JSON body:
{
  "movie_id": 550,
  "movie_title": "Fight Club",
  "rating": 5,
  "comment": "A valid review ..."
}

Backend:
Router -> AuthMiddleware -> CsrfMiddleware
-> ReviewController::create()
-> Session::getUserid()
-> Request::json()
-> Validator::int/stringTrim
-> ReviewValidator::normalizeComment()
-> ReviewValidator::validateCreate()
-> ReviewService::create()
-> check user_id + movie_id
-> Review::create()
-> INSERT INTO reviews (...) VALUES (...)
-> REVIEW_CREATED
```

The row that ends up in MySQL contains the `user_id` from the session; the `movie_id` and `movie_title` from the TMDB movie; and the `rating/comment` entered by the user. The review is not stored in Pinia or TMDB.

```
INSERT INTO reviews (
  user_id, movie_id, movie_title, rating, comment
) VALUES (
  :user_id, :movie_id, :movie_title, :rating, :comment
)
```

When the POST completes successfully, `MovieReviews` closes the form and executes `fetchReviews(1)`. In other words, it does not invent locally what the screen should look like: it reads from the backend again. This updates `user_review`, the average, the total, and the community list with the state confirmed in MySQL.

26.4 Editing and deleting

To edit or delete, the backend first looks up the review indicated by the id. If it does not exist, it returns `REVIEW_NOT_FOUND`. If it exists but `user_id` does not match the user in the session, it returns `NOT_REVIEW_OWNER`. Only after that check does it update or delete the row.

26.4.1 Editing a review

```

user clicks Edit
-> openEditForm()
-> copies userReview.rating and userReview.comment into the form
-> formMode = "edit"
-> PUT /reviews/{id}
    Body: { rating, comment }
    Cookie + X-CSRF-Token
-> ReviewController::update(id)
-> ReviewValidator::validateUpdate()
-> ReviewService::update()
-> Review::findReviewById(id)
-> check review.user_id === Session user_id
-> UPDATE reviews SET rating=?, comment=? WHERE id=?
-> REVIEW_UPDATED
-> fetchReviews(1)

```

movie_id and movie_title are not modified during editing. The backend allows only rating and comment to change. If the review does not exist, it responds with REVIEW_NOT_FOUND; if it belongs to another user, it responds with NOT_REVIEW_OWNER.

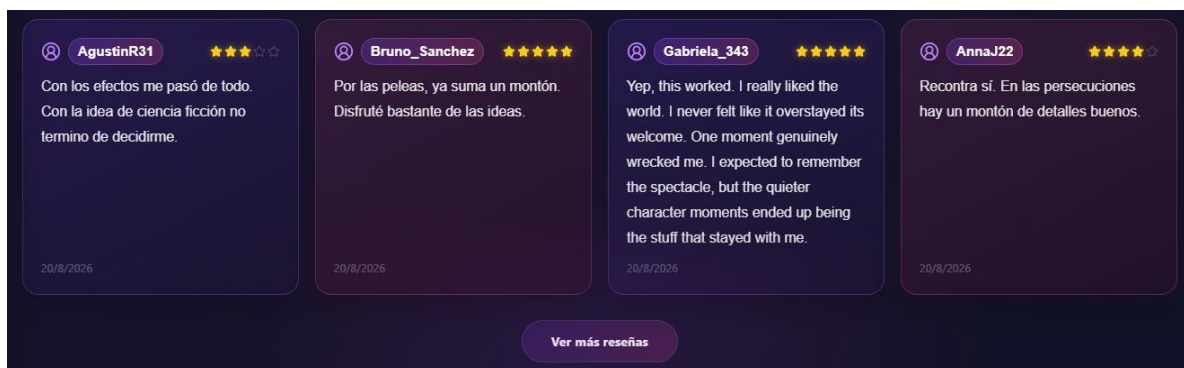
26.4.2 Deleting a review

```

user clicks Delete
-> confirmation appears
-> confirmDelete()
-> DELETE /reviews/{id}
    Cookie + X-CSRF-Token
-> ReviewService finds the row
-> checks ownership
-> DELETE FROM reviews WHERE id=:id
-> REVIEW_DELETED
-> fetchReviews(1)
-> user_review becomes null
-> "Write Review" appears again
-> average and total are recalculated

```

26.5 Public reviews and statistics



Community review cards within a movie detail page.

```

{
  "stats": {
    "average_rating": 4.2,
    "total_reviews": 110
  },
  "pagination": {
    "page": 1,
    "has_next": true
  },
  "user_review": null,
  "reviews": [ ... ]
}

```

On the first page, if the authenticated user has already reviewed the movie, their own review is returned in `user_review` and excluded from the community array so it is not displayed twice. The overall statistics still include all reviews.

26.5.1 When and how a movie's reviews are retrieved

The query does not run at the same time as the TMDb endpoints when the detail page opens. MovieTabs starts on Cast. When the user opens Reviews for the first time, Vue mounts `MovieReviews` and `onMounted(loadInitial)` executes `fetchReviews(1)`.

```
MovieReviews.onMounted(loadInitial)
-> getMovieReviews(movieId, 1)
-> GET /movies/{movieId}/reviews?page=1
-> public endpoint (auth=false, csrf=false)
-> ReviewController::getMovieReviews(movieId)
-> page from Request::query()
-> Session::getUserId() may be null or an id
-> ReviewService::getMovieReviews()
-> MySQL
-> REVIEWS_FETCHED
```

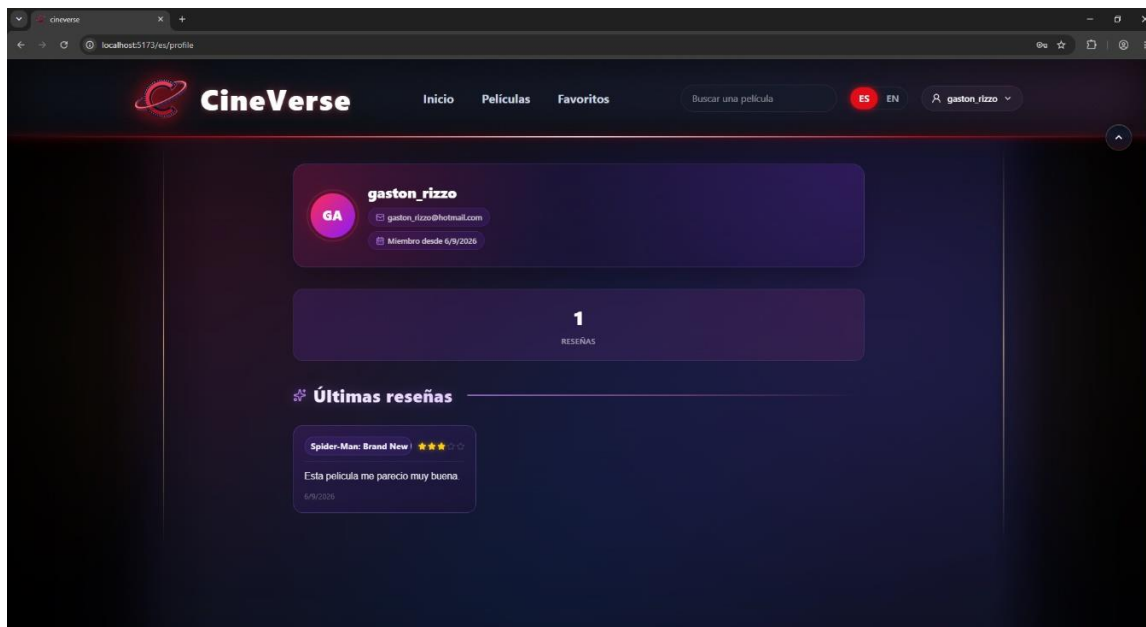
Although the endpoint is public, `backendApi` uses `withCredentials:true`. If a valid cookie exists, `Session::getUserId()` identifies the user. On the first page, `ReviewService` finds that user's review and returns it in `user_review`; it then excludes that `user_id` from the community list to avoid duplication.

Community reviews are retrieved with an INNER JOIN between reviews and users to add username. They are ordered by `created_at` DESC and up to 12 are retrieved per page. `getMovieReviewStats` executes `COUNT(*)` and `ROUND(AVG(rating),1)`; these statistics include all reviews, including the user's own. `countCommunityReviews` uses the same exclusion criterion as the list to calculate `has_next`.

26.5.2 Loading more reviews

`MovieReviews` maintains `currentPage` and `hasNext`. When the user clicks Load More, it requests `currentPage + 1`. For page > 1, `fetchReviews` does not replace the list: it concatenates `data.reviews` to the existing array. `user_review` is updated only with page 1 because the backend returns it only there.

26.6 Profile



Profile: total number of reviews and most recent activity.

The profile shows the user's total number of reviews and their six most recent contributions. The `LATEST_REVIEWS_LIMIT` constant is 6. The MyReviews screen uses `GET /reviews?page=N` to browse all of the user's reviews in pages of 12.

ProfileController.php: profile HTTP entry point

`ProfileController.php` is located at `backend/controllers/ProfileController.php`. The `/profile` endpoint already arrives protected by `AuthMiddleware`, so the Controller obtains `user_id` from `Session`, delegates construction of the summary to `ProfileService`, and returns `PROFILE_FETCHED` through `Response`.

```
$userId = Session::getUserId();
$profileSummary = $this->profileService->getProfile($userId);

Response::success(
    "PROFILE_FETCHED",
    $profileSummary
);
```

The fragment shows the actual core of `getProfile()` without exception handling. The Controller does not write SQL or decide how many recent reviews should be displayed: both responsibilities are delegated to the following layers.

ProfileService.php: the six-recent-reviews rule

`ProfileService.php` is located at `backend/services/ProfileService.php`. It is deliberately small: the `LATEST_REVIEWS_LIMIT = 6` rule belongs to this layer because it expresses an application decision, and the Service passes that limit to the Profile Model.

```
private const LATEST_REVIEWS_LIMIT = 6;

public function getProfile(int $userId): array
{
    return Profile::getProfile(
        $this->pdo,
        $userId,
        self::LATEST_REVIEWS_LIMIT
    );
}
```

In this way, the Model receives the number of rows it must query, but it does not decide on its own that the interface should display six. If the rule changed, `ProfileService` would be the natural place to modify it.

Profile.php: queries that build the summary

`Profile.php` is located at `backend/models/Profile.php`. It is a read-only Model: it receives PDO, the user id, and the limit decided by the Service; it then executes one query to count all reviews and another to retrieve the most recent ones.

```

$reviewsCountStmt = $pdo->prepare(
    "SELECT COUNT(*) AS total
      FROM reviews
     WHERE user_id = ?"
);
$reviewsCountStmt->execute([$userId]);
$reviewsCount = (int) $reviewsCountStmt->fetchColumn();

$latestReviewsStmt = $pdo->prepare(
    "SELECT
      id,
      movie_id,
      movie_title,
      rating,
      comment,
      created_at
    FROM reviews
   WHERE user_id = ?
  ORDER BY created_at DESC
  LIMIT " . $latestReviewsLimit
);
$latestReviewsStmt->execute([$userId]);
$latestReviews = $latestReviewsStmt->fetchAll(PDO::FETCH_ASSOC);

```

Both queries use the same `user_id`. The first obtains `reviews_count`; the second produces `latest_reviews` ordered from newest to oldest. `Profile::getProfile()` returns both results together, and `ProfileService` passes them to the Controller without Vue having to execute those queries separately.

26.6.1 Profile: activity summary

```

ProfileView.onMounted(loadProfile)
-> GET /profile
-> AuthMiddleware
-> ProfileController::getProfile()
-> Session::getUserId()
-> ProfileService::getProfile()
-> Profile::getProfile()
    SELECT COUNT(*) FROM reviews WHERE user_id=?
    SELECT latest 6 ORDER BY created_at DESC
-> PROFILE_FETCHED
-> reviewsCount + latestReviews

```

Profile cards use `movie_id` to navigate back to the movie detail page. The profile does not query TMDB to reconstruct the title: it uses `movie_title` persisted with each review.

26.6.2 My Reviews: complete paginated list

```

MyReviewsView.onMounted(loadReviews)
-> GET /reviews?page=1
-> AuthMiddleware
-> ReviewController::getUserReviews()
-> user_id from Session
-> ReviewService::getUserReviews()
    offset = (page - 1) * 12
    SELECT ... WHERE user_id=? ORDER BY created_at DESC
    COUNT(*) total
    calculate has_next
-> REVIEWS_FETCHED
-> page 1 replaces the array
-> subsequent pages are concatenated

```

If these screens receive `AUTH_REQUIRED`, they clear the authentication store and redirect to Login. This covers the case where the frontend still believed a user was present but the server session had already expired.

26.7 Current page edge case

Validator::int() checks that page is an integer, but does not explicitly require `page >= 1`. ReviewService calculates `offset = (page - 1) * 12`. Therefore, a manual `page=0` request can produce a negative offset. The normal interface always starts at 1, but the backend should validate the range to harden the endpoint against manual calls.

27. Validation and security

CineVerse does not rely on a single check. The frontend helps prevent errors before a form is submitted; the backend validates what actually arrives over HTTP; PDO uses prepared statements; and MySQL maintains constraints on stored data. Each layer covers different problems.

```

DEVELOPMENT
TypeScript + vue-tsc
  -> detect type errors before execution

AT RUNTIME
User
  ↓
Vue form
  ↓
HTTP request
  ↓
Validator / AuthValidator / ReviewValidator
  ↓
Service: CineVerse rules
  ↓
Prepared statements with PDO
  ↓
MySQL: UNIQUE / FOREIGN KEY / CHECK

```

27.1 Password protection

The backend never stores the password entered by the user in `users`. `password_hash()` generates a hash, and `password_verify()` later checks whether a password corresponds to that hash. `PASSWORD_DEFAULT` allows PHP to use the algorithm recommended by the installed version.

27.2 SQL injection

SQL injection occurs when user-controlled text becomes an executable part of a query. CineVerse Models use PDO prepared statements: the SQL structure is written separately and values are sent as parameters. `LIMIT` and `OFFSET` are incorporated as already defined numbers or values converted to integers, not as arbitrary text received from the user.

27.3 Session, CSRF, and protection against account enumeration

Account enumeration consists of discovering which email addresses are registered by observing differences in responses. This is why verification-resend and password-recovery flows use neutral messages when the email address does not exist.

- `HttpOnly` prevents JavaScript from directly reading the session cookie.
- `SameSite=Lax` reduces certain automatic cross-site requests.
- CSRF adds a second proof for operations that modify data.
- Regenerating the id reduces the risk of session fixation.
- Verification resend and forgot password keep neutral responses when the email address does not exist.

27.4 CORS

CORS is a browser policy that controls which web origin may call an API when frontend and backend are at different addresses. In development, CorsMiddleware allows http://localhost:5173, credentials, the Content-Type and X-CSRF-Token headers, and the methods used by CineVerse. Preflight OPTIONS requests respond with 204.

27.5 Possible security improvements

- Store hashes of verification/reset tokens instead of usable tokens in plaintext.
- Also apply rate limiting to registration, resend, and password recovery if the application is published on the Internet.
- Review HTTPS detection when an intermediary server (reverse proxy) sits in front of PHP before relying only on `$_SERVER["HTTPS"]`.
- Avoid 0777 permissions when creating the logs folder in a production environment.

27.6 How a POST /reviews is validated across multiple layers

1. Router
 - > requires auth and csrf according to routes/api.php
2. AuthMiddleware
 - > checks `Session::has("user_id")`
3. CsrfMiddleware
 - > reads `HTTP_X_CSRF_TOKEN`
 - > `hash_equals(session token, received token)`
4. `Request::json()`
 - > reads `php://input`
 - > `json_decode(..., true)`
5. Validator
 - > `int()` / `stringTrim()`
6. ReviewValidator
 - > `movie_id > 0`
 - > `movie_title required <= 255`
 - > `rating 1..5`
 - > `comment 20..1000` and normalized
7. ReviewService
 - > one review per user/movie
 - > ownership when editing/deleting
8. PDO + MySQL
 - > prepared statements
 - > `UNIQUE` / `FOREIGN KEY` / `CHECK`

Each layer protects something different. Form validation improves the user experience, but it is not a security boundary because someone can call the API without using Vue. This is why the backend repeats the checks and MySQL maintains final constraints.

27.7 Why a password is not normalized like an email address

`Validator::stringTrim()` is used on data such as email, username, or comment before other normalization. The password is read with `Validator::string()` without `trim()`, because a space can be a real part of the password. `AuthValidator` then applies length and composition rules, and `password_hash/password_verify` handle storage and comparison.

27.8 Prepared statements and numeric pagination values

User-controlled values are sent as PDO parameters, for example `:user_id`, `:movie_id`, or `:comment`. `LIMIT` and `OFFSET` appear interpolated in some queries, but they come from backend constants and calculations on a page

value converted to an integer; no arbitrary user text is incorporated into the SQL statement. Even so, validating `page >= 1` would eliminate the negative-offset edge case described in section 26.

28. Interface states and error handling

CineVerse does not display a different screen only when an error occurs. The same SPA also represents loading states, empty results, expected validation errors, and nonexistent routes. To understand why each message appears, the state must be followed from the layer that produces it to the component that ultimately renders it. This section uses the actual frontend and backend files and, in each example, identifies which file participates, what decision it makes, and how that decision becomes visible text.

28.1 Loading, empty state, expected error, and technical failure

Before following the code, it is useful to separate situations that may look similar visually but mean different things technically.

State type	What it means	Real example in CineVerse
Loading	The operation has not finished yet.	"Loading movie".
Empty state	The operation completed successfully, but there are no useful items to display.	"We couldn't find any movies", empty favorites, or a profile with no reviews.
Expected error	The system knows the cause and can express it with a stable code.	Incorrect credentials, invalid rating, or invalid CSRF.
Technical failure	A dependency failed or an unexpected exception occurred.	MySQL unavailable, SMTP failure, or a TMDB failure.
Invalid navigation	The URL does not match a valid SPA view.	Vue Router 404 screen.

The distinction is important when diagnosing the system. "We couldn't find any movies" may appear after a successful TMDB response; a profile with zero reviews is also a valid response; and the SPA's 404 screen can be resolved without PHP receiving any request. By contrast, a MySQL outage or an unexpected exception belongs to the technical-failure category.

28.2 How to read the examples: from the cause to the visible text

In Vue, there is usually no separate HTML file for each state. A ref, a computed value, a Pinia store, a TanStack Vue Query state, or an HTTP response changes; Vue reevaluates the template; and `v-if` / `v-else-if` decides which block remains in the DOM. When text is internationalized, `vue-i18n` transforms a key into the message for the active language.

```

CAUSE
↓
file that detects or produces the situation
↓
local state / store / Vue Query / HTTP response
↓
file that consumes that state
↓
v-if / v-else-if decides which block is rendered
↓
vue-i18n obtains the text for the current language
↓
VISIBLE MESSAGE

```

The following examples repeat this pattern. Code fragments include their full path and, where applicable, the specific method or block. This makes it possible to locate each piece within the project instead of reading the code as isolated fragments.

28.3 Example 1: a nonexistent URL ends at the 404 screen

The SPA's 404 screen is resolved entirely in the browser when the URL does not match any valid route. PHP, MySQL, and TMDB do not participate in this flow.

File: frontend/src/core/router/index.ts **Block:** catch-all routes at the end of the router

```
{
  path: "/:lang(en|es)/:pathMatch(.*)"*,
  name: "NotFound",
  component: NotFoundView,
},
{
  path: "/:pathMatch(.*)"*,
  name: "NotFoundNoLang",
  component: NotFoundView,
}
```

The first catch-all preserves a valid language at the beginning of the URL; the second captures any other address that also failed to match the preceding routes. Because both rules are at the end, Vue Router reaches them only after trying the valid routes.

File: frontend/src/views/NotFoundView.vue **Block:** 404 view template

```
<h1 class="not-found-code">404</h1>
<h2 class="not-found-title">
  {{ t("notFound.title") }}
</h2>
<p class="not-found-sub">
  {{ t("notFound.message") }}
</p>
<RouterLink :to="`/${locale}`">
  {{ t("notFound.goHome") }}
</RouterLink>
```

```
/es/nonexistent-route
↓
Vue Router compares all valid routes
↓
none matches
↓
catch-all -> NotFoundView.vue
↓
t("notFound.title") + t("notFound.message")
↓
404 visible on screen
```

The router decides which component to mount; NotFoundView decides what is displayed. This separation explains why an SPA 404 does not require an HTTP 404 response from the backend.

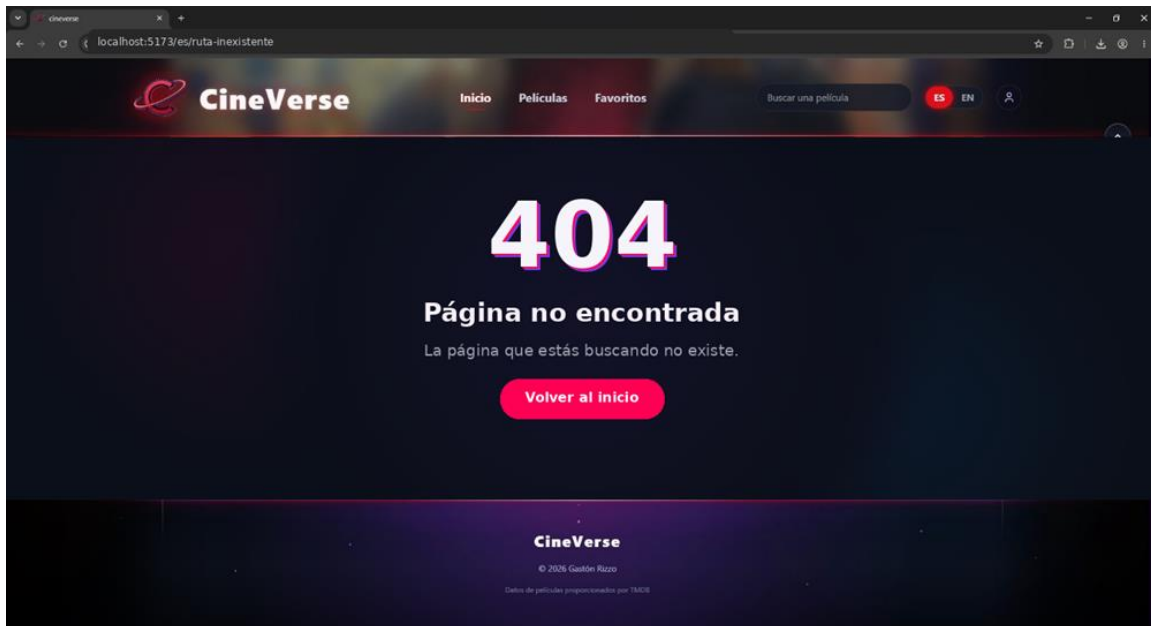


Figure. SPA 404 screen. Vue Router did not find a valid route and mounted NotFoundView inside RouterView; the PHP backend does not participate in this flow.

There is an important difference between an invalid route and a nonexistent remote resource. `/es/movies/abc` does not satisfy the numeric pattern for the id parameter and ends at the router's 404. By contrast, `/es/movies/99999999` does match the detail route; `MovieDetailView` is mounted and the subsequent TMDB request may fail. Both cases communicate "not found", but they originate in different layers.

28.4 Example 2: why "Loading movie" appears

The movie detail page does not remove the overlay merely because the request has finished. `MovieDetailView` combines two conditions: the TanStack Vue Query request must no longer be loading, and the `MovieHero` component must have reported that its poster is ready to display. This is why the state depends on several files.

File: `frontend/src/features/movies/views/MovieDetailView.vue` **Block:** `loadingstate` and `onHeroReady()` handler

```
const { data, isLoading, isError } = useMovieDetail(id, locale);
const heroReady = ref(false);

function onHeroReady() {
  heroReady.value = true;
}

<MovieHero
  v-if="movie"
  ...
  @ready="onHeroReady"
/>

<div v-if="isLoading || !heroReady" class="loading-overlay">
  ...
  <h1>{{ t('movieDetail.loadingTitle') }}</h1>
  <p>{{ t('movieDetail.loadingSubtitle') }}</p>
</div>
```

`isLoading` comes from the `useMovieDetail()` composable. `heroReady`, by contrast, is a ref owned by the view. The expression `isLoading || !heroReady` means: keep the loader visible while data is still being fetched or while `MovieHero` has not yet emitted ready.

File: `frontend/src/features/movies/composables/movie-detail/useMovieDetail.ts` **Block:** `useQuery()` that retrieves the complete detail

```
return useQuery<MovieDetailResult>({
  queryKey: ["movie-detail", id, language],
```

```

    retry: false,
    queryFn: async () => {
      const movieData = await getMovieDetail(id, language.value);
      if (!movieData) throw new Error("Movie not found");
      const [creditsData, videosData, similarData] = await Promise.all([
        getMovieCredits(id, language.value),
        getMovieVideos(id, language.value),
        getSimilarMovies(id, 1),
      ]);
      ...
    },
  });
});

```

While this query is pending, Vue Query keeps `isLoading` active. When it finishes, that state is no longer sufficient on its own: the view still waits for the hero event.

File: frontend/src/features/movies/components/movie-detail/MovieHero.vue **Block:** `onPosterLoad()` and `checkReady()`

```

function onPosterLoad() {
  posterLoaded.value = true;
  checkReady();
}

function checkReady() {
  if (posterLoaded.value) {
    setTimeout(() => {
      isReady.value = true;
      intro.value = true;
      emit("ready");
    }, 150);
  }
}

```

The current implementation is specific: `MovieHero` waits for the poster to load and then applies a 150 ms delay before emitting `ready`. If the movie has no `poster_path`, the watcher considers the poster ready so it does not block the interface. There is no generic check for "all images and the layout"; the actual condition is the poster plus that short delay.

```

MovieDetailView mounts
↓
useMovieDetail() -> Vue Query -> isLoading = true
↓
isLoading || !heroReady = true -> "Loading movie" appears
↓
TMDB requests finish -> isLoading = false
↓
MovieHero loads the poster -> waits 150 ms -> emit("ready")
↓
onHeroReady() -> heroReady = true
↓
false || false = false -> the overlay disappears

```

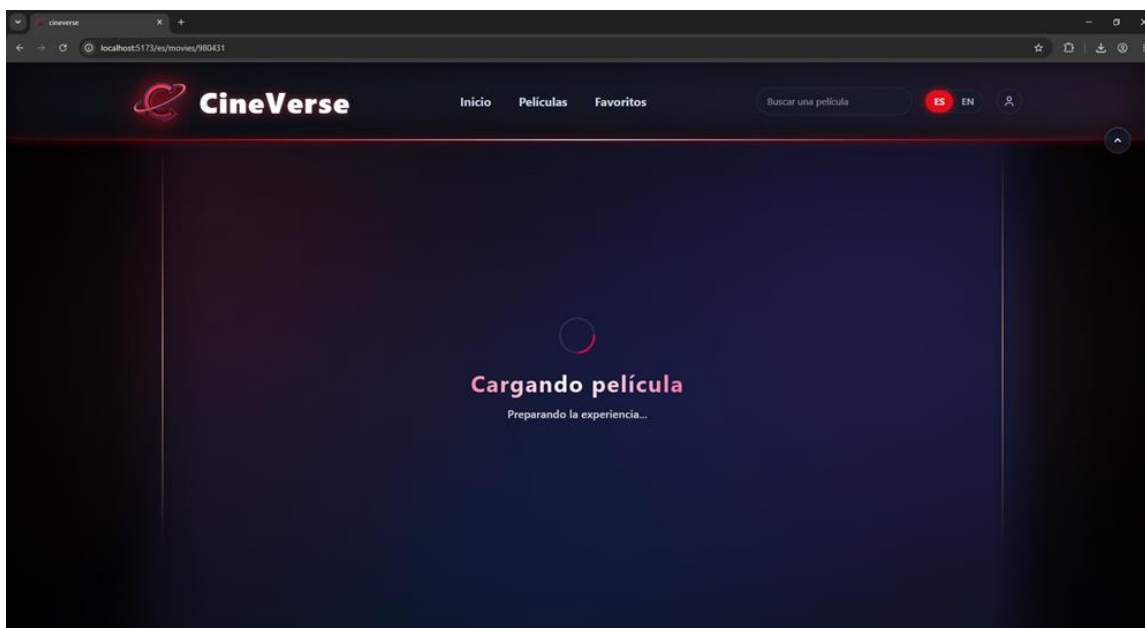


Figure. Movie-detail loading state. The overlay remains visible while the query is still loading or MovieHero has not yet emitted ready after loading the poster.

28.5 Example 3: "We couldn't find any movies" is an empty state, not necessarily an error

This case illustrates the difference between "the request failed" and "the request finished, but there was nothing useful left to display". `MoviesView` obtains pages through `useMoviesQuery`; `MovieResults` receives those pages and only after processing them decides whether to display results, an error, or an empty state.

File: `frontend/src/features/movies/composables/useMoviesQuery.ts` **Block:** `explorer useInfiniteQuery()`

`useMoviesQuery` uses `TanStack Vue Query` to produce `data.pages`, `isLoading`, `error`, `hasNextPage`, and `isFetchingNextPage`. Depending on the search and filters, its `queryFn` calls functions in `movies.service.ts`, which make the requests to TMDB.

File: `frontend/src/features/movies/components/MovieResults.vue` **Block:** `props.data watcher`

```
hasProcessedData.value = false;

const pages = data?.pages;
if (!pages || pages.length === 0) return;

const allMovies = pages.flatMap(p => p.results);

validMovies.value = allMovies.filter(
  movie => movie.poster_path && !movie.poster_path.includes("null")
);

hasProcessedData.value = true;
```

`MovieResults` flattens all pages and removes movies whose `poster_path` is not usable. `hasProcessedData` prevents the empty state from appearing while the list is still being recalculated.

File: `frontend/src/features/movies/components/MovieResults.vue` **Block:** `loading/error and results/empty-state container`

```
<div v-if="isLoading" class="cinema-loading">...</div>
<div v-else-if="error" class="empty-state">...</div>

<div name="movie">
  <div v-if="validMovies.length">...</div>
  <div v-else-if="hasProcessedData && validMovies.length === 0"
    class="empty-state">
    {{ t("common.emptyResultsTitle") }}
  </div>
</div>
```

```

    </div>
  </div>

  MoviesView / filters
  ↓
  useMoviesQuery.ts -> useInfiniteQuery()
  ↓
  movies.service.ts -> TMDB
  ↓
  data.pages
  ↓
  MovieResults.vue -> flatMap + poster_path filter
  ↓
  hasProcessedData = true
  ↓
  validMovies.length === 0
  ↓
  "We couldn't find any movies"

```

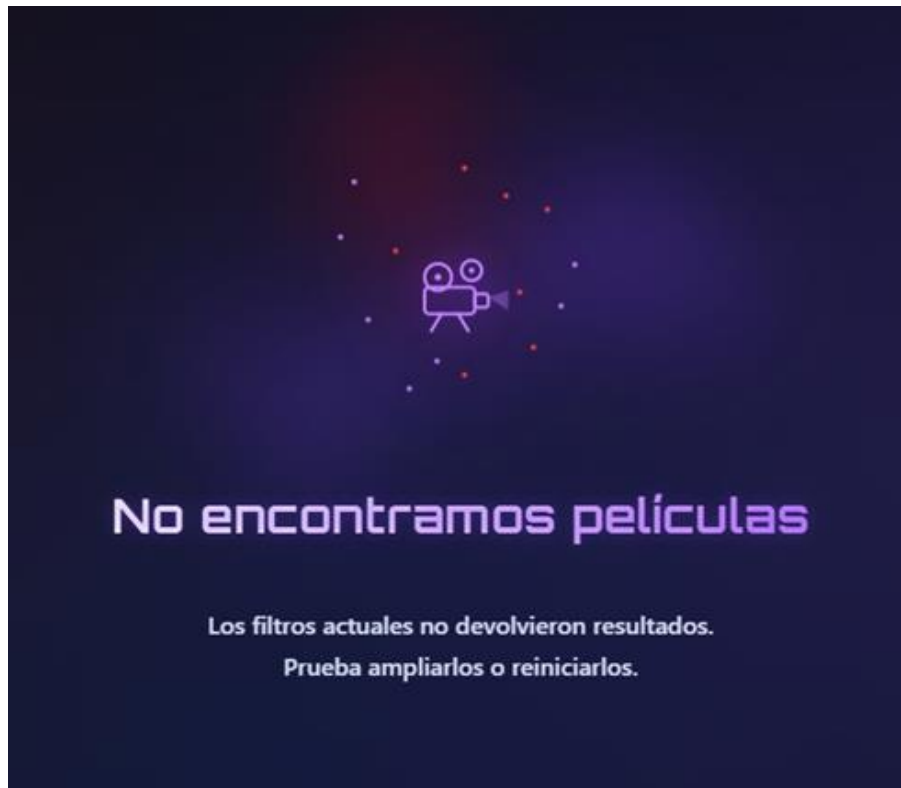


Figure. Explorer empty state. The request has already been processed, but the final list of usable movies is empty; therefore, Vue displays "We couldn't find any movies".

What this state does not mean: it does not prove that TMDB is down. If `props.error` exists, `MovieResults` uses the general error keys. The "We couldn't find any movies" message belongs to the path in which there is no error and the processed collection is empty.

28.6 Example 4: empty favorites without any HTTP request

Favorites is the simplest example showing that an empty screen does not require a server. The state originates in Pinia, is persisted in `localStorage`, and `MoviesFavoritesView` consumes it directly.

File: `frontend/src/main.ts` **Block:** `pinia-plugin-persistedstate` registration

```

const pinia = createPinia();
pinia.use(piniaPluginPersistedState);

const app = createApp(App);
app.use(pinia);

```

File: frontend/src/features/movies/stores/useMoviesFavoritesStore.ts **Block:** favorites store and persistence

```
const favorites = ref<Movie[]>([]);

return {
  favorites,
  isFavorite,
  addFavorite,
  removeFavorite,
  clearFavorites,
  favoritesCount
};

{
  persist: true,
}
```

File: frontend/src/features/movies/views/MoviesFavoritesView.vue **Block:** empty-hero state

```
<div v-if="favoritesStore.favorites.length === 0" class="empty-hero">
  <h1>{{ t('favorites.emptyTitle') }}</h1>
  <p>{{ t('favorites.emptySubtitle') }}</p>
</div>
```

```
localStorage
↓
pinia-plugin-persistedstate restores the store
↓
useMoviesFavoritesStore.ts -> favorites = []
↓
MoviesFavoritesView.vue
↓
favorites.length === 0
↓
Vue displays "You don't have any favorites yet"
```

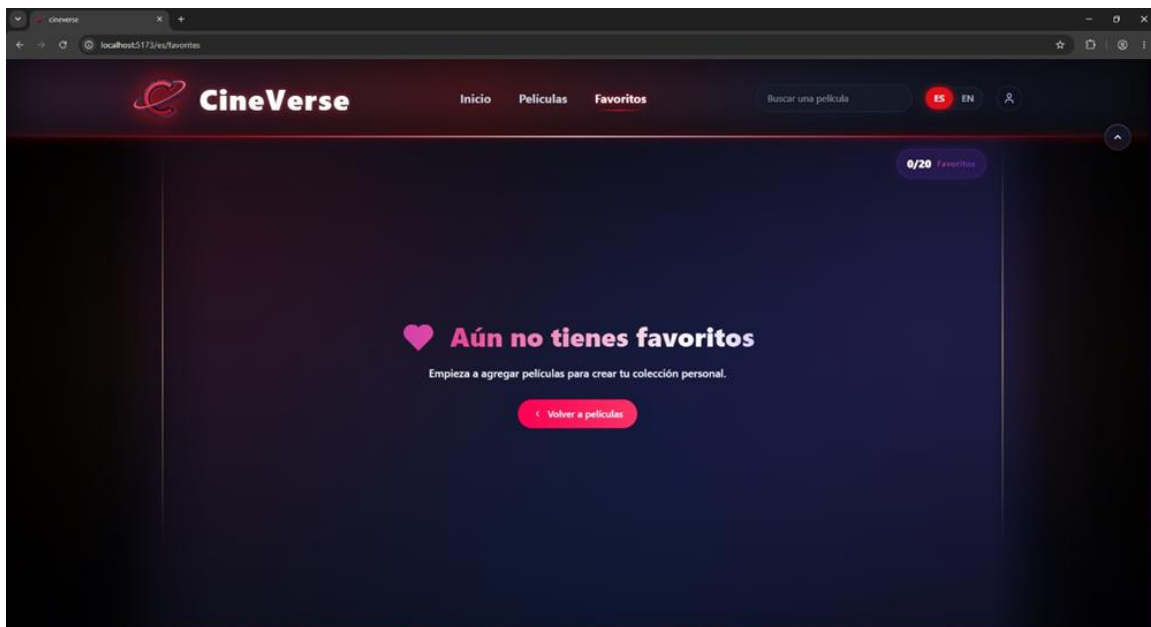


Figure. Emptyfavorites. The state originates in Pinia/localStorage and is resolved entirely in the browser; Axios, PHP, MySQL, and TMDB are not executed to display this message.

28.7 Example 5: a valid profile with zero reviews

The profile contrasts with Favorites because here the full frontend -> backend -> MySQL chain does participate. An account with no reviews does not generate an error: the backend correctly responds with reviews_count = 0 and latest_reviews = []. Vue interprets that empty array and displays the corresponding message.

File: frontend/src/features/profile/services/profile.service.ts **Block:** getProfile()

```
export const getProfile = async (): Promise<ProfileResponse> => {
  const { data } = await backendApi.get<ProfileResponse>("/profile");
  return data;
};
```

File: backend/routes/api.php **Block:** GET /profile

```
"/profile" => [
  "handler" => [ProfileController::class, "getProfile"],
  "auth" => true,
  "csrf" => false
]
```

The route requires a valid session before reaching the Controller. ProfileController obtains user_id from the session and calls ProfileService::getProfile(); the Service establishes that the profile shows at most the six most recent reviews and delegates the queries to the Model.

File: backend/models/Profile.php **Block:** Profile::getProfile()

```
$reviewsCountStmt = $pdo->prepare(
  "SELECT COUNT(*) AS total FROM reviews WHERE user_id = ?"
);
...
$latestReviewsStmt = $pdo->prepare(
  "SELECT id, movie_id, movie_title, rating, comment, created_at
  FROM reviews
  WHERE user_id = ?
  ORDER BY created_at DESC
  LIMIT " . $latestReviewsLimit
);

return [
  "reviews_count" => $reviewsCount,
  "latest_reviews" => $latestReviews
];
```

File: frontend/src/features/profile/views/ProfileView.vue **Block:** loadProfile() and no-reviews state

```
const response = await getProfile();

reviewsCount.value = response.data.reviews_count;
latestReviews.value = response.data.latest_reviews;

<div v-if="latestReviews.length === 0" class="no-reviews">
  <p>{{ t("profile.noReviewsYet") }}</p>
</div>
```

```
ProfileView.vue
  ↓ getProfile()
profile.service.ts -> backendApi.get("/profile")
  ↓
routes/api.php -> ProfileController::getProfile()
  ↓
ProfileService::getProfile()
  ↓
Profile::getProfile() -> MySQL
  ↓
reviews_count = 0 / latest_reviews = []
  ↓ JSON response
ProfileView -> latestReviews.length === 0
  ↓
"You haven't written any reviews yet."
```

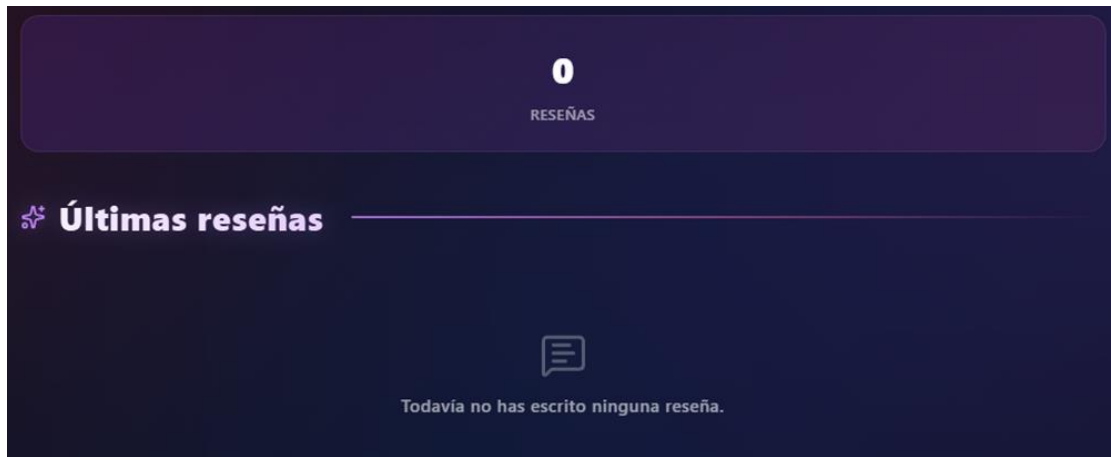


Figure. Profile with no reviews. The backend responded successfully; latest_reviews arrived as an empty array and the template condition activated the empty-state message.

28.8 Example 6: incorrect credentials from the form to the Login banner

Sign-in provides an end-to-end example of a controlled error. The view first validates what it can check without the server; if the form passes that validation, the frontend sends POST /login. The decision about whether the password matches can only be made in the backend, because that is where the MySQL password_hash is stored.

File: frontend/src/features/auth/services/auth.service.ts **Block:** loginUser()

```
export const loginUser = async (payload: LoginPayload): Promise<LoginResponse> => {
  const { data } = await backendApi.post<LoginResponse>("/login", payload);
  return data;
};
```

File: backend/services/AuthService.php **Block:** AuthService::login()

```
$user = User::findByEmail($this->pdo, $email);

if (!$user || !password_verify($password, $user["password_hash"])) {
  ... // crea o incrementa login_attempts
  throw new RuntimeException("INVALID_CREDENTIALS");
}
```

AuthService also records the failed attempt for the IP + email combination. The exception is not sent raw to the browser: AuthController translates it into the API's standard public response.

File: backend/controllers/AuthController.php **Block:** AuthController::login() catch

```
if ($e->getMessage() === "INVALID_CREDENTIALS") {
  Response::error(
    $e->getMessage(),
    401
  );
  return;
}
```

File: frontend/src/features/auth/views/LoginView.vue **Block:** login() catch

```
const code = error.response?.data?.code as string;

switch (code) {
  case "INVALID_CREDENTIALS":
    formError.value = "invalidCredentials";
    break;
  ...
}

<div v-if="formError" class="form-error-banner">
  {{ t(`auth.errors.${formError}`) }}
</div>
```

The final key is located in `frontend/src/core/i18n/es.json`: `auth.errors.invalidCredentials = "Usuario o contraseña incorrectos."`. This is why the backend communicates a stable cause while the frontend decides how to present it.

```

LoginView validates the form
↓
auth.service.ts -> POST /login
↓
AuthController -> AuthService -> User::findByEmail() / password_verify()
↓
INVALID_CREDENTIALS
↓
HTTP 401 + JSON { success:false, code:"INVALID_CREDENTIALS" }
↓
Axios rejects the Promise -> LoginView catch
↓
formError = "invalidCredentials"
↓
vue-i18n
↓
"Incorrect username or password."

```



Figure. Incorrect-credentials banner. The visible text is the final step in a chain that begins with the AuthService check, passes through HTTP 401, and ends in formError inside LoginView.

This example also separates frontend validation from backend authentication. An empty field or invalid format can be detected before sending the request; verifying a password requires querying the user stored on the server.

28.9 Example 7: why "Invalid link" appears when resetting the password

ResetPasswordView distinguishes three states: checking, valid, and invalid. This case is especially useful because there are two paths to invalid: one entirely local, when the URL contains no token, and one remote, when a token exists but backend validation fails.

File: `frontend/src/features/auth/views/ResetPasswordView.vue` **Block:** `onMounted()` that validates the token

```

const tokenStatus = ref<"checking" | "valid" | "invalid">("checking");

onMounted(async () => {
  if (!token) {
    tokenStatus.value = "invalid";
    return;
  }

  try {
    await validateResetToken(token);
    tokenStatus.value = "valid";
  } catch {
    tokenStatus.value = "invalid";
  }
});

```

If no token exists, the return occurs before any HTTP request. If one does exist, the `validateResetToken()` service function sends `GET /reset-password/validate`.

File: `frontend/src/features/auth/services/auth.service.ts` **Block:** `validateResetToken()`

```
export const validateResetToken = async (token: string) => {
  const { data } = await backendApi.get("/reset-password/validate", {
    params: { token }
  });
  return data;
};
```

backend/routes/api.php routes that GET to AuthController::validateResetToken(). The Controller normalizes the parameter and calls the Service. AuthService does not duplicate the validation: it delegates to a private method also shared by the actual password-change operation.

File: backend/services/AuthService.php **Block:** getValidPasswordResetUser()

```
$user = User::findByPasswordResetToken($this->pdo, $token);

if (!$user) {
  throw new RuntimeException("INVALID_PASSWORD_RESET_TOKEN");
}

if (strtotime($user["password_reset_expires_at"]) < time()) {
  throw new RuntimeException("PASSWORD_RESET_TOKEN_EXPIRED");
}
```

File: frontend/src/features/auth/views/ResetPasswordView.vue **Block:** template selection based on tokenStatus

```
<template v-if="tokenStatus === 'checking'"> ... </template>
<template v-else-if="tokenStatus === 'valid'"> ... </template>
<template v-else>
  <h1>{{ t("auth.resetPassword.invalidTitle") }}</h1>
  <p>{{ t("auth.resetPassword.invalidMessage") }}</p>
</template>
```

CASE A: ?token=... is missing
ResetPasswordView -> tokenStatus = "invalid" -> no HTTP

CASE B: token is present
ResetPasswordView
↓ validateResetToken()
auth.service.ts -> GET /reset-password/validate
↓
AuthController -> AuthService -> getValidPasswordResetUser()
↓
token missing / expired -> error response
↓
Axios rejects -> onMounted catch
↓
tokenStatus = "invalid"
↓
"Invalid link"

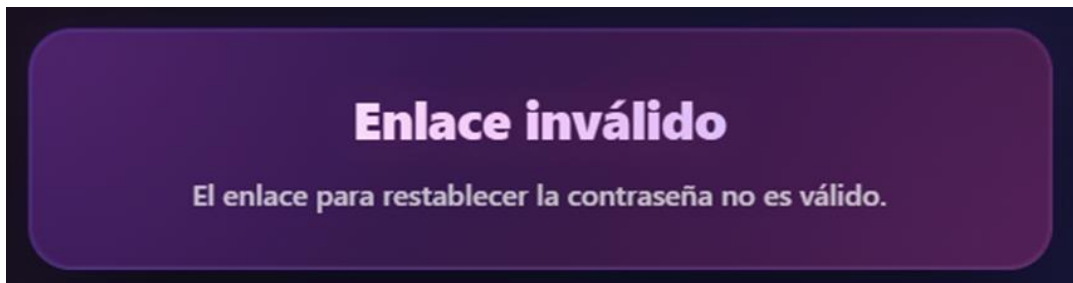


Figure. ResetPasswordView "Invalid link" state. It may appear without calling the backend if the token is missing, or after a rejected remote validation.

Current limitation. The onMounted catch does not inspect error.response.data.code: any rejection from validateResetToken(), including an infrastructure outage during that check, ends in tokenStatus = "invalid". Therefore, the screen does not visually distinguish an invalid token from certain technical failures.

28.10 Three different meanings of "404" or "not found"

CineVerse can encounter the idea of "not found" in different layers. Distinguishing them prevents attributing the problem to the wrong system.

Source	Example	What happens
Vue Router	/es/nonexistent-route	No SPA route matches and NotFoundView is mounted. The backend is not queried.
PHP backend	GET to a nonexistent endpoint	The PHP Router may return HTTP 404 with ROUTE_NOT_FOUND.
TMDB	/movie/{id} does not exist	The remote request fails; Vue Query marks isError and the detail view displays its error state.

The HTTP number or the phrase "not found" alone does not explain which screen will appear. First identify which layer produced the state and which frontend component consumes that result.

28.11 How the backend expresses controlled errors

When the problem originates in the application's own API, CineVerse combines two levels: the HTTP status describes the general category and the code field identifies the stable cause the frontend can interpret. Central construction is in backend/core/http/Response.php.

File: backend/core/http/Response.php **Block:** Response::error()

```
public static function error(string $code, int $statusCode): void {
    self::json(
        $statusCode,
        false,
        $code
    );
}
```

Situation	HTTP	code
Invalid JSON	400	INVALID_JSON_BODY
Unauthenticated	401	AUTH_REQUIRED
Incorrect credentials	401	INVALID_CREDENTIALS
Invalid CSRF	403	INVALID_CSRF_TOKEN
Nonexistent resource	404	REVIEW_NOT_FOUND / ROUTE_NOT_FOUND
Duplicate review	409	REVIEW_ALREADY_EXISTS
Validation	422	INVALID_RATING, COMMENT_TOO_SHORT, etc.
Sign-in rate limit	429	LOGIN_TEMPORARILY_BLOCKED
Unexpected error	500	INTERNAL_SERVER_ERROR
MySQL unavailable	503	DATABASE_UNAVAILABLE

Example of expected validation: a rating of 6 does not mean the server is broken. ReviewValidator rejects the data and the backend responds with a known cause.

```
POST /reviews
{
```

```

"movie_id": 550,
"movie_title": "Fight Club",
"rating": 6,
"comment": "Comment of sufficient length."
}

-> HTTP 422
-> { "success": false, "code": "INVALID_RATING" }

```

CSRF is another early cutoff: if a protected write does not contain a valid X-CSRF-Token, `CsrfMiddleware` responds with HTTP 403 and `INVALID_CSRF_TOKEN` before the operation's Controller runs.

28.12 Technical failures, external dependencies, and imperfect classification

A technical failure follows a different path. If MySQL is unavailable during startup, `backend/public/index.php` catches the `PDOException` before dispatching the route, `AppLogger` preserves the technical detail, and `Response::error()` returns a safe code to the browser.

File: `backend/public/index.php` **Block:** initial connection with `Database::connect()`

```

try {
    $pdo = Database::connect();
} catch (PDOException $exception) {
    AppLogger::error($exception->getMessage(), ...);
    Response::error("DATABASE_UNAVAILABLE", 503);
    exit;
}

Technical detail that may remain in the log:
SQLSTATE[HY000] [1049] Unknown database 'cineverse2'

Public response:
HTTP 503
{ "success": false, "code": "DATABASE_UNAVAILABLE" }

```

TMDB is also an external dependency and can fail even while PHP and MySQL continue working. In the current implementation, `MovieDetailView` and `PersonDetailView` use general Vue Query error states and do not visually distinguish every possible variation of a remote failure (for example, 404, 429, or 5xx).

Email has another imperfect classification: `EMAIL_DELIVERY_FAILED` is translated specifically in `RegisterView`, while other flows may fall back to `unknownError`. Likewise, as shown in 28.9, the initial reset validation may turn an infrastructure failure into `tokenStatus = "invalid"`. The document describes current behavior, not an idealized behavior.

28.13 What is logged and what is not

The presence of a negative message on screen does not mean an ERROR-level log should be written. Normal states and expected validations are part of normal operation; technical logs are reserved for failures that require diagnosis.

Situation	ERROR log?	Reason
Search with no results	No	Valid empty state.
Empty favorites	No	Normal local state.
Profile with 0 reviews	No	Successful API response.
Incorrect credentials	Usually no	Expected authentication result.
Invalid rating	No	Controlled validation.
MySQL unavailable	Yes	Technical infrastructure failure.

Situation	ERROR log?	Reason
Unexpected SMTP exception	Yes	Technical failure requiring diagnosis.

The internal AppLogger and Monolog implementation - path, RotatingFileHandler, LineFormatter, and daily rotation - is covered in section 29. What matters here is the separation between the safe state received by the user and the technical detail that may be logged.

28.14 Final map: from the cause to the screen

The table summarizes the preceding examples and makes it possible to quickly identify which layer originates each state and which decision ultimately makes it visible.

Screen/ state	Primary source	HTTP?	Final condition or decision
SPA 404	Vue Router	No	catch-all -> NotFoundView
Loading movie	Vue Query + MovieHero	Yes, TMDB in progress	isLoading !heroReady
We couldn't find any movies	MovieResults	Yes, TMDB	hasProcessedData && validMovies.length === 0
No favorites	Pinia + localStorage	No	favorites.length === 0
Profile with no reviews	PHP/MySQL + Vue	Yes	latestReviews.length === 0
Incorrect credentials	PHP/MySQL + LoginView	Yes	INVALID_CREDENTIALS -> formError
Invalid link	ResetPasswordView and, sometimes, PHP/MySQL	Sometimes	tokenStatus === "invalid"

The practical way to read any CineVerse case is always the same: locate the cause, identify the file that produces it, follow the data or code as it crosses layers, find the reactive state received by Vue, and finally observe which template condition selects the text. This flow explains not only which message appears, but why it appears and which real parts of the system participated in reaching it.

29. Logs: error logging and diagnosis

A log is a technical record that the backend preserves so failures can be investigated without showing internal details to the user. CineVerse does not write directly to a file from each Controller. It uses a project-specific class, AppLogger, as the entry point and delegates writing, rotation, and formatting to Monolog.

This section follows the complete flow from AppLogger::error() to the daily file. It first explains AppLogger and Monolog, then shows how the logger is configured, and only afterward analyzes actual entries.

```

Controller / bootstrap
-> AppLogger::error(...)
-> AppLogger::get()
-> Monolog Logger
-> RotatingFileHandler
-> LineFormatter
-> cineverse-backend-YYYY-MM-DD.log

```

29.1 AppLogger and Monolog: two different responsibilities

AppLogger.php belongs to the CineVerse codebase. Its purpose is to provide a single entry point for logging errors and hide configuration details from the rest of the backend. Controllers can call AppLogger::error() without knowing which folder stores the file, how many rotated files are retained, or which formatter Monolog uses.

Monolog, by contrast, is an external logging library. Several classes from that library participate in CineVerse: `Logger` receives the record; `RotatingFileHandler` decides to write it to rotating files; `LineFormatter` converts each record to text; and `Level::Error` sets the minimum level accepted by the handler.

```
namespace App\Core\logging;

use Monolog\Formatter\LineFormatter;
use Monolog\Handler\RotatingFileHandler;
use Monolog\Level;
use Monolog\Logger;

final class AppLogger
{
    private static ?Logger $logger = null;
    ...
}
```

The use statements do not create those objects; they only allow Monolog classes to be referenced by short names. The `$logger` property will be the shared instance that `AppLogger` configures the first time something needs to be logged.

29.2 How Monolog enters the project

Monolog is not part of PHP. It is declared as a dependency in `backend/composer.json`. When composer install is run, Composer installs the library inside `vendor` and generates `vendor/autoload.php`. `bootstrap.php` loads that autoloader at the start of every request.

```
"require": {
    "vlucas/phpdotenv": "^5.6",
    "monolog/monolog": "^3.10",
    "phpmailer/phpmailer": "^7.1"
}

// core/bootstrap.php
require_once __DIR__ . '/../vendor/autoload.php';
```

Thanks to Composer autoloading, `AppLogger` can create `Logger`, `RotatingFileHandler`, and `LineFormatter` without using `require_once` for every Monolog file. The same mechanism also loads the backend's own classes through the PSR-4 configuration explained in section 6.

29.3 How the Logger is built inside AppLogger

29.3.1 One shared instance during execution

```
private static ?Logger $logger = null;

public static function get(): Logger
{
    if (self::$logger === null) {
        self::$logger = new Logger("app");
        ...
    }

    return self::$logger;
}
```

`$logger` starts as null because no instance exists yet. The first call to `get()` enters the if and builds the `Logger`. Subsequent calls during that PHP execution reuse the same instance. "app" is the `Logger` name or channel; it is not the filename.

29.3.2 RotatingFileHandler: daily file, retention, and level

```
$logPath = self::resolveLogPath();

$handler = new RotatingFileHandler(
    $logPath,
    30,
    Level::Error
);
```

The first argument indicates the base file. The second sets 30 as the maximum number of rotated files to retain. The third sets Level::Error as the minimum level. Therefore, this handler is prepared to record ERROR and higher levels; it does not act as a general history of application INFO messages. Because the configured rotation is daily, those files will normally correspond to different dates, but the parameter limits files; it does not guarantee a retention period expressed in days.

RotatingFileHandler appends the date to the base name and automatically switches files when rotating. When the configured maximum is exceeded, Monolog can delete the oldest files to prevent unbounded growth.

29.3.3 LineFormatter: what each entry looks like

```
$handler->setFormatter(
    new LineFormatter(
        "[%datetime%] %level_name%: %message%\n",
        "Y-m-d H:i:s",
        true,
        true
    )
);

self::$logger->pushHandler($handler);
```

The first argument defines the text structure. %datetime% is replaced with the date and time, %level_name% with the level, and %message% with the message sent to Monolog. "Y-m-d H:i:s" produces dates such as 2026-09-07 07:14:16.

The first true allows line breaks within the message, which is necessary because AppLogger adds the JSON context below the main text. The second true prevents Monolog's automatically empty context and extra fields from being printed. Finally, pushHandler() connects the already configured handler to the Logger; before that call, the handler existed but was not yet associated with the logger.

29.4 From backend/.env to the physical file: CINEVERSE_LOG_PATH

CINEVERSE_LOG_PATH is an environment variable defined by CineVerse in backend/.env. It is not a PHP constant or a native Monolog option. bootstrap.php loads .env through phpdotenv and makes the value available in \$_ENV; AppLogger uses it when calculating the destination folder.

```
# backend/.env
CINEVERSE_LOG_PATH=storage/logs

// AppLogger::resolveLogPath()
$basePath = dirname(__DIR__, 2);
$relativePath =
    $_ENV["CINEVERSE_LOG_PATH"] ?? "storage/logs";
```

AppLogger.php is located inside backend/core/logging. dirname(__DIR__, 2) moves up two levels and obtains the physical backend root. If .env contains storage/logs, that relative path is combined with the root and normally ends at backend/storage/logs. The ?? operator leaves storage/logs as an internal AppLogger safeguard if the variable is unavailable; however, during normal startup bootstrap.php declares CINEVERSE_LOG_PATH as required through phpdotenv, so incomplete configuration should be detected before reaching this fallback.

```

$isAbsolutePath = preg_match(
    '/^(?:[A-Za-z]:[\\\/\\|][\\\/\\|])/',
    $relativePath
) === 1;

$logDir = $isAbsolutePath
    ? $relativePath
    : $basePath . DIRECTORY_SEPARATOR . $relativePath;

$logPath = rtrim($logDir, "\\\/")
    . DIRECTORY_SEPARATOR
    . "cineverse-backend.log";

```

The check allows two configuration forms. A relative path such as `storage/logs` is interpreted inside the backend. An absolute Windows or Linux path is used directly. `DIRECTORY_SEPARATOR` avoids manually writing the operating system's folder separator.

```

CINEVERSE_LOG_PATH=storage/logs
-> backend/storage/logs
-> cineverse-backend.log
-> cineverse-backend-2026-09-07.log

CINEVERSE_LOG_PATH=D:\logs\cineverse
-> D:\logs\cineverse
-> cineverse-backend-AAAA-MM-DD.log

```

The executed code uses `cineverse-backend.log` as the base name. An old comment inside `AppLogger.php` still mentions `app-YYYY-MM-DD.log`; the documentation follows the actual implementation, which generates `cineverse-backend-YYYY-MM-DD.log`.

29.4.1 Automatic folder creation

```

$logDir = dirname($logPath);

if (!is_dir($logDir)) {
    mkdir($logDir, 0777, true);
}

return $logPath;

```

`is_dir()` checks whether the folder exists. If it does not, `mkdir()` attempts to create it. The third argument `true` allows intermediate folders to be created. Permission `0777` is broad and convenient during development, but it is not a recommended production choice; production should assign only the permissions required by the user under which PHP runs.

29.5 AppLogger::error(): turning a message and context into an entry

The rest of the backend does not call Monolog's `Logger` directly. It uses `AppLogger::error(message, context)`. `message` contains the main technical cause, and `context` is an optional array with data useful for locating the operation.

```

public static function error(
    string $message,
    array $context = []
): void {
    self::get()->error(
        $message . "\n" .
        json_encode(
            $context,
            JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE
        )
    );
}

```

`self::get()` returns the already configured `Logger` and `->error()` creates an `ERROR`-level record. Before calling `Monolog`, `AppLogger` converts the `context` array to JSON and concatenates it to the message.

JSON_PRETTY_PRINT adds indentation and line breaks; JSON_UNESCAPED_UNICODE keeps Unicode characters readable.

This is a concrete project decision: the array is not passed as Monolog's structured second argument. The context is manually converted to text and remains inside %message%. This is why LineFormatter allows line breaks and the entry appears as a message followed by a JSON block.

29.6 What is ultimately written to the file

```
[2026-09-07 07:14:16] ERROR: technical message
{
  "controller": "...",
  "method": "...",
  "status": 500
}
```

The date comes from %datetime%, ERROR from %level_name%, and the rest from %message%. The JSON block exists because AppLogger::error() added it to the message before passing it to Monolog. The specific file depends on the day: for example, cineverse-backend-2026-09-07.log.

29.7 Real example: MySQL unavailable during startup

public/index.php opens the MySQL connection before creating the Router. If Database::connect() throws PDOException, index.php logs the technical cause and returns a stable code to the client without exposing the SQLSTATE.

```
try {
    $pdo = Database::connect();
} catch (\PDOException $exception) {
    AppLogger::error($exception->getMessage(), [
        "controller" => "Application",
        "method" => "bootstrap",
        "status" => 503,
        "pdo_code" => $exception->getCode(),
    ]);

    Response::error("DATABASE_UNAVAILABLE", 503);
    exit;
}
```

If MySQL returns error 1049 because the configured database does not exist, the file may contain an entry like the following:

```
[2026-09-07 07:14:16] ERROR: SQLSTATE[HY000] [1049] Unknown database 'cineverse2'
{
  "controller": "Application",
  "method": "bootstrap",
  "status": 503,
  "pdo_code": 1049
}
```

The browser does not receive that detail. Response::error("DATABASE_UNAVAILABLE", 503) returns a safe response. This separates two different audiences: the developer retains diagnostic information in the log while the client receives a stable code for handling the error.

29.8 Example from a Controller

Controllers use the same mechanism when a Throwable occurs that does not correspond to an expected validation or business error. ReviewController::create() adds context related to the operation it was trying to execute.

```

catch (Throwable $e) {
    AppLogger::error($e->getMessage(), [
        "controller" => "ReviewController",
        "method" => "create",
        "user_id" => $userId,
        "movie_id" => $movieId
    ]);

    Response::error(
        "INTERNAL_SERVER_ERROR",
        500
    );
}

```

The context is not fixed for every error. During bootstrap, status and pdo_code are useful; when creating a review, controller, method, user_id, and movie_id are useful. The idea is to record enough data to locate the problem without turning the log into an indiscriminate copy of the entire request.

29.9 Special example: preserving the original cause of an SMTP error

MailService transforms certain PHPMailer failures into RuntimeException("EMAIL_DELIVERY_FAILED") and preserves the original exception as previous. AuthController uses mailExceptionMessage() to log the actual technical cause instead of losing it behind the generic code.

```

private function mailExceptionMessage(
    RuntimeException $e
): string {
    return $e->getPrevious()?->getMessage()
        ?? $e->getMessage();
}

AppLogger::error(
    $this->mailExceptionMessage($e),
    [
        "controller" => "AuthController",
        "method" => "register",
        "email" => $email
    ]
);

```

In this way, the client can receive EMAIL_DELIVERY_FAILED while the file preserves, for example, an SMTP authentication message produced by PHPMailer. The distinction between a safe response and internal diagnosis connects with the error handling explained in section 28.

29.10 What should be logged, privacy, and permissions

Not every negative result should be logged as ERROR. A rating of 6, an incorrect password, or a duplicate review are expected situations that the backend knows how to classify and answer. By contrast, an unexpected PDOException, an SMTP outage, or an unforeseen Throwable are natural candidates for AppLogger.

- Log context that is useful for locating the operation: controller, method, related ids, technical code, or email address when it genuinely helps diagnosis.
- Do not log passwords, CSRF tokens, verification tokens, or password-recovery tokens.
- Keep storage/logs outside the server's public root so the files cannot be downloaded over HTTP.
- In production, replace broad permissions such as 0777 with the minimum required for PHP to create and rotate files.
- Remember that the current system is configured for ERROR or higher; it is not a history of every normal user action.

The complete flow is therefore: part of the backend detects a technical failure and calls AppLogger::error(); AppLogger obtains or reuses the Logger; Monolog passes the record to RotatingFileHandler; LineFormatter produces the text; and the daily file becomes available for diagnosis. Separately, the user receives a controlled HTTP response.

30. Local CineVerse installation

When CineVerse is cloned or downloaded from GitHub, the backend and frontend folders are already part of the project. Neither project needs to be created from scratch. What must be rebuilt or configured on each computer are the installed dependencies, the .env files, and the local services the application will use.

The current .gitignore files reflect this flow: backend/vendor and backend/.env are not versioned; on the frontend, node_modules, dist, dist-ssr, and .env are excluded. By contrast, composer.json, composer.lock, package.json, package-lock.json, and the .env.example files allow a fresh installation to be reconstructed.

Item	Expected on GitHub?	What happens in a fresh installation
backend/ and frontend/	Yes	They contain the CineVerse source code and already exist after cloning or downloading the repository.
backend/vendor/	No	Composer rebuilds it when composer install is run.
frontend/node_modules/	No	npm rebuilds it when npm install is run.
frontend/dist/	No	It is a build output; it is generated when the frontend is built.
.env.example	Yes	It serves as a template listing the required variable names.
.env	No	It is created locally with values specific to that computer and may contain credentials.

30.1 Required programs and services

Component	Use in CineVerse
Visual Studio Code	It is the editor used in this guide to open backend and frontend and work with the integrated terminal. Another editor can be used, but the screenshots and steps follow VS Code.
PHP 8.1 or later	Runs the PHP backend and its dependencies. The version can be checked with <code>php -v</code> or with the PHP executable installed on the computer.
Composer	Installs the PHP dependencies declared in <code>composer.json</code> and <code>composer.lock</code> and generates <code>vendor/autoload.php</code> .
Node.js + npm	Installs frontend dependencies and makes it possible to run Vite and TypeScript tools. The project resolves Vite 7.3.6, which requires Node.js ^20.19.0 or >=22.12.0.
MySQL 8	Database engine that stores users, reviews, and login_attempts. The CineVerse SQL file uses MySQL 8 features.
MySQL Workbench	Graphical tool used in this guide to open and import the SQL file and verify the database. It is not required for CineVerse to run once the database has been installed.
XAMPP / Apache	In the documented development environment, XAMPP's Apache serves backend/public through <code>cineverse.local</code> .
TMDB Read Access Token	Credential used by the frontend to query the TMDB API.
SMTP server	Required if verification and password-recovery emails are to be tested.

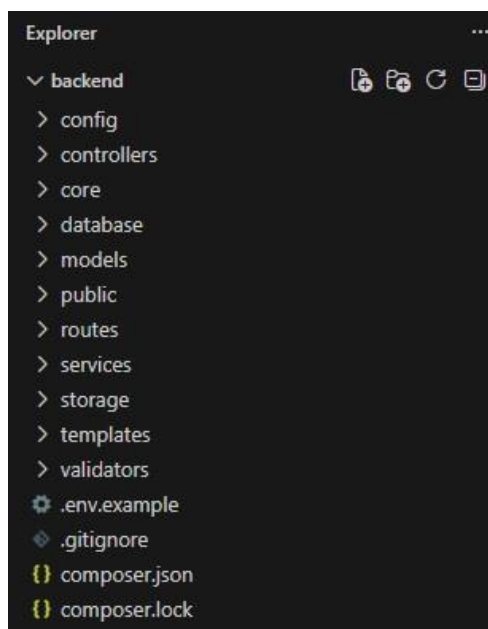
Component	Use in CineVerse
Postman (optional)	Tool used in this documentation to manually test PHP backend endpoints without depending on the Vue interface. It makes it possible to inspect methods, JSON, headers, cookies, and responses.

The backend does not use Laravel or any other framework, so there is no `php artisan serve` command. In the documented primary development environment, XAMPP's Apache serves `backend/public` directly and `.htaccess` rewrites routes so they reach `public/index.php`.

30.2 Opening the backend, installing its dependencies, and configuring `.env`

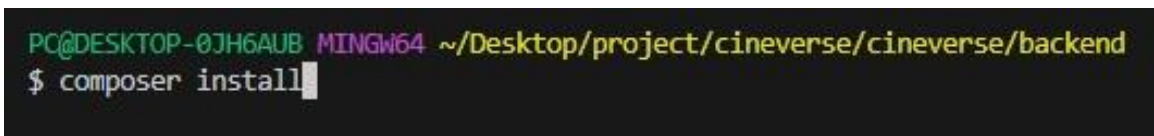
The backend folder already contains the CineVerse PHP code. To install the project on a new computer, you do not create a new backend: you open that folder, install its dependencies, and complete the local configuration.

Step 1. Open Visual Studio Code and choose `File > Open Folder`. Select the backend folder that came with the project.



CineVerse backend opened as a project folder in Visual Studio Code.

Step 2. With backend open, choose `Terminal > New Terminal`. The integrated terminal should be located inside the backend folder. If the path ends in `/backend`, there is no need to run `cd backend`.



Integrated terminal located inside backend before running Composer.

Step 3. Run Composer to rebuild the PHP dependencies declared by the project:

```
composer install
```

Composer reads `composer.json` and `composer.lock`, downloads the corresponding versions, and creates `vendor`. Dependencies used by CineVerse include `phpdotenv` for reading environment variables, `Monolog` for logging, and `PHPMailer` for email. Composer also generates `vendor/autoload.php`, which the backend uses to load classes and libraries.

Step 4. Create `backend/.env` using `backend/.env.example` as the template. `.env` contains values specific to that computer and must not be published or made accessible over HTTP.

The file must define the runtime environment, the MySQL server connection, the log path, the SMTP server, and the frontend URL. A neutral example for a local installation is:

```
APP_ENV=local
APP_DEBUG=true

DB_HOST=localhost
DB_PORT=3306
DB_NAME=cineverse
DB_USER=user
DB_PASSWORD=password
DB_CHARSET=utf8mb4

CINEVERSE_LOG_PATH=storage/logs

MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=user
MAIL_PASSWORD=password
MAIL_FROM=no-reply@example.com
MAIL_FROM_NAME=CineVerse

FRONTEND_URL=http://localhost:5173
```

The values above are illustrative. DB_HOST, DB_PORT, DB_USER, and DB_PASSWORD must match the MySQL server actually used by that installation; SMTP details must also be replaced with valid credentials if email delivery will be tested.

Alternative case. If Visual Studio Code was opened at a root folder that contains backend and frontend instead of opening backend directly, the terminal will be one level higher and `cd backend` must be run before composer install.

30.3 Database

CineVerse includes an SQL file prepared to build the database used by the backend and load the project's test data. Within the repository, the file is located at `database/cineverse.sql`. There is no need to create the database or tables manually: the script itself contains the required instructions.

The script creates the cineverse database if it does not already exist, selects it, and rebuilds the active users, login_attempts, and reviews tables. It then loads the test users and reviews. It also contains `DROP TABLE IF EXISTS` statements to remove previous versions of those tables and some legacy tables no longer used by the current PHP backend. For that reason, this procedure is intended to prepare or rebuild a local test environment and must not be run against a database whose data needs to be preserved.

The server must be MySQL 8 and must be the same server the backend will connect to afterward. The DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, and DB_CHARSET values in `backend/.env` must describe that same MySQL installation.

30.3.1 Preparing the database with MySQL Workbench

MySQL Workbench is used in this guide as a graphical client to connect to the MySQL server, open the script, and execute its statements. Workbench does not contain the database or replace the server: it operates on the connection selected by the developer.

Step 1. Locate `cineverse.sql` in the project root, alongside `backend` and `frontend`. This is the database script to use for this installation. There is no need to copy its statements manually or create the database or tables one by one.

Step 2. Start the MySQL 8 server that CineVerse will use. Open MySQL Workbench and connect to that server with the corresponding host, port, username, and password. The user used to run the script must have permission to create the database, create and drop tables, and insert data.

Step 3. With the correct connection already open, select `File > Open SQL Script` and choose the file located in Step 1. Workbench will display its contents in an SQL editor tab.

Step 4. Before running anything, verify again that the script tab is associated with the MySQL server intended for the local CineVerse environment. The file drops existing tables before rebuilding them, so it must not be run against a database whose data needs to be preserved. There is no need to create a schema named cineverse beforehand: the script itself executes CREATE DATABASE IF NOT EXISTS cineverse and then USE cineverse.

Step 5. Execute the entire script. To avoid running only the statement where the cursor is positioned, select all the content and use the Workbench option that executes the selection or the full script. During this process, cineverse is created and selected, users, login_attempts, and reviews are rebuilt, and the test users and reviews are loaded.

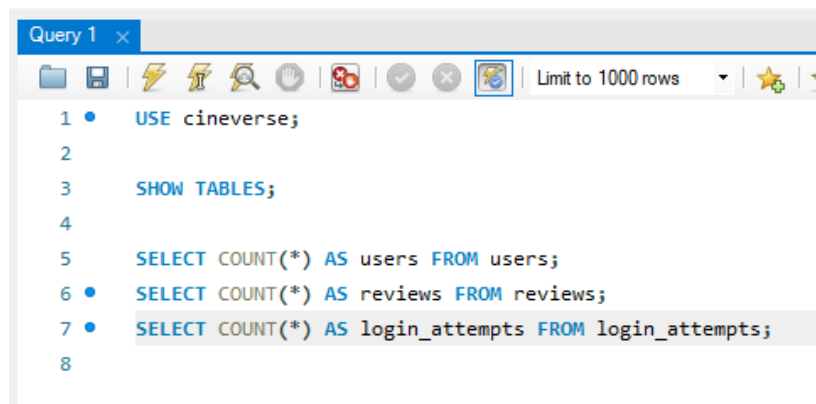
Step 6. When execution finishes without errors, refresh the Schemas panel in MySQL Workbench. The cineverse database should appear and, inside Tables, the users, login_attempts, and reviews tables.

Step 7. Run the following verification queries. SHOW TABLES checks the visible schema structure, and the COUNT(*) queries make it possible to verify how many records were loaded into each table:

```
USE cineverse;

SHOW TABLES;

SELECT COUNT(*) AS users FROM users;
SELECT COUNT(*) AS reviews FROM reviews;
SELECT COUNT(*) AS login_attempts FROM login_attempts;
```



Queries used to verify the structure and data after running the CineVerse SQL file.

Table	Expected result after a clean load
users	5.000
reviews	24.322
login_attempts	0

After a clean load, the expected result is 5,000 rows in users, 24,322 in reviews, and 0 in login_attempts. The SQL file does not load initial records into login_attempts: that table begins to be used when the backend records failed sign-in attempts during application use.

Step 8. Verify that backend/.env points to the same MySQL server used in Workbench. A correctly created database does not automatically connect the backend: host, port, and credentials must correspond to that installation, and DB_NAME must be cineverse.

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=cineverse
DB_USER=<MySQL user>
DB_PASSWORD=<MySQL password>
DB_CHARSET=utf8mb4
```

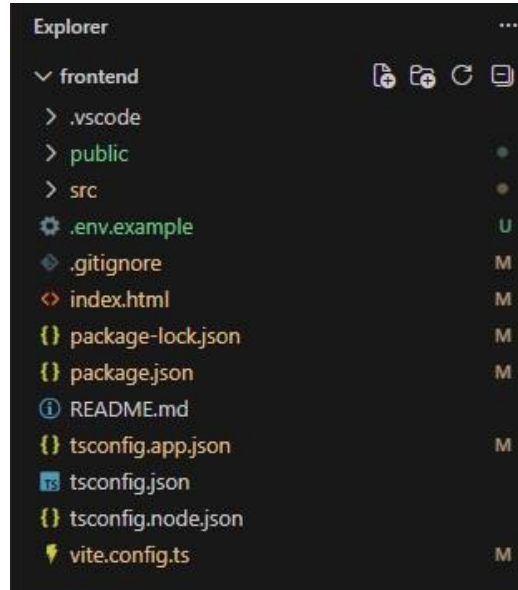
The DB_HOST, DB_PORT, DB_USER, and DB_PASSWORD values depend on the local installation. DB_NAME=cineverse matches the database created and selected by the script. DB_CHARSET=utf8mb4 causes

the PDO connection to use utf8mb4, consistent with the character set defined by the schema. If Workbench connected to a different host or port, those same details must be reflected here.

30.4 Opening the frontend, installing its dependencies, and configuring .env

The frontend folder is also part of the project downloaded from GitHub. It contains the Vue code, package.json, and package-lock.json. node_modules is normally not versioned because it can be reconstructed from those files.

Step 1. Open VisualStudio Code and choose File > Open Folder. Select the CineVerse frontend folder.



CineVerse frontend opened as a project folder in Visual Studio Code.

Step 2. Choose Terminal > New Terminal. If frontend was opened directly as the project folder, the terminal should already be located inside frontend and there is no need to run cd frontend.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse/frontend (main)
$ npm install
```

Integrated terminal located inside frontend before installing dependencies.

Step 3. Install the dependencies declared in package.json and pinned in package-lock.json:

```
npm ci
```

npm ci installs exactly the versions pinned in package-lock.json and creates node_modules from a clean installation, making it the preferred command for reproducing the project. npm install remains useful when dependencies are added or updated. Neither command keeps the frontend running; daily startup is explained in section 31.

Step 4. Before completing frontend/.env, create or sign in to a TMDb account and open Account Settings > API. The API Read Access Token is obtained there. Then copy frontend/.env.example to frontend/.env and complete the values required for that installation. Sections 3.3-3.5 explain this process with screenshots and the complete credential flow.

```
VITE_TMDB_BASE_URL="https://api.themoviedb.org/3"
VITE_TMDB_TOKEN="token"
VITE_BACKEND_URL="/api"
```

VITE_TMDB_BASE_URL indicates where the TMDb v3 API is located. VITE_TMDB_TOKEN must contain the API Read Access Token obtained from the account's API settings; tmdbApi.ts sends it as Authorization: Bearer.

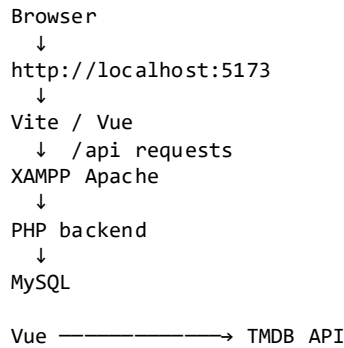
VITE_BACKEND_URL indicates how Vue reaches CineVerse's own API; during development, /api is resolved through the Vite proxy.

Alternative case. If a root folder containing frontend and backend was opened, then cd frontend is run before npm install. In this guide, the main flow consists of opening frontend directly in Visual Studio Code.

31. Running CineVerse in development

Once the application is installed, local development depends on several pieces with different roles. Vite serves the Vue frontend, XAMPP's Apache serves the PHP backend, and MySQL provides the database configured in backend/.env. TMDB is an external service queried directly from the frontend.

Initial configuration must be distinguished from daily startup. The VirtualHost and hosts file are normally configured only once per computer. Apache, the configured MySQL server, and Vite, however, must be available whenever CineVerse is being used for development.



Part	Development address or reference
Vue frontend / Vite	http://localhost:5173
PHP backend / Apache	http://cineverse.local
Application API used by Vue	/api
Example backend endpoint	http://cineverse.local/user
TMDB API	https://api.themoviedb.org/3

31.1 Configuring the Apache VirtualHost step by step

The VirtualHost allows Apache to associate the name cineverse.local with the backend's public folder. This configuration is normally performed once on each computer, unless the project is later moved to another path.

Step 1. Find the absolute path to backend/public on the computer. This is the only backend folder Apache should expose directly.

Step 2. Open the Apache VirtualHosts configuration. In a typical XAMPP installation for Windows, it is located at C:\xampp\apache\conf\extra\httpd-vhosts.conf. It can also be opened from XAMPP Control Panel > Apache > Config > Apache (httpd-vhosts.conf).

Step 3. Add a block for CineVerse and replace the example path with the actual location of backend/public:

```

<VirtualHost *:80>
    ServerName cineverse.local

    DocumentRoot "C:/path/to/project/cineverse/backend/public"

    <Directory "C:/path/to/project/cineverse/backend/public">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
  
```

```

<VirtualHost *:80>

    ServerName cineverse.local

    DocumentRoot "C:/Users/PC/Desktop/project/cineverse/cineverse/backend/public"

    # Header always set Access-Control-Allow-Origin "*"
    # Header always set Access-Control-Allow-Methods "GET, POST, PUT, DELETE, OPTIONS"
    # Header always set Access-Control-Allow-Headers "*"

    <Directory "C:/Users/PC/Desktop/project/cineverse/cineverse/backend/public">
        AllowOverride All
        Require all granted
    </Directory>

</VirtualHost>

```

Actual CineVerse VirtualHost in the development environment used for these screenshots.

Note about the screenshot: the visible Header always set lines beginning with # are commented out, so they are inactive and are not part of the required VirtualHost configuration. CineVerse handles CORS in backend/core/middleware/CorsMiddleware.php; they do not need to be uncommented to follow this guide.

The path visible in the screenshot belongs to the computer where CineVerse was developed. It must not be copied literally to another machine: each installation must point to its own backend/public folder.

ServerName defines the name Apache will receive. DocumentRoot indicates the public folder it may serve. Pointing only to backend/public prevents controllers, services, templates, storage, .env, and other internal files from being exposed directly. AllowOverride All allows .htaccess to apply its rules, and Require all granted authorizes Apache to serve that folder.

Step 4. Check in C:\xampp\apache\conf\httpd.conf that Apache loads the VirtualHosts file and has mod_rewrite enabled:

```

Include conf/extra/httpd-vhosts.conf
LoadModule rewrite_module modules/mod_rewrite.so

```

If either of those lines is commented out with #, Apache will not use that part of the configuration until it is enabled and Apache is restarted. mod_rewrite is required so the rules in backend/public/.htaccess can send API routes to public/index.php.

Step 5. Save the changes. After modifying httpd-vhosts.conf or httpd.conf, Apache must be restarted so it reloads the configuration.

31.2 Configuring cineverse.local in the Windows hosts file

cineverse.local is a local name used only during development. It does not exist on the Internet and is not registered in public DNS. Windows needs to know that this name should resolve to the local computer.

The address 127.0.0.1 represents the local machine, also called localhost. The hosts file makes it possible to manually associate a name with an IP address before consulting a DNS server.

Step 1. Open Notepad as administrator. This is necessary because hosts is a Windows-protected file.

Step 2. Choose File > Open and navigate to C:\Windows\System32\drivers\etc. If the file does not appear, change the filter from Text Documents to All Files.

Step 3. Open hosts and add the association used by CineVerse at the end:

```

127.0.0.1 cineverse.local

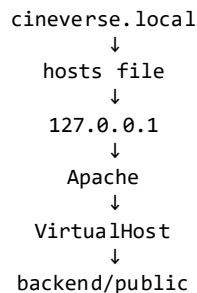
```



Windows hosts file associating cineverse.local with 127.0.0.1.

Step 4. Save the file. From that point on, Windows can resolve `cineverse.local` to the local computer.

The hosts file does not decide which application should respond or where the backend is located. Its only role is to convert `cineverse.local` into `127.0.0.1`. The `VirtualHost` from the previous section completes the path by telling Apache that this name corresponds to `backend/public`.



Modifying hosts does not by itself require Apache to be restarted. Apache must be restarted when its own configuration has changed, for example `httpd-vhosts.conf` or `httpd.conf`.

31.3 How CineVerse uses the .htaccess file

`backend/public/.htaccess` is already part of the project. The developer does not need to create it during installation. Its role is to allow API URLs to reach the single PHP entry point even though no physical file exists with the same name as the route.

For example, when Vue requests `http://cineverse.local/user`, there is no `backend/public/user` file or `user.php`. Apache must send that URL to `backend/public/index.php`, and from there the PHP Router decides which code to execute.

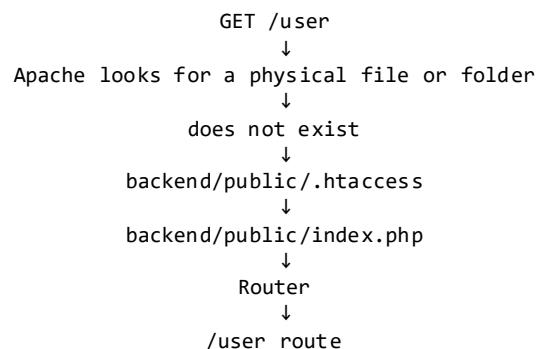
```

RewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d

RewriteRule ^ index.php [QSA,L]
  
```

`RewriteEngine On` enables the rewrite engine. The two conditions specify that the rule should apply only when the request does not correspond to an existing file (-f) or folder (-d). `RewriteRule` then internally sends the path to `index.php`, and QSA preserves the URL parameters.



The same mechanism allows `/login`, `/reviews`, or `/reviews/15` to reach the PHP Router without a separate file for each URL. For it to work, Apache must have `mod_rewrite` enabled and the `VirtualHost` must allow `AllowOverride All`.

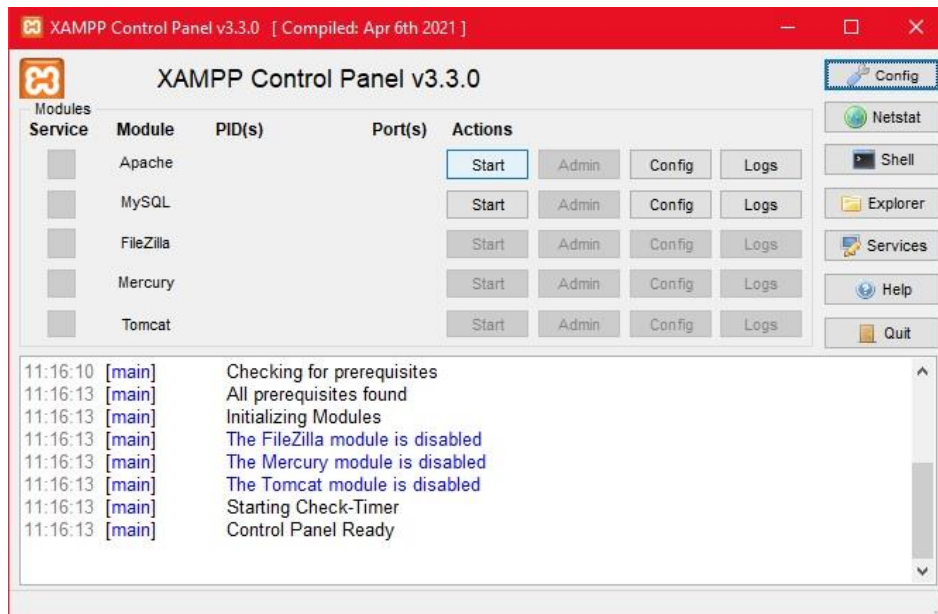
31.4 Starting the backend with XAMPP's Apache

In the documented development environment, the PHP backend is not started with `npm`, `Composer`, or a CineVerse-specific command. Apache serves `backend/public`, and the Apache bundled with XAMPP is currently used. If Apache is stopped, `cineverse.local` cannot reach the backend.

Step 1. Verify that the MySQL server specified by `DB_HOST` and `DB_PORT` in `backend/.env` is running and that the database specified by `DB_NAME` exists.

That MySQL server does not have to be XAMPP's MySQL module. CineVerse connects to the server defined by the `DB_*` variables. In this environment, XAMPP is used mainly for Apache.

Step 2. Open XAMPP Control Panel. In the Apache row, the Start button should appear if the server is still stopped.



XAMPP Control Panel before starting Apache.

Step 3. Press Start in the Apache row. When it starts correctly, XAMPP shows the processes and ports in use and changes the module's visual status.



Before: Apache stopped and Start button available.



After: Apache active and listening on the configured ports.

Step 4. With Apache active, test a backend route. If the `VirtualHost` and `hosts` configuration is correct, `http://cineverse.local/user` should reach the PHP code.

There is no need to run `php -S` when using this method. Apache continues serving `http://cineverse.local` and routes requests to `backend/public` while the module is running.

31.5 Starting the Vue frontend with Vite

The frontend runs differently. Apache keeps the PHP backend available, while Vue needs Vite to serve the project during development.

Step 1. Open the frontend folder in Visual Studio Code, as during installation, and choose `Terminal > New Terminal`.

Step 2. Verify that the terminal path ends in /frontend. If frontend was opened directly, there is no need to run cd frontend.

Step 3. Start the Vite development server:

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse/frontend (main)
$ npm run dev
```

npm run dev command executed from the frontend folder.

```
npm run dev
```

npm reads the dev script in package.json, whose value is vite. Vite then starts the development server. The terminal must remain open while working with the frontend.

Step 4. Wait for Vite to display the local address. In this project it normally appears as http://localhost:5173/. This is the URL opened in the browser to use CineVerse.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/cineverse/cineverse/frontend (main)

VITE v7.3.6 ready in 3657 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Vite started successfully and displaying http://localhost:5173/.

Important distinction: npm install installs or rebuilds the dependencies in node_modules; npm run dev starts Vite. These are two separate steps.

During normal development use, the browser opens localhost:5173. cineverse.local remains the backend address to which Vite forwards requests that begin with /api.

31.6 How an /api request travels from Vue to PHP

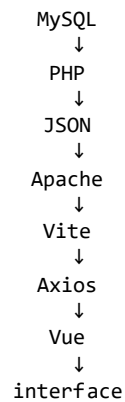
With frontend and backend running, a real request can be followed to understand how all the pieces connect. Suppose CineVerse needs to know the authenticated user and executes backendApi.get("/user").

backendApi.ts uses VITE_BACKEND_URL=/api as its baseUrl. Therefore, the browser, which is running the frontend at localhost:5173, first requests http://localhost:5173/api/user.

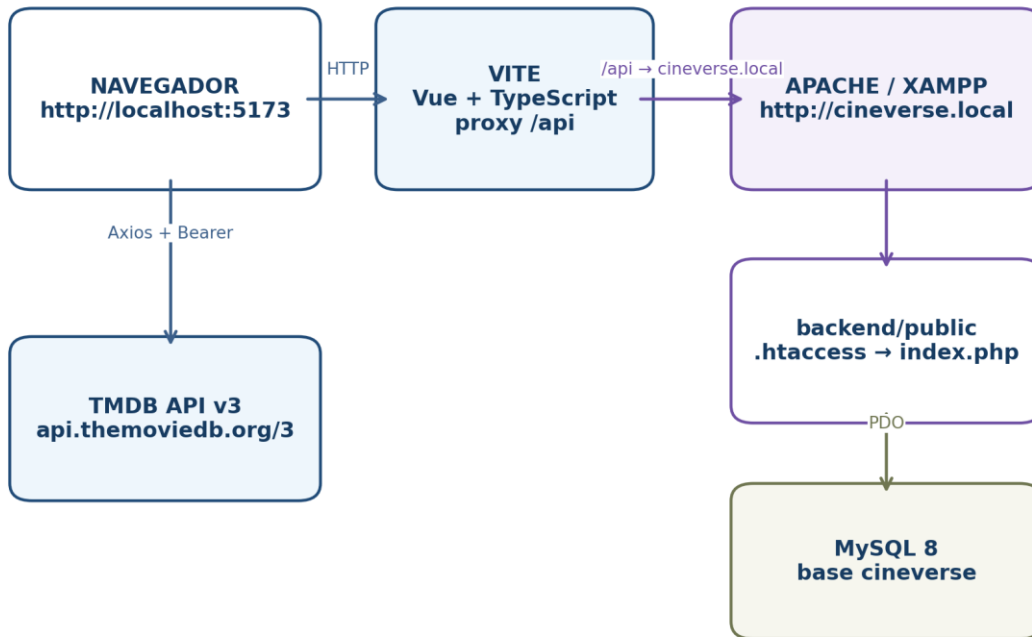
vite.config.ts contains a proxy rule for /api. Vite recognizes that prefix, removes it, and forwards the request to the configured backend target: http://cineverse.local. The resulting URL is http://cineverse.local/user.



The response travels back in the opposite direction: MySQL provides the data to PHP, the backend generates JSON, Apache returns the response, Vite delivers it to the browser, and Axios makes it available to the Vue code.



Ejecución en desarrollo



El usuario abre la SPA en localhost:5173. Vite reenvía /api al VirtualHost de Apache; el frontend consulta TMDB directamente.

Local execution flow: browser, Vite, Apache, backend, and MySQL.

TMDB does not use this flow. Movie and person requests leave the frontend through `tmdbApi.ts` directly toward the TMDB API; the `/api` proxy, Apache, and the PHP backend participate only when Vue needs CineVerse's own API.

31.7 Optional alternative: PHP's built-in server

PHP's built-in server can also be used for quick tests, but it is not the primary method documented for CineVerse. PHP does not interpret `.htaccess`, so `public/index.php` must be specified as the router when starting the server from the backend folder:

```
C:\xampp\php\php.exe -S 127.0.0.1:8000 -t public public/index.php
```

If the correct php executable is already available in PATH, `php -S 127.0.0.1:8000 -t public public/index.php` can be used. In this mode, the backend is available at `http://127.0.0.1:8000` and the Vite `/api` proxy target would need to point temporarily to that address instead of `cineverse.local`.

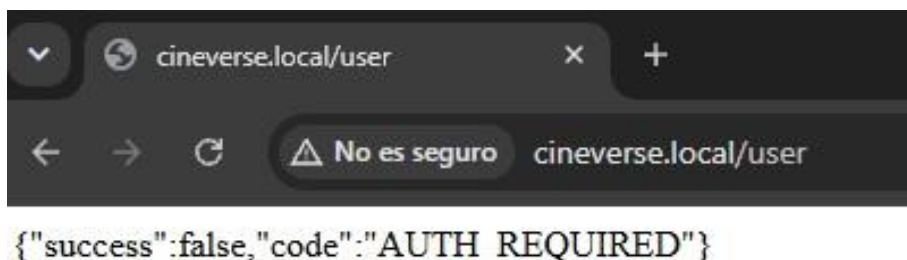
31.8 Final check and quick diagnosis

Once the configured MySQL server, Apache, and Vite are running, it is useful to check the environment piece by piece. This quickly identifies which component is failing if CineVerse does not work as expected.

Test	What it confirms	If it fails, check first
Open <code>http://localhost:5173</code>	Vite is serving the Vue frontend.	Vite terminal, <code>npm run dev</code> , and frontend errors.
Open Home/Movies or perform a search	The frontend can query TMDB.	<code>VITE_TMDB_BASE_URL</code> , <code>VITE_TMDB_TOKEN</code> , and Internet connection.

Test	What it confirms	If it fails, check first
Open <code>http://cineverse.local/user</code>	Windows resolves the name and Apache delivers the request to the PHP backend.	hosts, Apache, VirtualHost, mod_rewrite, and .htaccess.
Sign in	Sessions, backend, and MySQL connection work together.	DB_*, MySQL server status, and backend logs.
Create or view a review	The complete Vue -> PHP -> MySQL -> response flow works.	Reviews API, session, PDO, and database data.

One especially useful check is to open `http://cineverse.local/user` without signing in. The expected response is JSON with `AUTH_REQUIRED`. This result does not mean the installation is broken: it proves that the request correctly reached a protected backend route and that the system denied access because no authenticated session exists.



AUTH_REQUIRED response from /user without an authenticated session.

If, instead, the browser reports that `cineverse.local` does not exist, check hosts and Apache. If Apache returns a 404, check the VirtualHost, mod_rewrite, and .htaccess. If the backend responds with `DATABASE_UNAVAILABLE`, the request has already reached PHP and the problem is the connection to the configured MySQL server.

32. Publishing CineVerse on a server

Publishing the application means moving from the local development environment to a server accessible from the Internet. A clear arrangement is to serve the frontend from the main domain, for example `https://mydomain.com`, and the backend from a subdomain such as `https://api.mydomain.com`. The compiled frontend is served as static files, while PHP continues to run on the API server.

32.1 What the server must provide

The server must support PHP 8.1 or later, MySQL 8, HTTPS, URL rewriting, write access to storage/logs, and outbound SMTP. It must also be possible either to install Composer dependencies or to upload a vendor folder compatible with the PHP version used by the hosting environment.

VITE_* variables are incorporated into the JavaScript generated during compilation. For that reason, VITE_TMDB_TOKEN can be observed from the browser and must not be considered a server secret. A credential that truly needs to remain private would have to be used from a server-side API.

32.2 Building the frontend for production

Before generating the production build, the VITE_* variables are configured with the real addresses. If the backend is published on a subdomain, VITE_BACKEND_URL must contain the full URL of that API:

```
VITE_TMDB_BASE_URL=https://api.themoviedb.org/3
VITE_TMDB_TOKEN=<token_tmdb>
VITE_BACKEND_URL=https://api.midominio.com

cd frontend
npm install
npm run build
```

npm run build generates frontend/dist, which contains HTML, JavaScript, CSS, and the other files ready for deployment. Those files can be copied to the domain's public directory. Node.js does not need to remain running to serve Vite's final output.

32.3 Configuring the server for Vue Router routes

Vue Router uses normal URLs through createWebHistory. If the user opens /es/movies/550 directly or reloads that address, the frontend server must return index.html so Vue Router can decide which screen to display. Without that rule, Apache could respond with 404 before Vue has a chance to run.

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /index.html [L]
```

32.4 Publishing the PHP backend without exposing private files

The web server should not expose the entire PHP project. Whenever the hosting provider allows it, the API DocumentRoot should point only to backend/public. DocumentRoot is the folder the server considers accessible over HTTP; this keeps .env, logs, templates, services, models, and other internal files outside the public web root.

```
/home/user/cineverse-backend/
├── controllers/
├── core/
├── database/
├── models/
├── routes/
├── services/
├── storage/
├── templates/
├── validators/
├── vendor/
├── .env
└── public/          <- DocumentRoot for api.mydomain.com
```

32.4.1 Linux and PSR-4 compatibility

Windows normally does not distinguish uppercase and lowercase in folder names, whereas Linux does. CineVerse namespaces use names such as App\Core\Http and App\Core\Auth, but the current folders appear as core/http and core/auth. The observed mismatches are in auth, http, middleware, and validation. The core/logging folder does not have this problem: AppLogger.php currently declares namespace App\Core\logging. If it were normalized to App\Core\Logging, the folder would have to be renamed and the namespace and imports updated at the same time.

```
core/auth      -> core/Auth
core/http      -> core/Http
core/middleware -> core/Middleware
core/validation -> core/Validation
```

After correcting folder names, Composer autoloading should be regenerated with composer dump-autoload -o or composer install --optimize-autoloader. This updates the map Composer uses to locate classes.

32.5 Installing PHP dependencies in production

```
cd ~/cineverse-backend
composer install --no-dev --optimize-autoloader
```

If the hosting environment allows Composer, it is advisable to run composer install --no-dev --optimize-autoloader inside the backend. If it does not provide console or SSH access, meaning a remote terminal for running commands on the server, vendor can be prepared beforehand. In that case, verify that the dependencies are compatible with the PHP version on the destination server.

32.6 Creating and importing the MySQL database

On shared hosting, it is common to create the database and MySQL user first from the provider's administration panel, such as cPanel or Plesk. The schema is then imported. The current SQL file also contains test data and recreation statements, so it must not be treated as a harmless migration against a database containing real data.

If the provider requires a prefixed name, for example `user_cineverse`, `CREATE DATABASE cineverse` and `USE cineverse` must be adapted before importing. The backend `DB_NAME` variable must match the actual name created on the server.

32.7 Configuring the backend for production

```
APP_ENV=production
APP_DEBUG=false

DB_HOST=localhost
DB_PORT=3306
DB_NAME=user_cineverse
DB_USER=user_cv
DB_PASSWORD=<password>
DB_CHARSET=utf8mb4

CINEVERSE_LOG_PATH=storage/logs

MAIL_HOST=smtp.mydomain.com
MAIL_PORT=587
MAIL_USERNAME=no-reply@mydomain.com
MAIL_PASSWORD=<password>
MAIL_FROM=no-reply@mydomain.com
MAIL_FROM_NAME=CineVerse

FRONTEND_URL=https://mydomain.com
```

In production, `FRONTEND_URL` must contain the frontend's actual HTTPS address because MailService uses it to build verification and recovery links. `APP_DEBUG` should remain false. Passwords and other secrets remain only in the server's `.env` file.

32.8 CORS, HTTPS, and cookies

The current CORS configuration allows `http://localhost:5173` and must be adapted to the real frontend domain. A more maintainable approach would be to obtain that origin from `FRONTEND_URL` instead of hard-coding it in CorsMiddleware. Because the API uses cookies and `Access-Control-Allow-Credentials: true`, allowing any origin with `*` is not appropriate for this flow.

The frontend and backend should be published over HTTPS to encrypt traffic. Session enables the cookie's Secure option when PHP detects HTTPS. If the hosting environment uses an intermediary server (proxy) or CDN in front of PHP, verify that this information reaches PHP correctly so cookies are not created with an incorrect configuration.

32.9 Base path and URL structure

`public/index.php` contains `/cineverse/backend/public` as a fixed `basePath`. When the domain or subdomain already points to `backend/public`, a route such as `/login` does not contain that prefix and still works. If the API is actually published inside a different subfolder, that value should be reviewed or made configurable.

32.10 Logs and permissions

The user account under which PHP runs needs permission to write to `storage/logs`, but it does not need universal permissions. In production, use the minimum permissions that allow files to be created and rotated. The logs folder must remain outside the public root.

32.11 Verification after publishing the application

- Open the main domain and check /es and /en.
- Reload an internal route such as /es/movies/550 directly to verify the SPA fallback rule that returns index.html.
- Perform a search and a Discover request against TMDB.
- Register an account and verify receipt of the verification email.
- Sign in and out; check the cookie and CSRF on protected operations.
- Create, edit, and delete a review.
- Test password recovery and the links built with FRONTEND_URL.
- Confirm that a nonexistent API route returns JSON and that a nonexistent frontend route displays Vue's 404 screen.
- Confirm that storage/logs is writable and that technical details do not appear in the interface.

32.12 TMDB attribution and terms

TMDB states in its official documentation that the Developer API may be used free of charge for non-commercial purposes as long as TMDB is attributed as the source of the data and/or images. It also asks applications to identify their API use with the approved logo and display the corresponding attribution notice in an About or Credits section. If CineVerse is published, TMDB's current terms should be reviewed at that time, especially if the project becomes commercial.

Official reference: [TMDB FAQ - attribution and API use](#)

33. Project testing and verification

CineVerse is verified across several layers because no single test proves on its own that the entire system works. An action may look correct in Vue and yet not have been stored in MySQL; a PHP file may have valid syntax and still fail to connect to the database; and a successful build does not prove that authentication responds as expected. This is why this section combines functional testing, direct API testing, code analysis, and verification of persisted data.

Type of check	What it can demonstrate
Functional test	Manually verify from the interface that a CineVerse feature responds correctly when the user uses it. For example, sign in and verify that the application recognizes the authenticated user.
API test with Postman	Verify that the PHP backend receives requests and returns the expected responses without depending on the Vue interface.
Static analysis	Verify that PHP files have valid syntax and that TypeScript code respects the declared types.
Build	Verify that Vite can correctly generate the frontend production files.
MySQL verification	Verify that an operation performed from the application actually produced the expected change in the database.
Security check	Verify that the session, CSRF protection, and validators reject requests that do not satisfy the expected rules.
Automated tests	Automatically run a set of tests whenever needed to verify that project features continue behaving correctly after the code is modified. CineVerse does not currently include a complete suite of this kind.

33.1 Functional tests

Functional tests make it possible to manually verify that CineVerse's main features behave as expected when used through the interface. In this type of check, the result observed by the user matters: being able to sign in correctly, add or remove a movie from favorites, view movie information, or create, modify, and delete a review.

These tests are performed by using the application the same way a user would. For example, after signing in, you verify that CineVerse recognizes the authenticated account; when a movie is added to favorites, you verify that the change appears in the interface and remains after reloading the page; and when a review is published, you verify that it appears associated with the corresponding movie.

The functional tests described in this section are manual checks. They are not part of an automated suite and, on their own, do not allow the same checks to be rerun automatically after every code change. For that reason, they are complemented by direct API tests with Postman, static analysis, build verification, and control queries against MySQL.

33.2 Manual API tests with Postman

Postman is a tool for sending HTTP requests directly to an API and observing its response. In CineVerse, it makes it possible to test the PHP backend without going through Vue: choose the method, enter the URL, define the required JSON or headers, and inspect the HTTP status, cookies, and returned JSON body.

Why it is useful

If an operation works in Postman but fails from Vue, the problem is probably in the frontend integration. If it also fails in Postman, the Router, middleware, controllers, services, Models, or backend infrastructure can be investigated directly.

33.2.1 Login: verifying authentication and the backend response

The first test sends POST /login with an account prepared for the development environment. A successful response contains code = LOGIN_SUCCESS, the user object, and csrf_token. Postman also retains the session cookie PHP sends in the response. The values visible in the screenshot belong to the test dataset used to document the API.

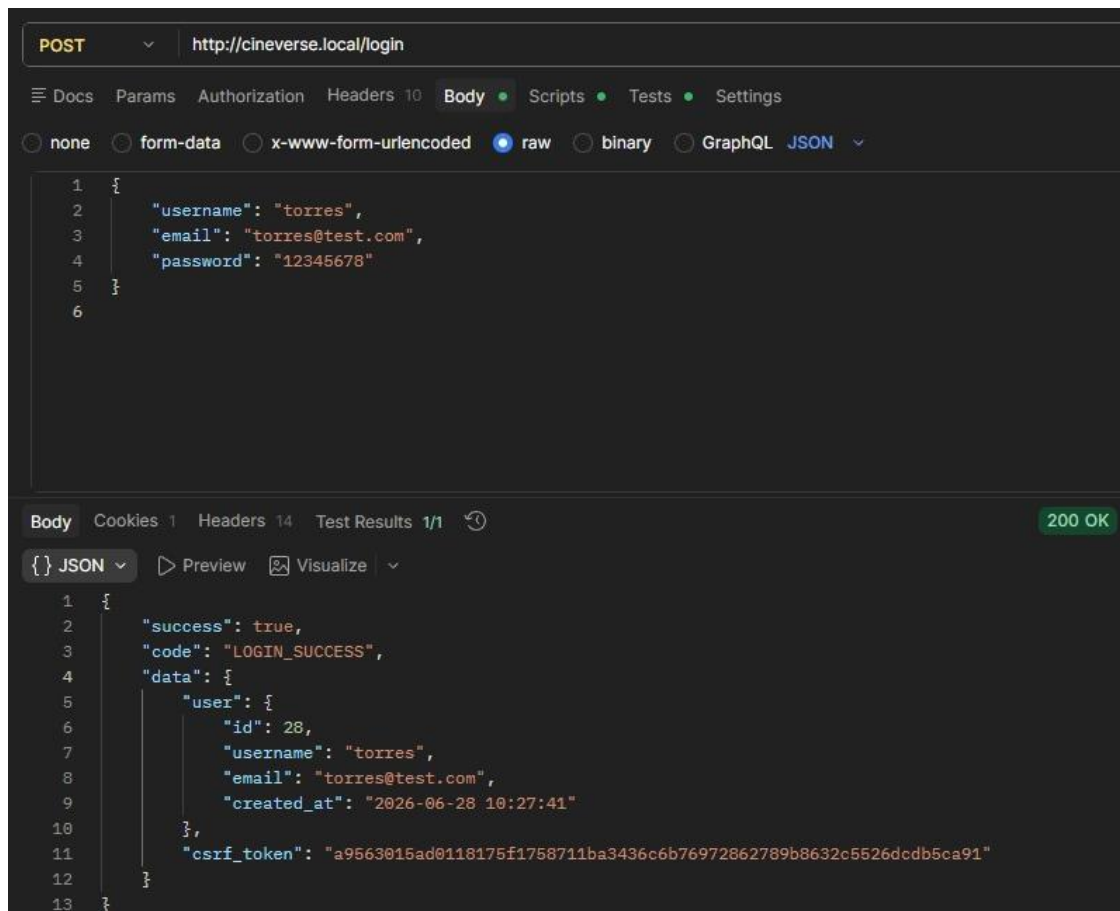


Figure. `POST /login` test in Postman: 200 OK response and `LOGIN_SUCCESS`.

What this test demonstrates: that the `POST /login` route reaches the backend, credentials can be validated against MySQL, PHP creates the session, and the backend returns the user together with the CSRF token that protected operations will use.

33.2.2 GET /user: verifying that the session remains active

After sign-in, `GET /user` verifies that a second request is still associated with the same user. Postman reuses the stored session cookie and the backend responds with `AUTHENTICATED_USER`. This test demonstrates that authentication does not depend on Pinia retaining the user: the session exists in PHP and can be reconstructed through the endpoint.

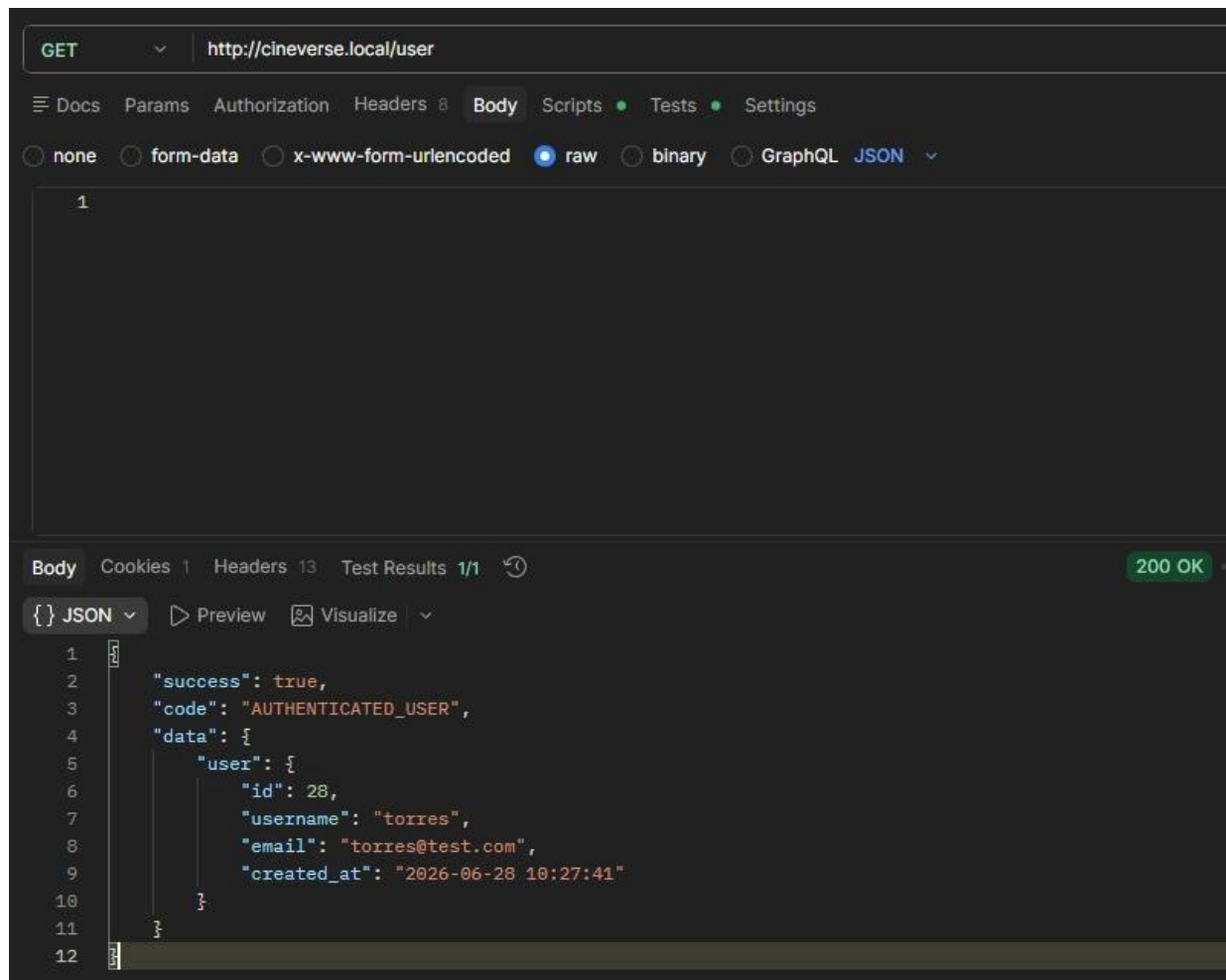


Figure. GET /user after sign-in: the backend recognizes the existing session.

What this test demonstrates: that the PHP session survives across independent requests and GET /user can reconstruct the authenticated user without depending on whatever state Pinia previously held in memory.

33.2.3 Reviews: statistics, pagination, and content

The GET /movies/550/reviews?page=1 request tests a public endpoint. The response includes stats, pagination, user_review, and reviews. page indicates the page received, and has_next reports whether later pages exist. user_review may be null when the current session has no review of its own for that movie.

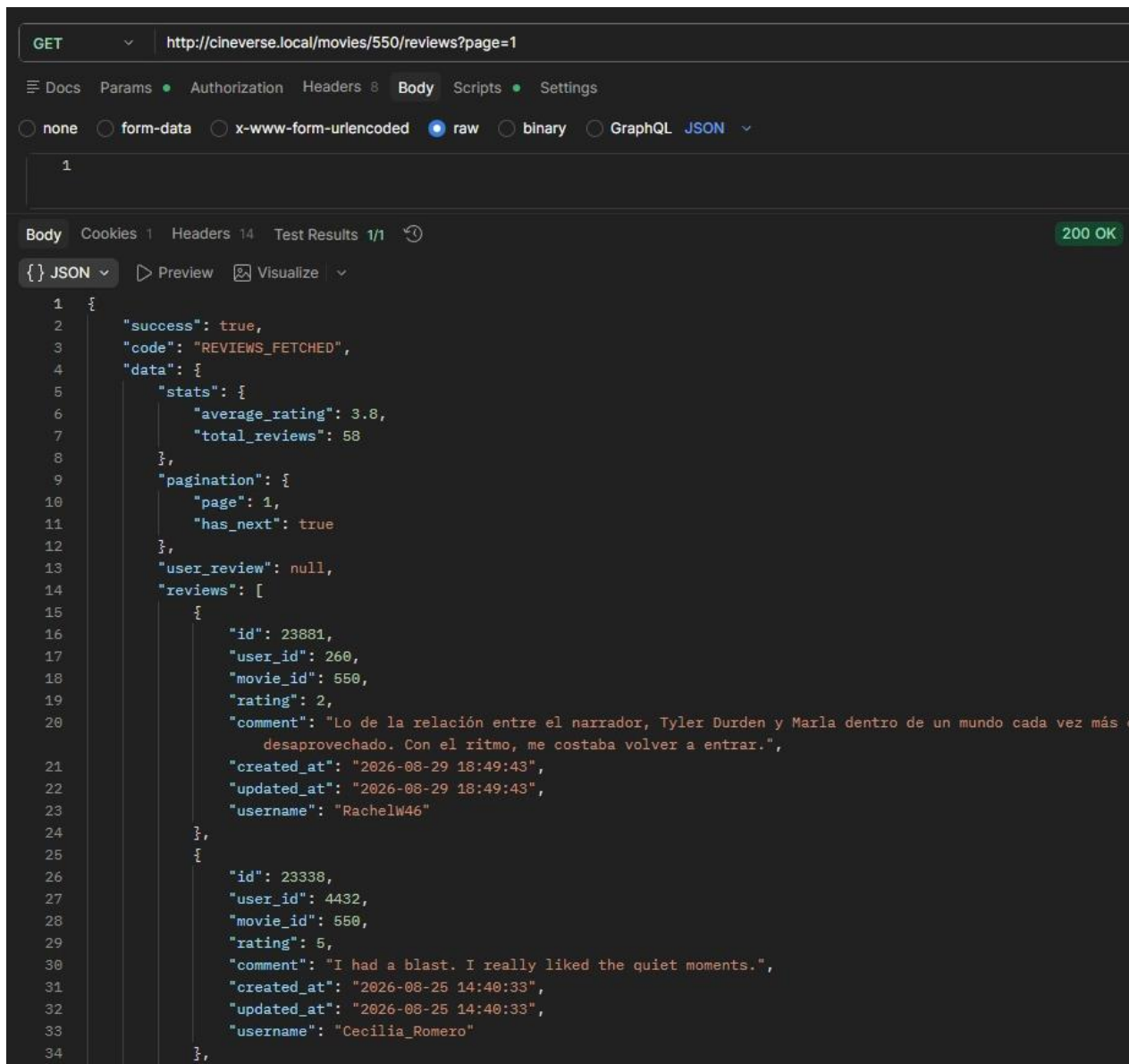


Figure. GET /movies/550/reviews?page=1: statistics, pagination, and first page of reviews.

What this test demonstrates: that a public endpoint can return statistics, pagination, and reviews directly from the backend and MySQL, and that `user_review` adapts to the available session without turning the route into a protected endpoint.

33.2.4 CSRF-protected logout

Operations that modify state require more than the session cookie. POST /logout sends X-CSRF-Token with the value obtained during authentication. If the session and token are valid, the backend responds with LOGOUT_SUCCESS. Omitting or altering the token should produce HTTP 403 + INVALID_CSRF_TOKEN.

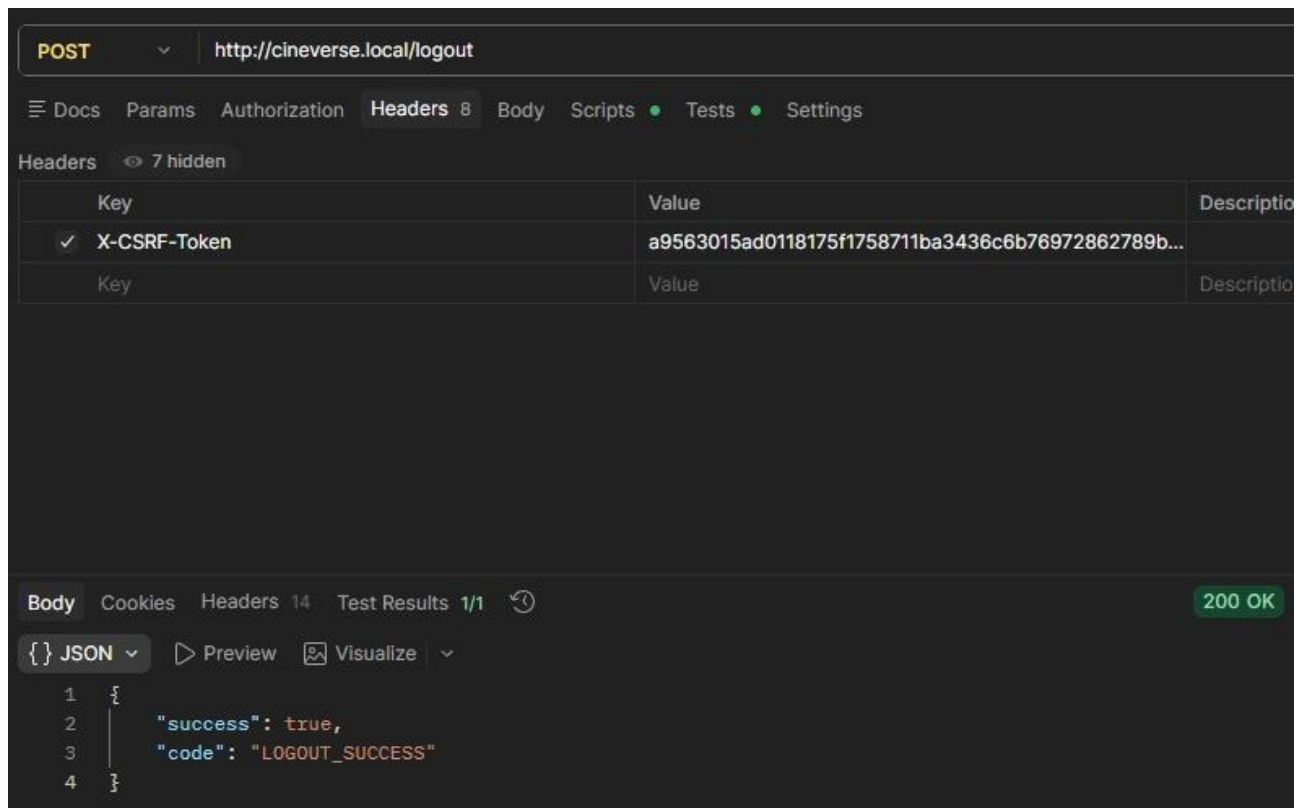


Figure. POST /logout protected by the X-CSRF-Token header.

What this test demonstrates: that a state-changing operation needs two different elements, the session cookie and the X-CSRF-Token. If the token does not match the value stored in the session, `CsrfMiddleware` must stop the request before executing logout.

33.2.5 Negative tests: sending invalid data

Postman can also demonstrate that the API rejects incorrect input. For example, sending `GET /movies/550/reviews?page=a` should return a controlled error with `code = MUST_BE_INTEGER`. This type of test verifies that validators act before an SQL query is built with an invalid value.

```
GET /movies/550/reviews?page=a
```

```
{
  "success": false,
  "code": "MUST_BE_INTEGER"
}
```

33.2.6 Testing TMDB independently

A useful test for separating frontend, application-backend, and external-service problems is to call TMDB directly with the same type of credential used by CineVerse. In Postman, choose `Authorization > Bearer Token` and send a detail request.

```
GET https://api.themoviedb.org/3/movie/550?language=es-ES
Authorization: Bearer <API Read Access Token>
Accept: application/json
```

An HTTP 200 response with JSON confirms that the credential and TMDB access work outside CineVerse. If this test works but the Vue screen fails, review `tmdbApi.ts`, the service, the composable, and result handling. If it also fails in Postman, the problem occurs before Vue: credentials, connectivity, parameters, or external-service availability.

33.3 Automatic code checks

These checks do not execute a complete functional flow, but they detect important problems before the application is tested manually.

33.3.1 PHP syntax

`php -l` analyzes a PHP file without running the application. It detects unmatched braces, syntax errors, and invalid statements. It does not check whether an SQL query is correct or whether MySQL is available.

```
php -l public/index.php
php -l controllers/AuthController.php
# repeat for the project's own PHP files
```

In the review performed on the provided code, the project's own PHP files pass this check without syntax errors.

33.3.2 TypeScript types

`vue-tsc` applies TypeScript checks to `.vue` components as well. It can detect type incompatibilities, nonexistent properties, and other static errors before the application is opened in the browser.

```
cd frontend
node node_modules/vue-tsc/bin/vue-tsc.js --noEmit
```

The current frontend review completes this check successfully. The result confirms static consistency, but does not validate at runtime that TMDB or the backend sent exactly the declared structure.

33.3.3 Frontend production build

`npm run build` verifies that Vite can generate the deployable frontend build. For the check to be reproducible, dependencies should be installed from `package.json` and `package-lock.json` on the system where the build is run, rather than reusing a `node_modules` directory created on another operating system.

```
cd frontend
npm install
npm run build
```

33.4 Complete tests by feature

The following tests combine several layers and make it possible to observe not only the immediate response, but also its later effects.

33.4.1 Favorites and the 20-item limit

- Save a movie, verify that the counter increases, and reload with F5 to confirm that it remains present.
- Remove the favorite, reload again, and confirm that it is not restored.
- Save 20 movies and try to add a 21st; the store must respect `MAX_FAVORITES` and the interface must report the limit according to the component.

33.4.2 Creating, editing, deleting, and paginating reviews

- Create a review from the Reviews tab and confirm that `user_review`, the average, and the total change.
- Edit rating and comment and verify that `movie_id` and `movie_title` remain unchanged.
- Delete the review and confirm that the interface once again allows Write Review.
- Use a movie with more than one page of reviews, load the next page, and confirm that it is concatenated without duplicating items.

33.4.3 Session and CSRF

- Sign in, reload the SPA, and confirm that `GET /user` reconstructs `user` and `csrfToken` from the cookie.
- Log out and confirm that a `requiresAuth` route redirects to Login again.
- Repeat a protected write without `X-CSRF-Token` and verify HTTP 403 + `INVALID_CSRF_TOKEN`.

33.4.4 Email, TMDB, and logs

- Register an account with working SMTP and verify the language, link, and receipt of the verification email.
- Test an invalid SMTP configuration and confirm that the frontend receives a safe code while the technical detail remains in the logs.
- Search for a movie, switch ES/EN, and confirm that the TMDB request changes language.
- Trigger a controlled infrastructure error and verify the flow explained in section 29: AppLogger must record the technical detail in the daily file while the HTTP response avoids exposing that detail to the user.

33.5 Verifying in MySQL a review created from the interface

Seeing a review on screen does not by itself prove that it was persisted. After creating it from the interface, a direct query against reviews can be run to verify that the backend actually stored it and that the fields match what is displayed.

```
USE cineverse;

SELECT
    id, user_id, movie_id, movie_title,
    rating, comment, created_at, updated_at
FROM reviews
WHERE user_id = <user_id>
    AND movie_id = <movie_id>;
```

The result must correspond to the visible review. This test explicitly connects MovieReviews -> POST /reviews -> ReviewController -> ReviewService -> Review::create() -> PDO -> MySQL.

33.6 Current state of automated testing

The project does not currently include a complete automated test suite. The preceding checks combine static analysis, build verification, manual interface tests, direct API tests with Postman, and control queries against MySQL.

To add automated tests to the PHP backend, **PHPUnit**, a tool designed to write and run tests against PHP code, could be used. CineVerse does not currently include PHPUnit in backend/composer.json, so it would need to be installed as a development dependency:

```
composer require --dev phpunit/phpunit
```

Composer would add PHPUnit to the backend development dependencies. The PHPUnit version should be chosen according to the PHP version used by the development environment. Once installed, it could be run from vendor/bin/phpunit.

One possible structure would be to create a dedicated tests folder:



Figure. Proposed structure for automated backend tests.

In this proposal, the tests folder is divided into Unit and Integration. Unit would contain tests intended to check specific components in isolation, such as validators, while Integration would contain tests that verify several backend layers working together. This structure represents a possible future addition of PHPUnit and is not currently part of the CineVerse project.

Unit tests would check specific pieces of code in isolation. For example, they could verify that AuthValidator rejects an invalid email address or that ReviewValidator does not accept a rating outside the allowed range.

A simplified example with PHPUnit would be:

```
<?php

use InvalidArgumentException;
use PHPUnit\Framework\TestCase;
use App\Validators\ReviewValidator;

final class ReviewValidatorTest extends TestCase
{
    public function test_rejects_rating_out_of_range(): void
    {
        $this->expectException(InvalidArgumentException::class);
        $this->expectExceptionMessage("INVALID_RATING");

        ReviewValidator::validateCreate(
            550,
            "Fight Club",
            11,
            "Test comment long enough"
        );
    }
}
```

The test tells PHPUnit to expect an InvalidArgumentException with the message INVALID_RATING. It then executes ReviewValidator::validateCreate() with a rating of 11. If the validator accepted that value or produced a result different from the expected one, PHPUnit would mark the test as failed.

The tests could be run from the backend folder with:

```
vendor/bin/phpunit tests
```

With that command, PHPUnit would scan the tests folder and execute the test files it finds there, showing which ones passed and which ones failed.

Integration tests could also be added to check several layers working together. For example, a sign-in test could prepare a test user, execute the authentication flow, and verify that an incorrect password produces the expected result.

Conceptually:

```
Prepare test user
↓
execute sign-in with an incorrect password
↓
AuthController
↓
AuthService
↓
User / MySQL
↓
```

`comprobar INVALID_CREDENTIALS`

On the frontend, Vue- and TypeScript-specific tools such as **Vitest** could be added for unit and component tests. For example, a test could automatically verify that an explorer function correctly selects the `top Rated`, `discover`, or `text-search` path according to the received filters.

Therefore, as a future improvement, it would be advisable to add:

- PHPUnit on the backend for validators, services, and integration tests.
- Vitest on the frontend for functions, composables, and Vue logic.
- Specific tests for the main flows: authentication, reviews, and the movie explorer.

These tools would make it possible to automatically run the same checks after modifying the code and detect when a change causes a function that previously worked correctly to stop working.

Scope

Postman, `php -l`, `vue-tsc`, and `npm run build` check different kinds of problems. However, they do not replace an automated test suite capable of verifying, after every change, that existing functions continue to work correctly.